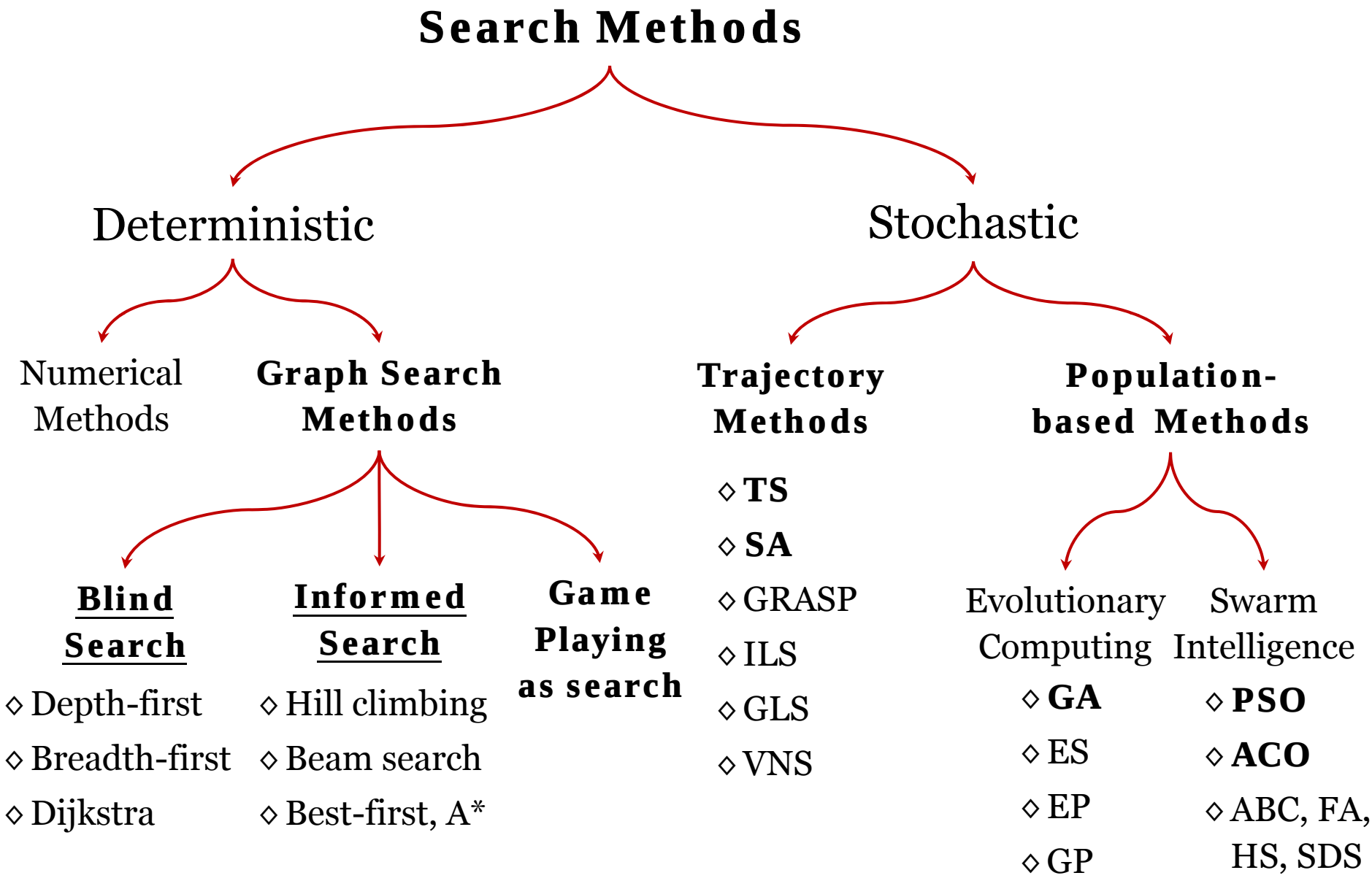
Graph Search Algorithms

Lecture 2 – Tuesday May 13, 2014

Outline


Search Methods
Deterministic
Stochastic
Numerical Methods
Graph Search Methods
Trajectory Methods
Population-based Methods
◇ TS
◇ SA
◇ GRASP
◇ ILS
◇ GLS
◇ VNS
Blind Search
◇ Depth-first
◇ Breadth-first
◇ Dijkstra
Informed Search
◇ Hill climbing
◇ Beam search
◇ Best-first, A*
Game Playing as search
Evolutionary Computing
◇ GA
◇ ES
◇ EP
◇ GP
Swarm Intelligence
◇ PSO
◇ ACO
◇ ABC, FA, HS, SDS

Outline

- Graph Search Methods
- Uninformed/Blind Search
- Informed Search
- Summary

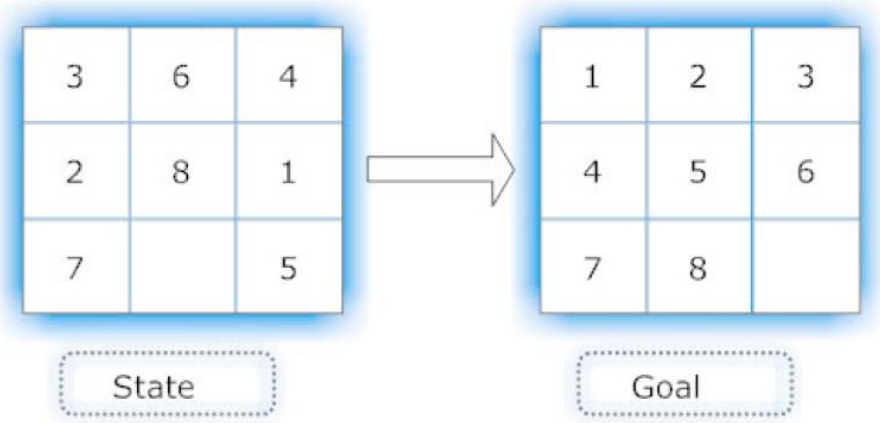
Outline

- **Graph Search Methods**
- Uninformed/Blind Search
- Informed Search
- Summary

Graph Search Methods

- **What is Search?**

  Search is the **systematic examination** of states to find a **path** from the **start state** to the **goal state**.



  - ◇ **States:** location of blank and location of the 8 tiles
  - ◇ O**perator (successor):** blank moves left, right, up and down
  - ◇ **Goal:** match the state given by the Goal state
  - ◇ **Path:** sequence through state space;
  - ◇ **Path Cost:** is the length of path.

Graph Search Methods

- **Why Search?**
  - ◇ To achieve goals or to **maximize our utility** we need to predict what the result of our actions in the future will be.
  - ◇ There are many **sequences of actions**, each with their own utility.
  - ◇ We want to find, or search for, the **best one**.

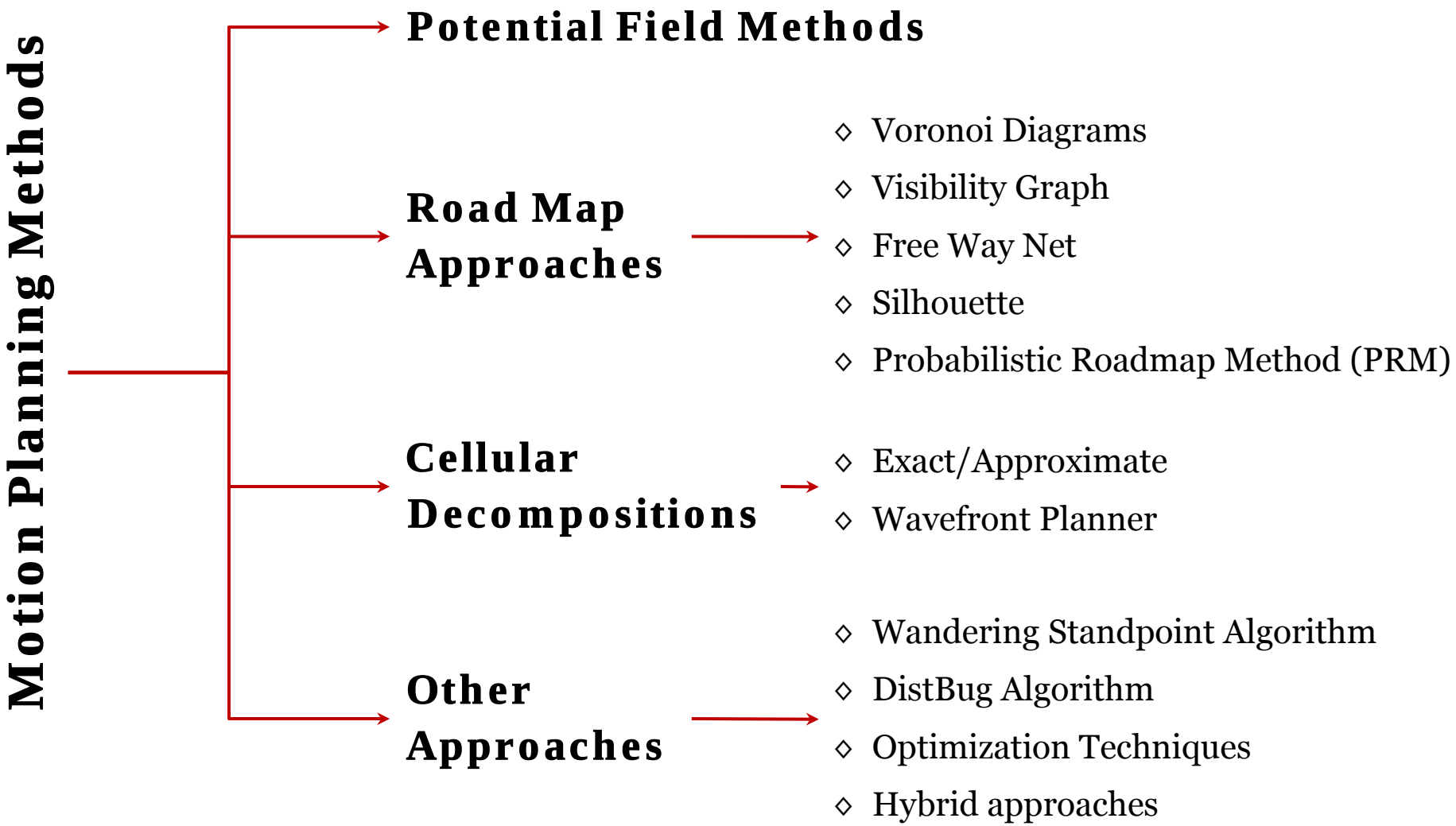
Graph Search Methods

- **Why Search? Motion Planning Problem**

Path planning is a process to compute a **collision-free path** for a robot from a start position (s) to a given goal position (G), amidst a collection of obstacles.

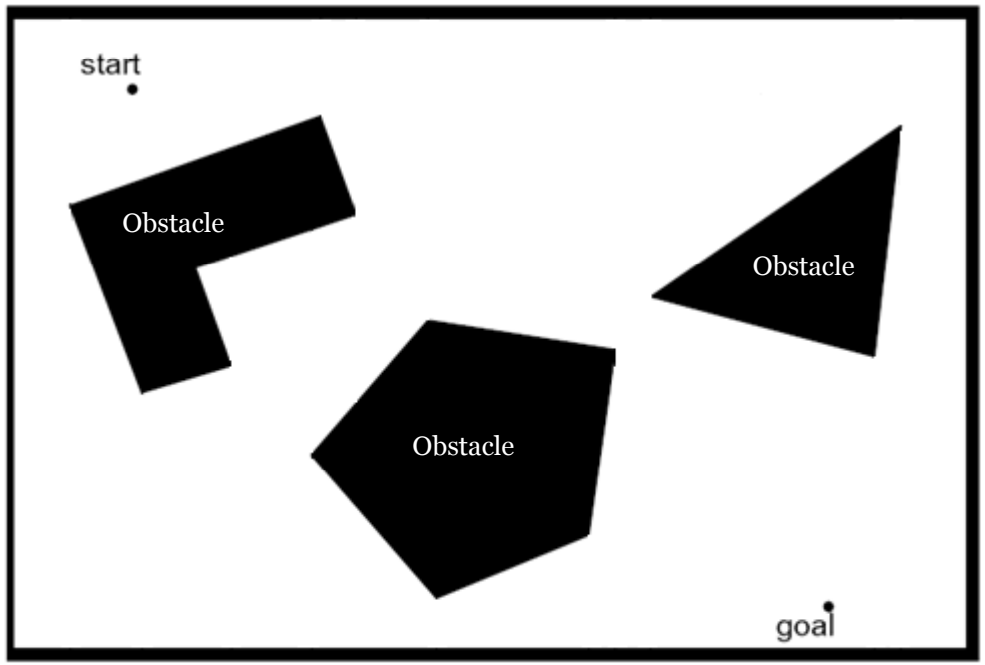
Graph Search Methods

- **Why Search? Motion Planning Problem**



Graph Search Methods

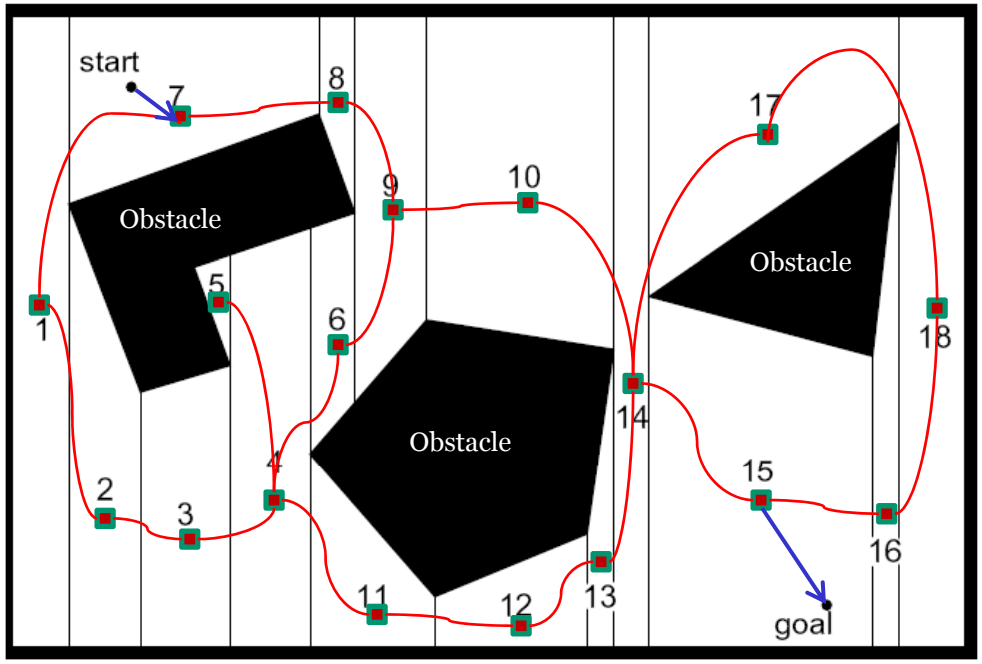
- **Why Search? Cell Decomposition Path Planning**

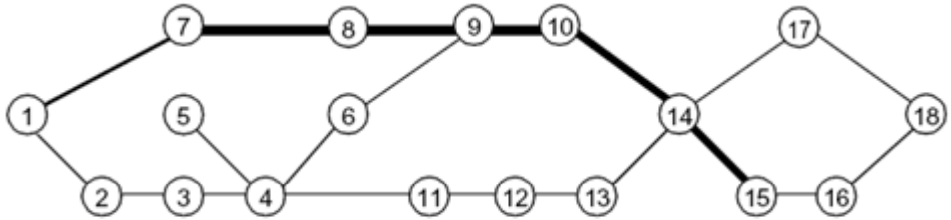


Working Environment

Graph Search Methods

- **Why Search? Cell Decomposition Path Planning**

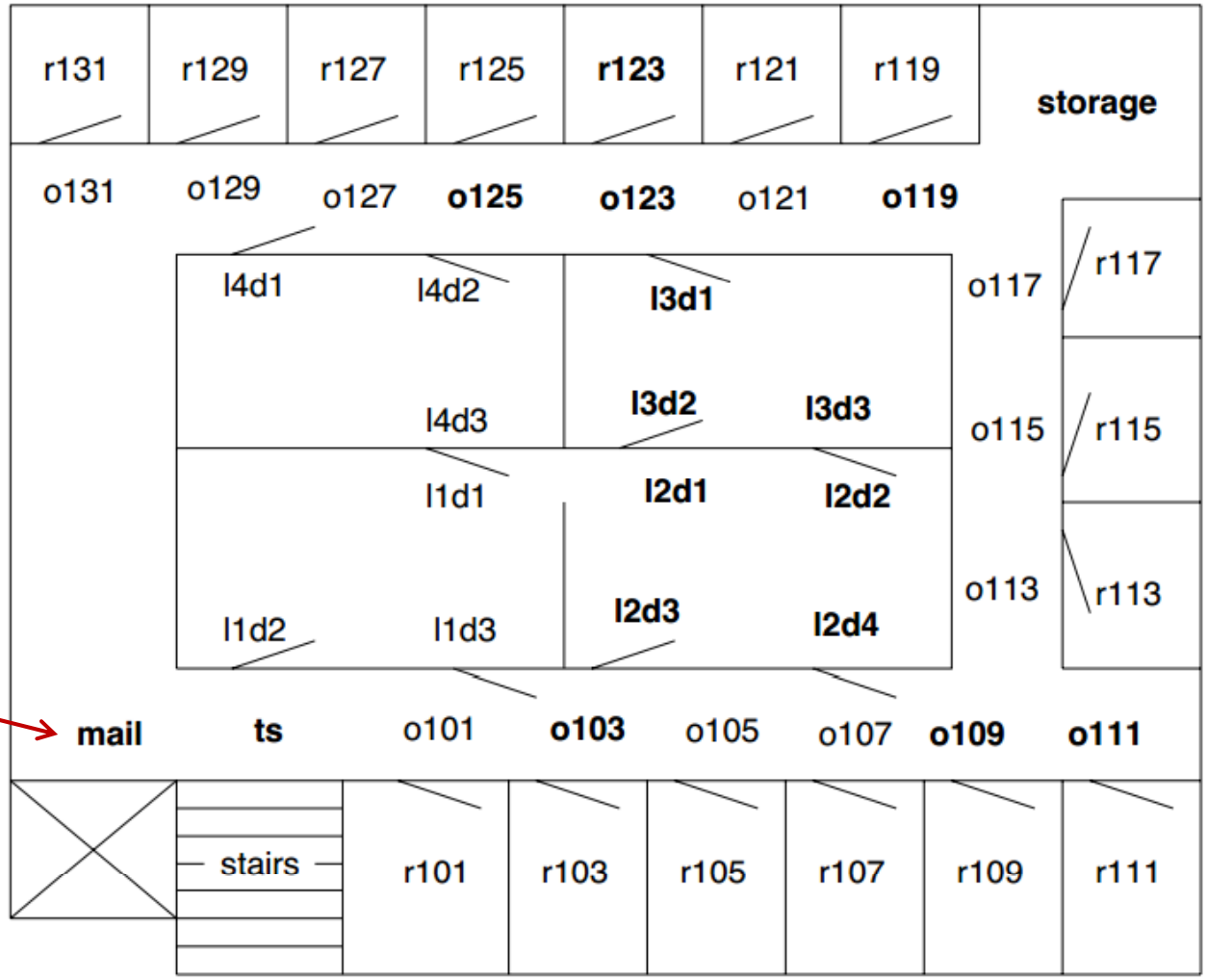




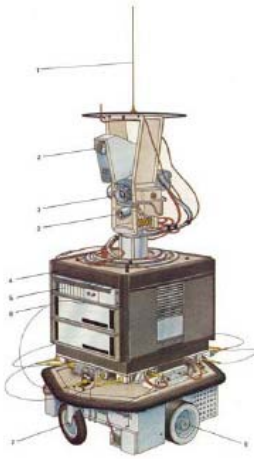
The connectivity graph of cells defines a roadmap

Graph Search Methods

- **Why Search? Topological Navigation**



Artificial or natural landmark

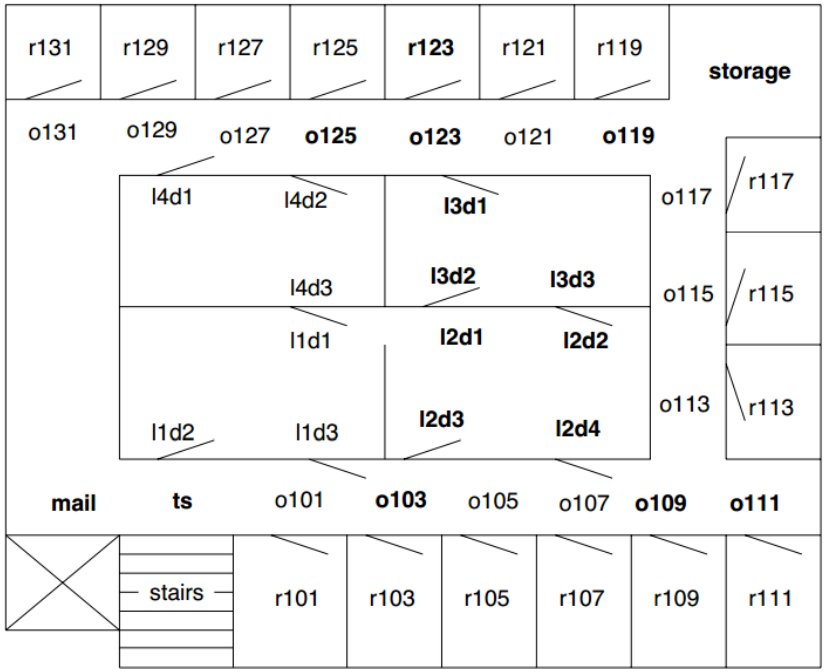


Shaky

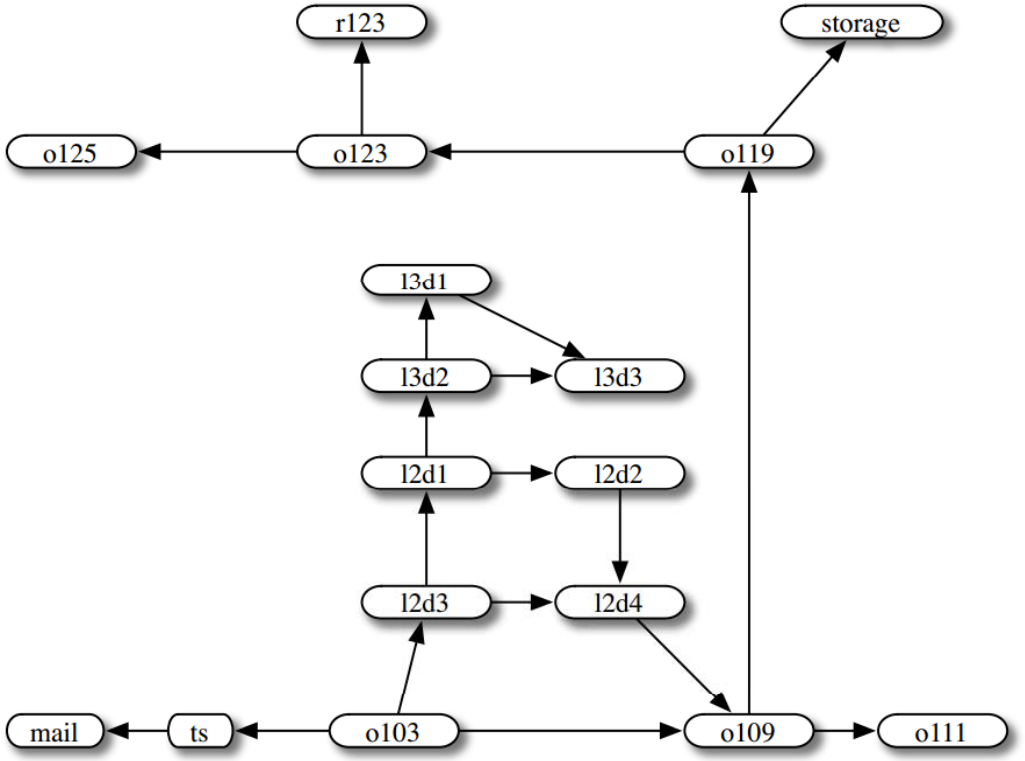
[5]

Graph Search Methods

- **Why Search? Topological Navigation**



The agent starts outside room 103, and wants to end up inside room 123



Mail Delivery Graph

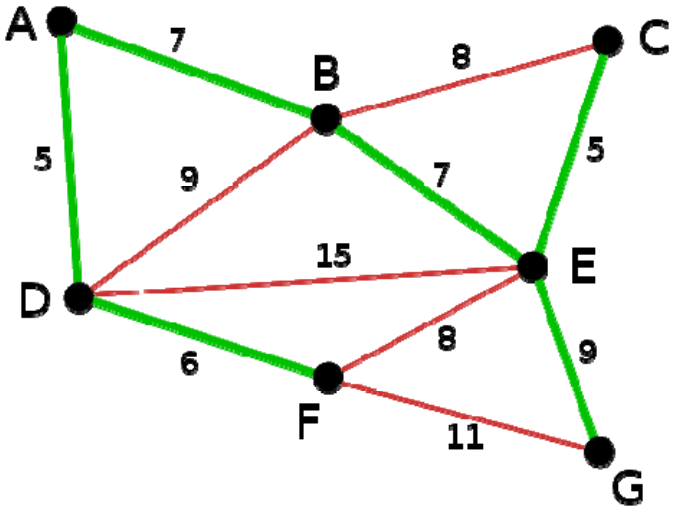
[5]

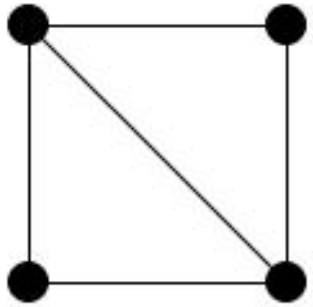
Graph Search Methods

- **Tree and Graph: What is the difference?**

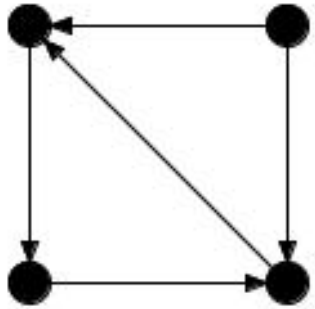
  A graph consists of 3 sets:

  - ◇ vertices,
  - ◇ edges and
  - ◇ a set representing relations between vertices and edges

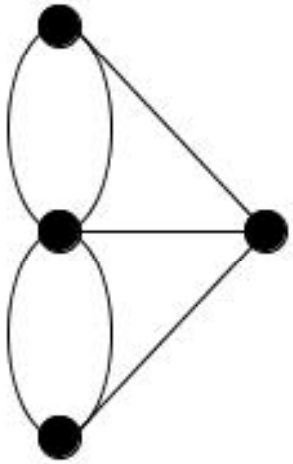




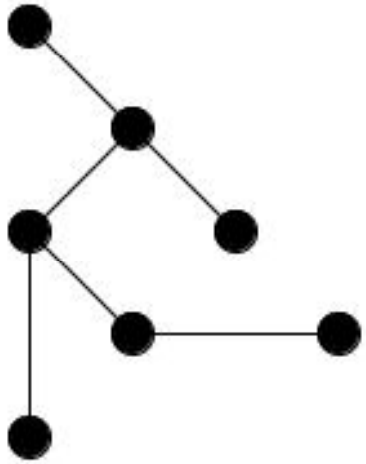
Undirected Graph



Directed Graph



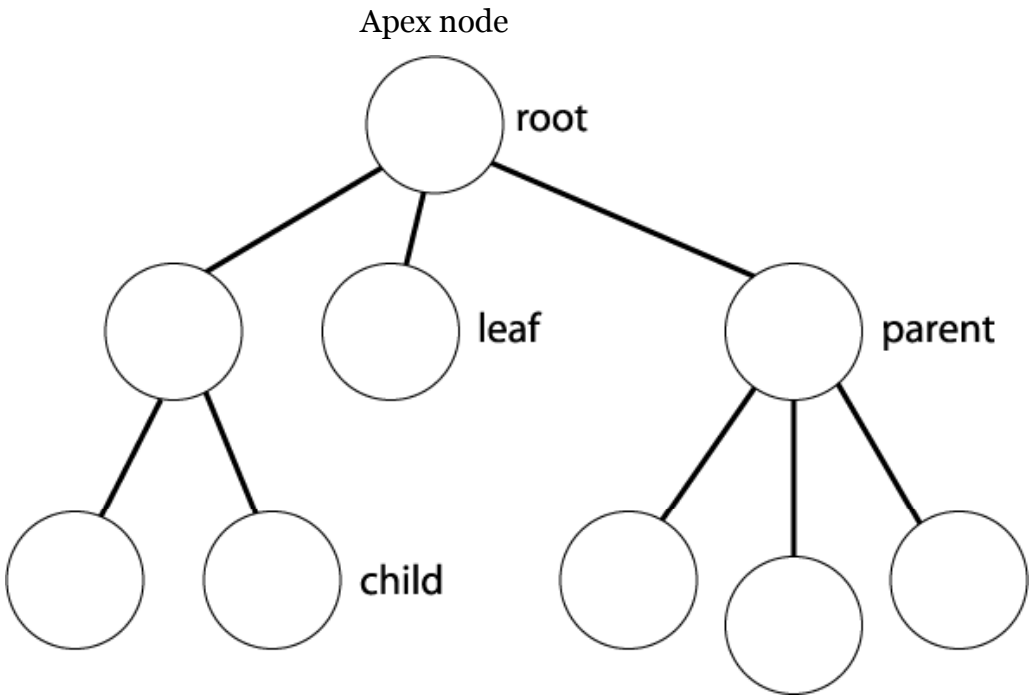
Multi-Graph



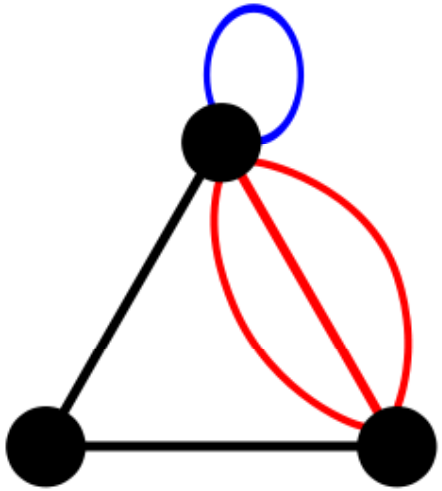
Acyclic Graph

Graph Search Methods

- **Tree and Graph: What is the difference?**
  - ◇ A tree is a specialized case of a graph.
  - ◇ A tree is a connected graph with **no circuits** and **no self loops.**



Tree



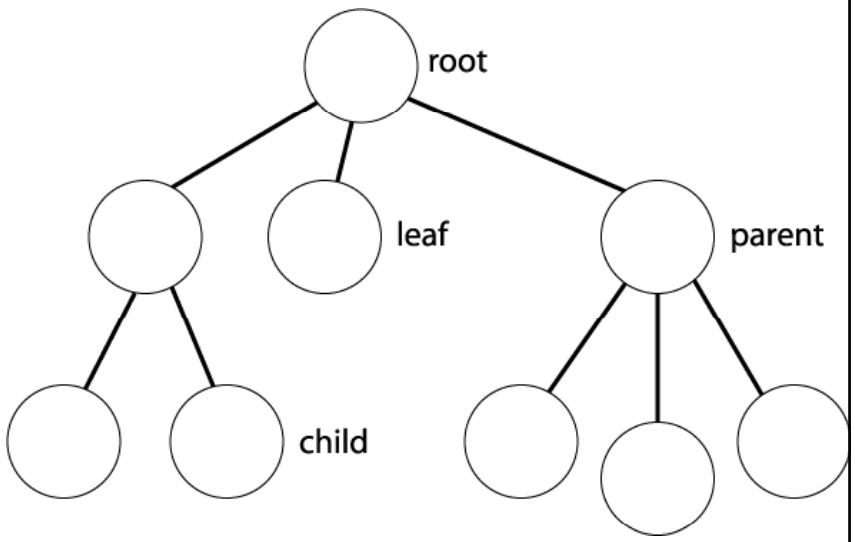
This is a graph not a tree

Graph Search Methods

- **Tree and Graph: What is the difference?**

  A **node** in the tree may be viewed as a **data structure** containing the following elements:

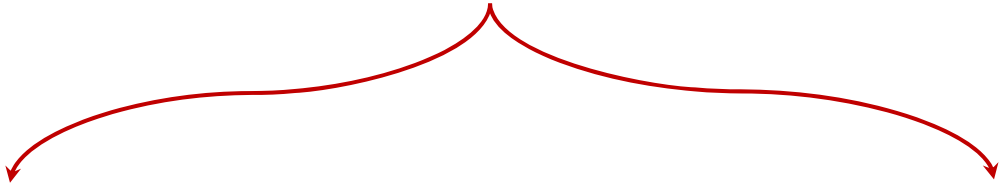
  - ◇ a state description;
  - ◇ a pointer to the parent of the node;
  - ◇ depth of the node;
  - ◇ the operator that generated this node;
  - ◇ cost of the path (sum of operator costs) obtained from the initial (start) state.



Graph Search Methods

Graph search methods are **deterministic optimization algorithms** that follow a rigorous procedure and their path and values of both design variables and the functions are repeatable. For the same starting point, these methods will follow the same path whether you run the program today or tomorrow.

Graph Search Methods/Enumerative Algorithms



Uninformed/Blind Search

- ◇ Depth-first
- ◇ Breadth-first
- ◇ Dijkstra, B&B

Informed Search

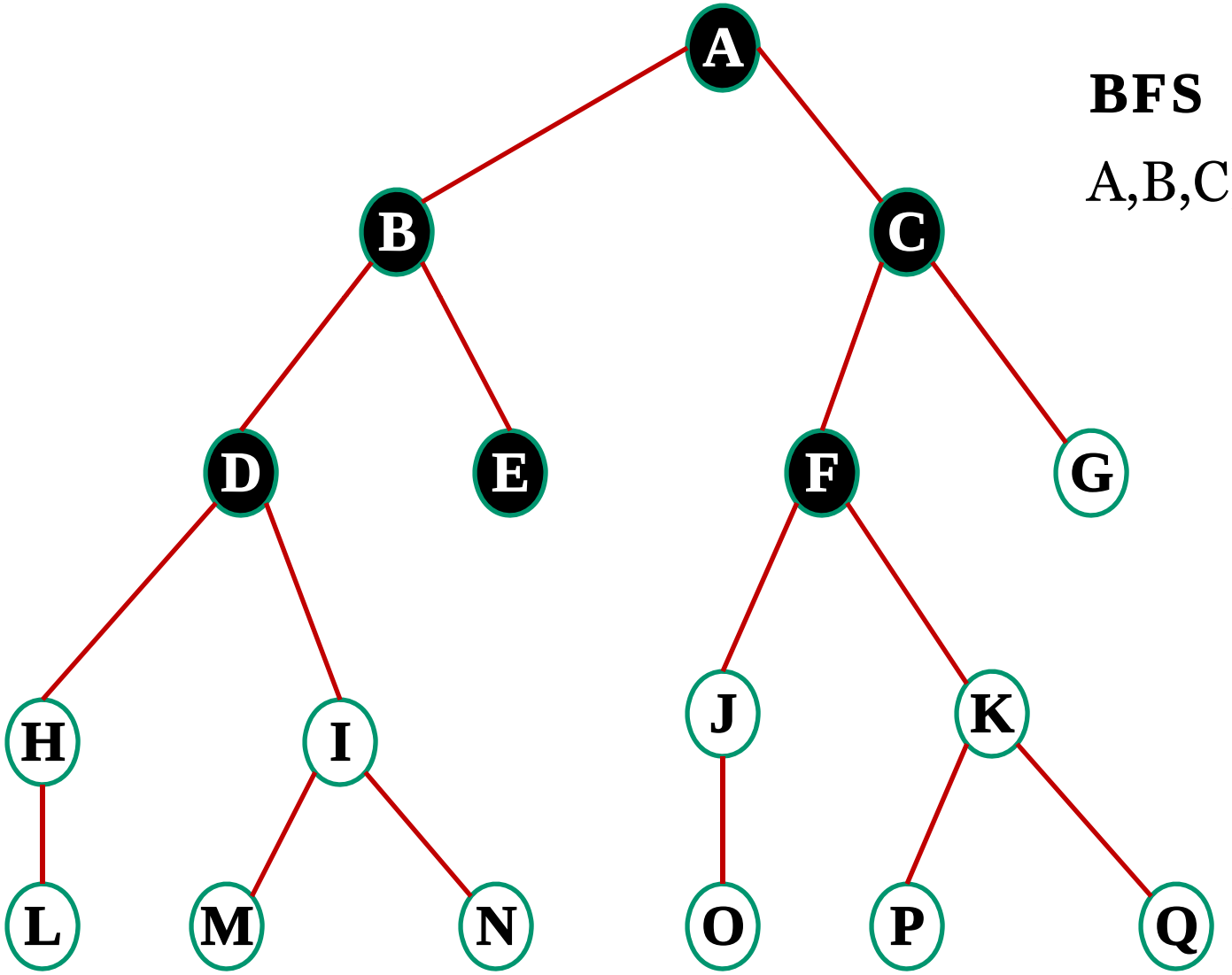
- ◇ Hill climbing
- ◇ Beam search
- ◇ Best-first, A*

Outline

- Graph Search Methods
- **Uninformed/Blind Search**
- Informed Search
- Summary

Uninformed/Blind Search

- **Breadth-first Search (BFS)**

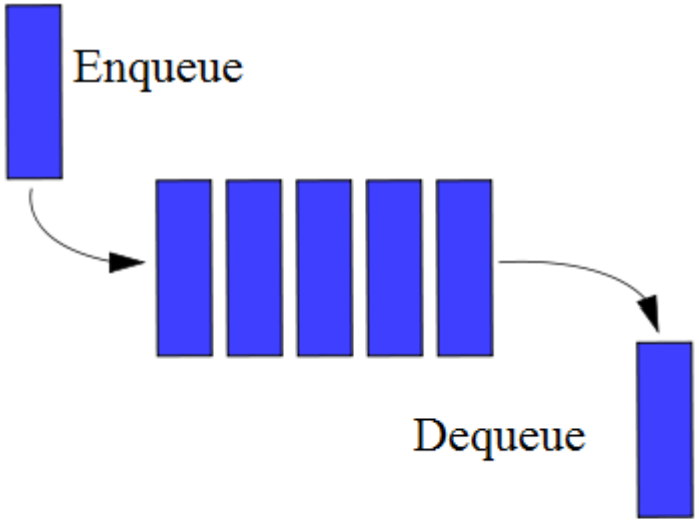


BFS

A,B,C,D,E,F,...

Uninformed/Blind Search

- **Motion Planning Problem: BFS**
  - ◇ BFS uses the **queue** as data structure.
  - ◇ Queue is a **First-In-First-Out (FIFO)** data structure.
  - ◇ A FIFO means that the node that has been sitting on the queue for the **longest amount of time** is the **next node to be expanded**.



Uninformed/Blind Search

- **Breadth-first**

Step 1. Form a queue Q and set it to the initial state (for example, the Root).

Step 2. Until the Q is empty or the goal state is found

do:

Step 2.1 Determine if the first element in the Q is the goal.

Step 2.2 If it is not

Step 2.2.1 remove the first element in Q.

Step 2.2.2 Apply the rule to generate new state(s) (successor states).

Step 2.2.3 If the new state is the goal state quit and return this state

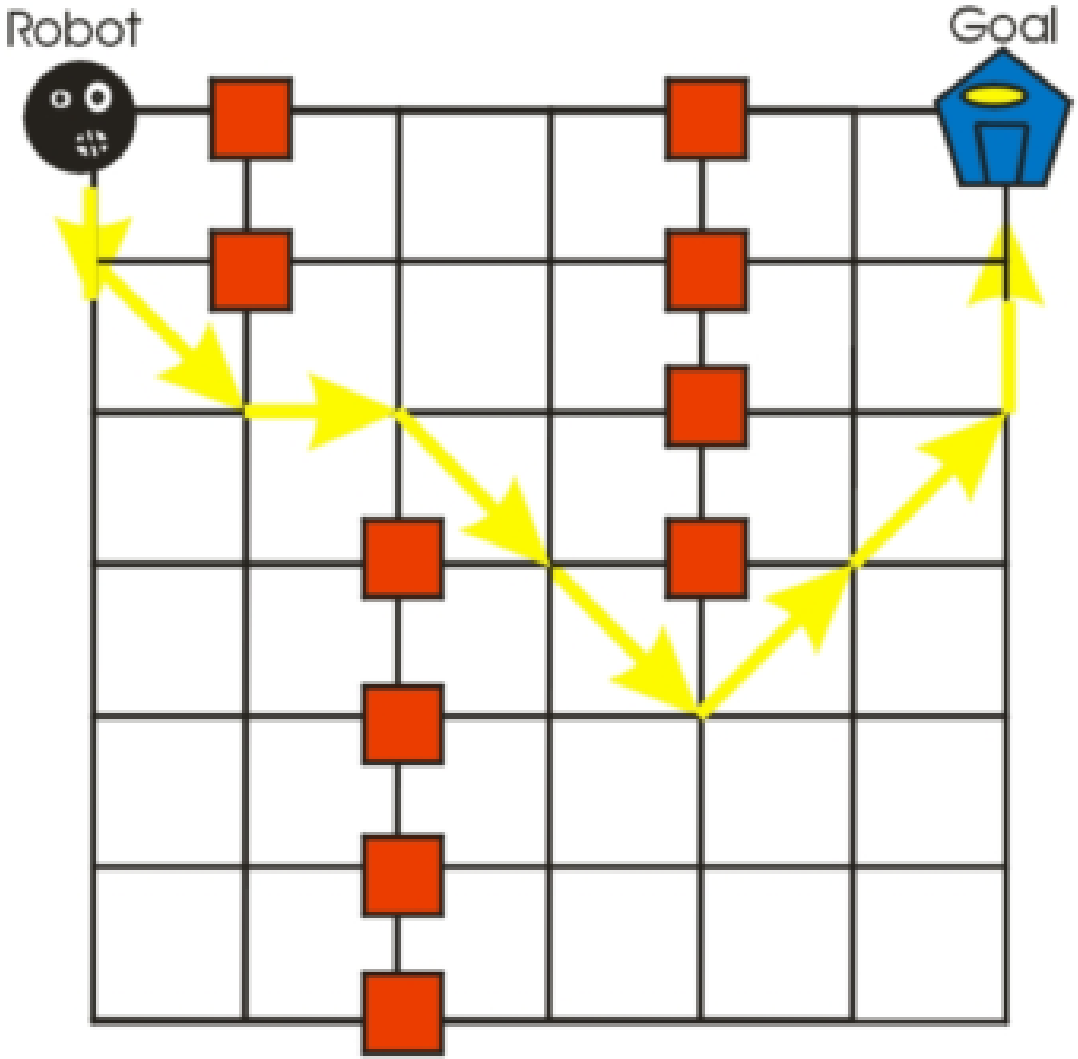
Step 2.2.4 Otherwise add the new state to the end of the queue.

Step 3. If the goal is reached, success; else failure.

[1]

Uninformed/Blind Search

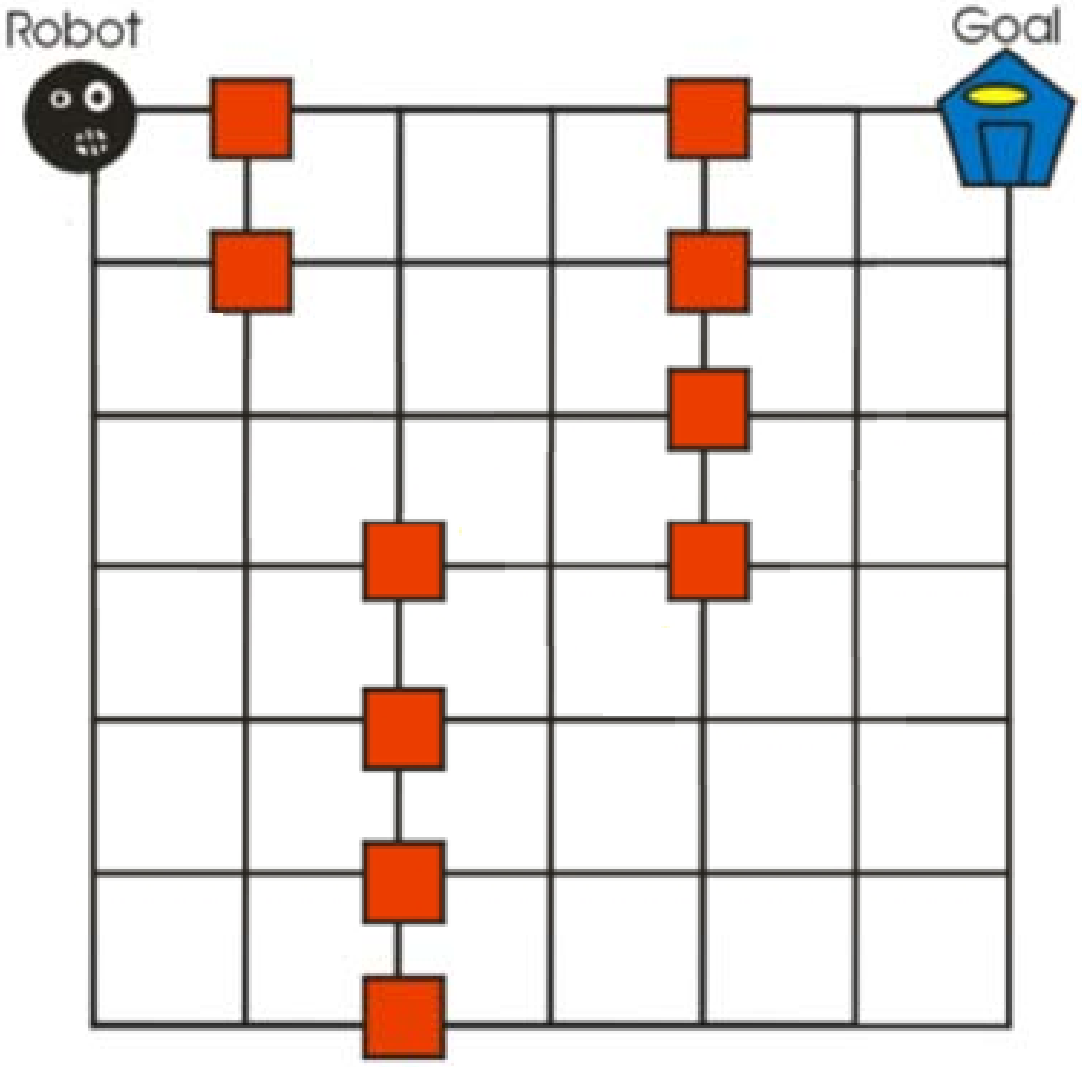
- **Motion Planning Problem**



[2]

Uninformed/Blind Search

- **Motion Planning Problem: BFS**



Start

Queue

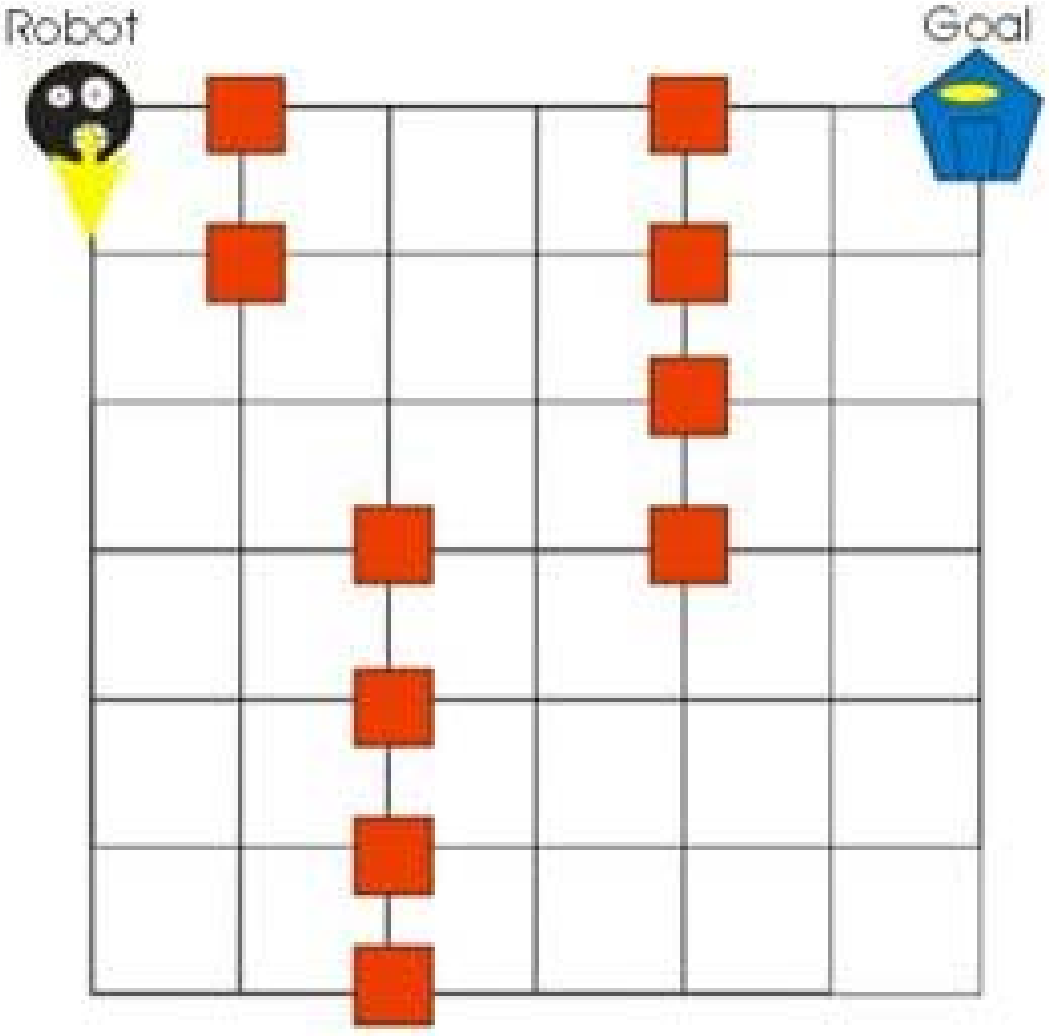
Start

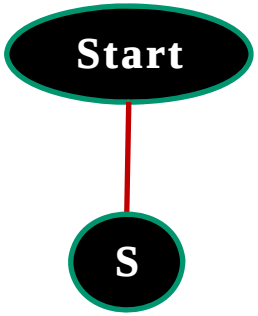
IN

OUT

Uninformed/Blind Search

- **Motion Planning Problem: BFS**





Queue

S

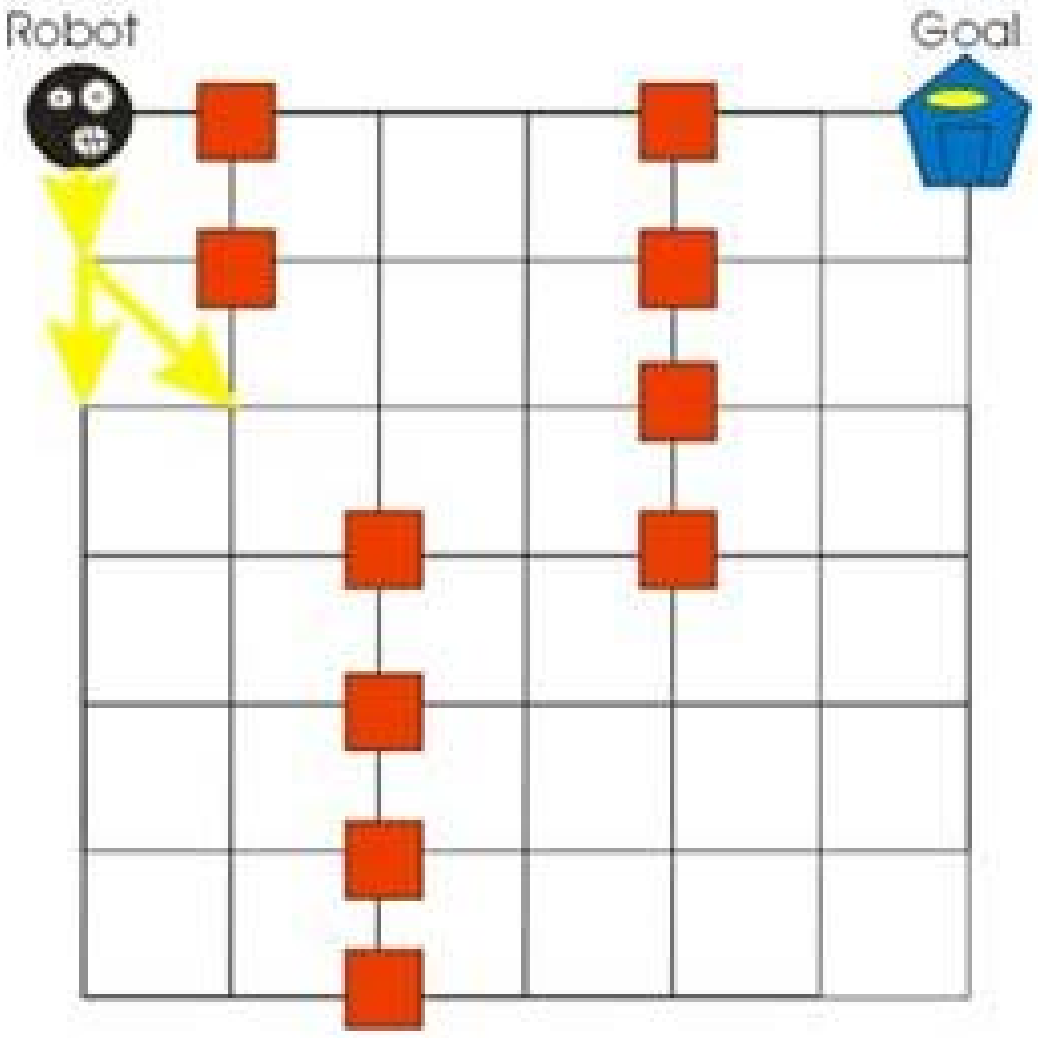
IN

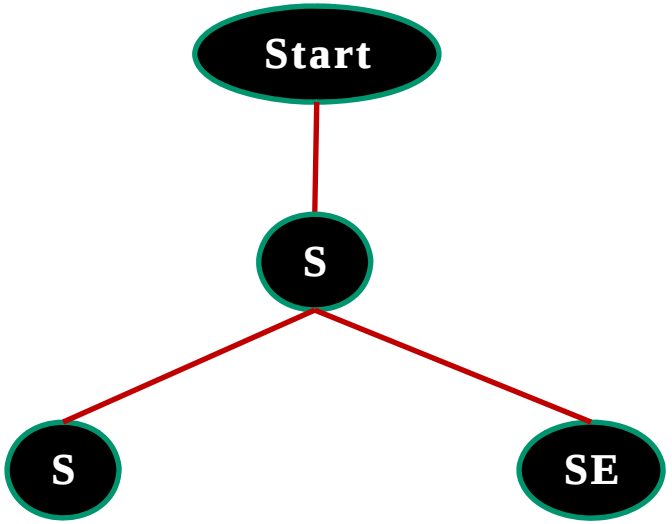
Start

OUT

Uninformed/Blind Search

- **Motion Planning Problem: BFS**





Queue

SE S

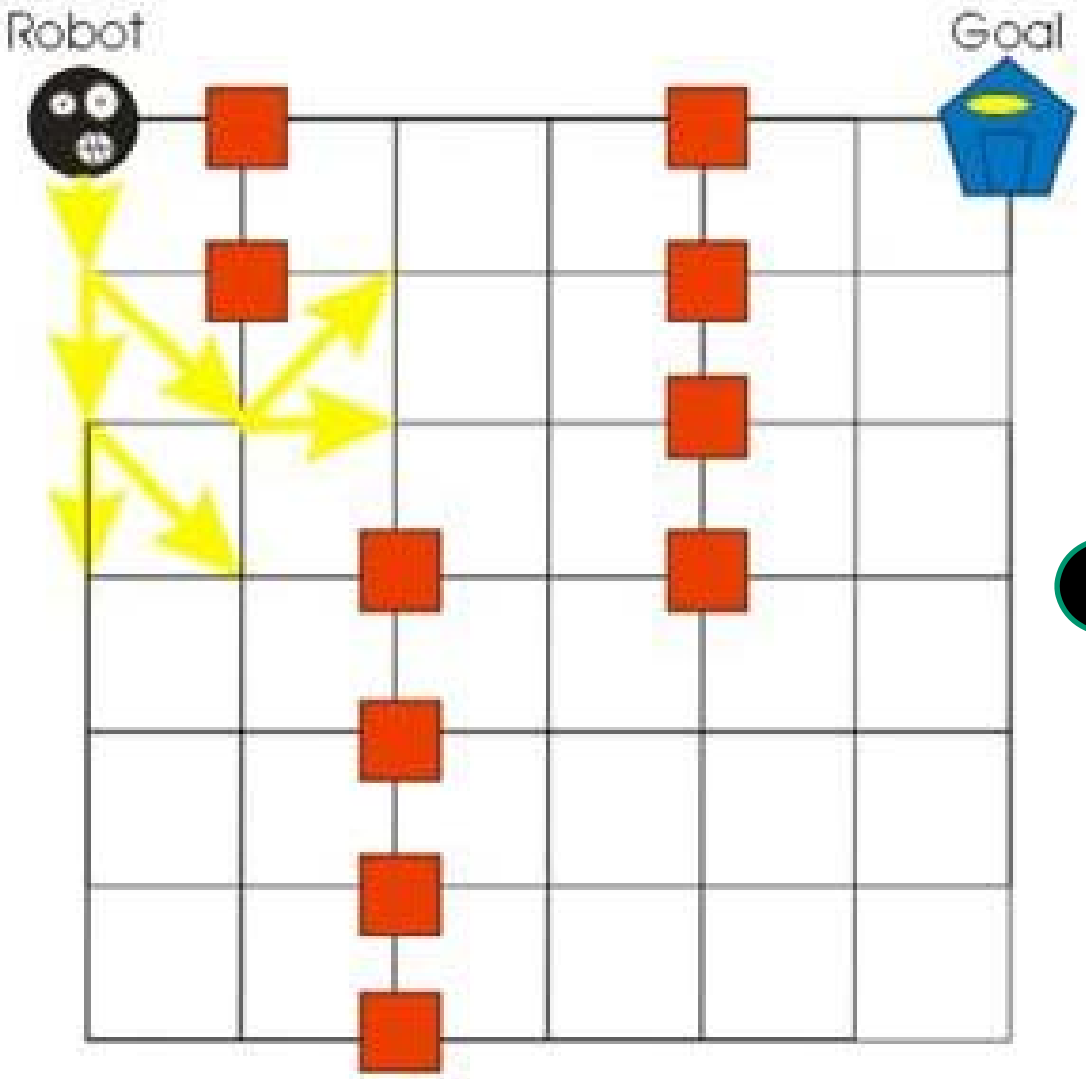
IN

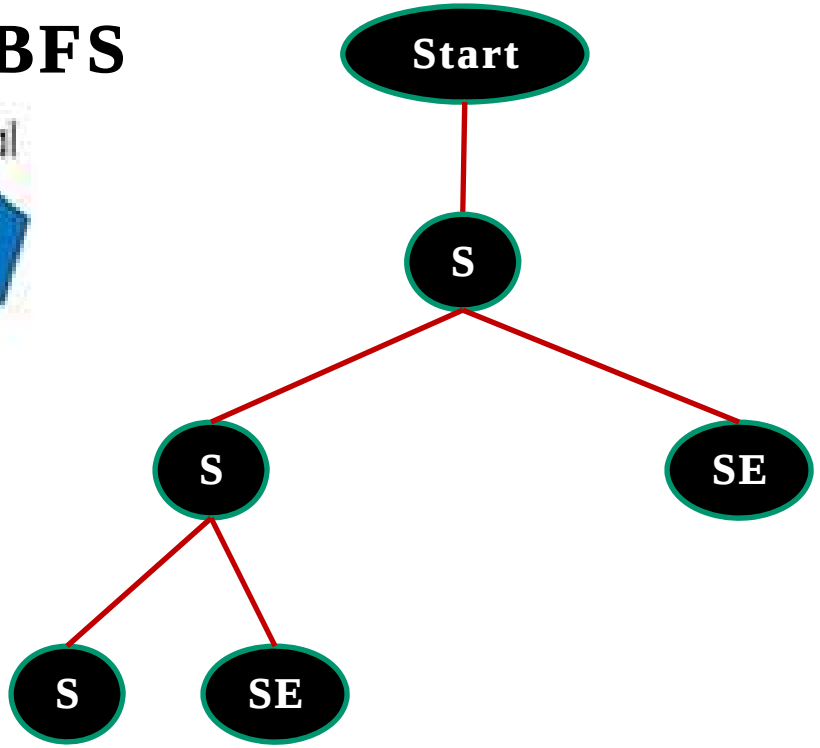
Start S

OUT

Uninformed/Blind Search

- **Motion Planning Problem: BFS**





Queue

SE S SE

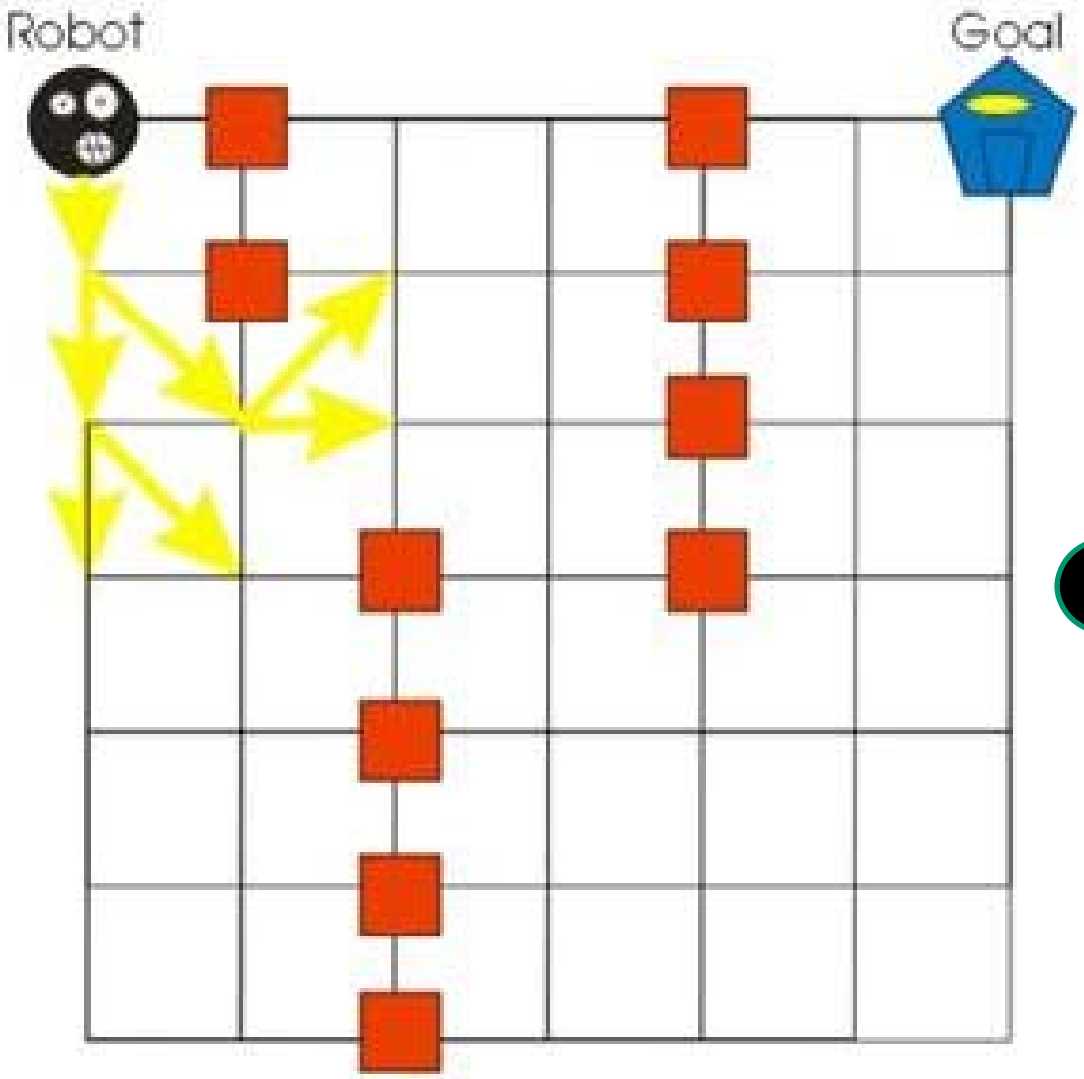
IN

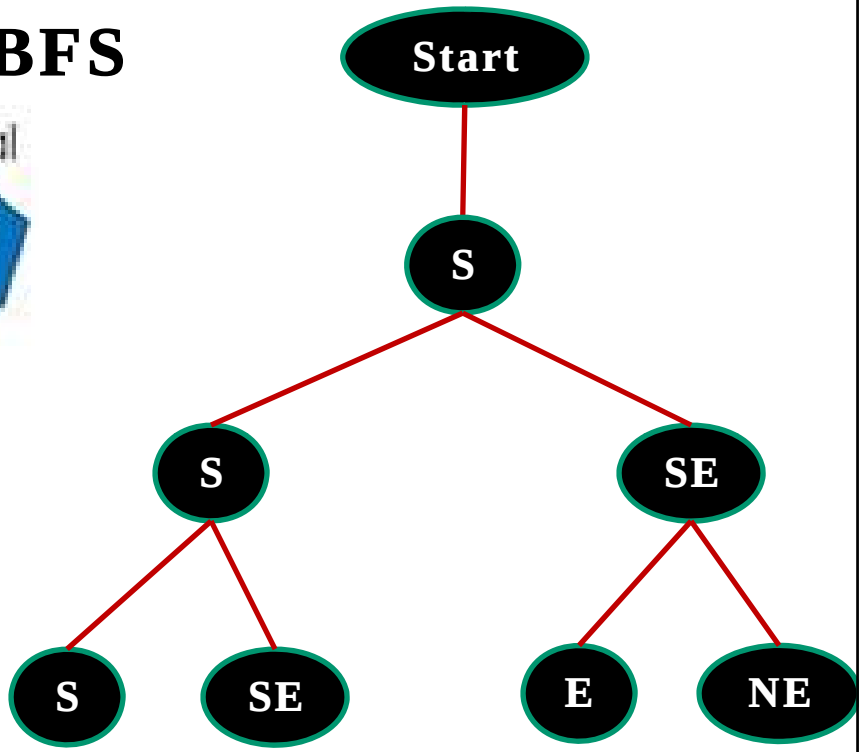
Start S S

OUT

Uninformed/Blind Search

- **Motion Planning Problem: BFS**





Queue

NE	E	SE	S

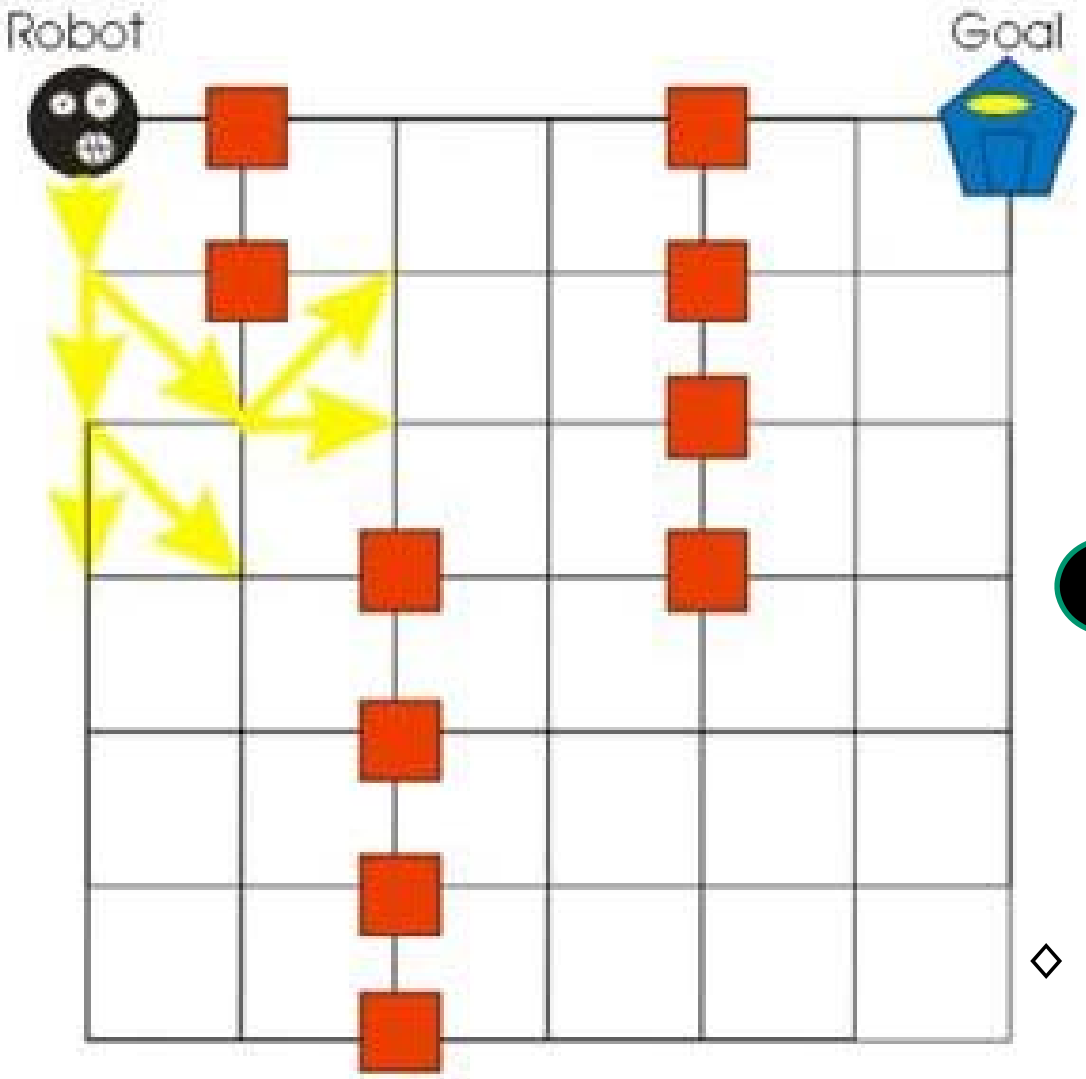
IN

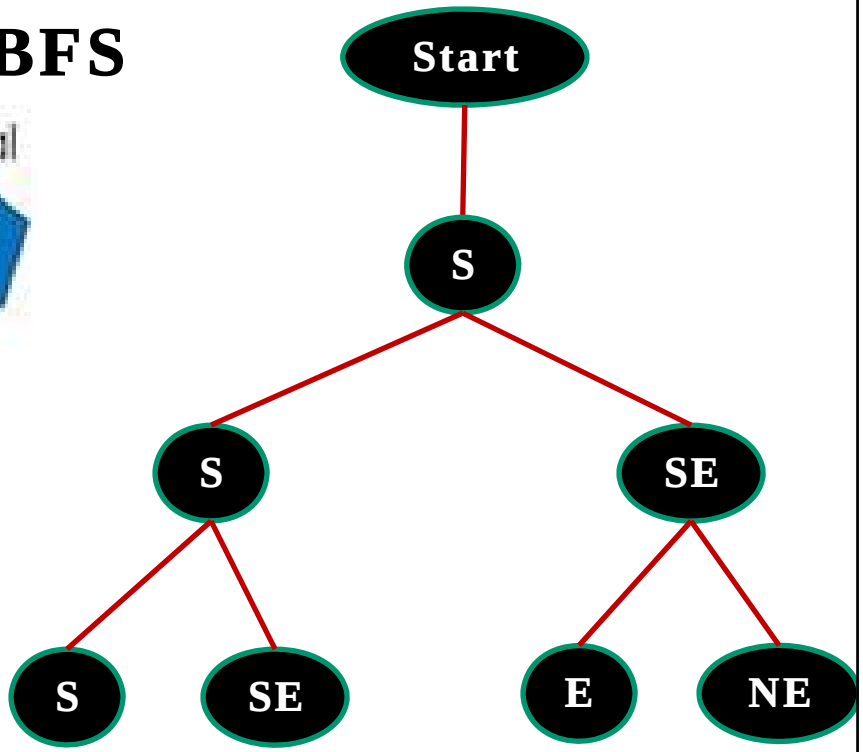
Start S S SE

OUT

Uninformed/Blind Search

- **Motion Planning Problem: BFS**

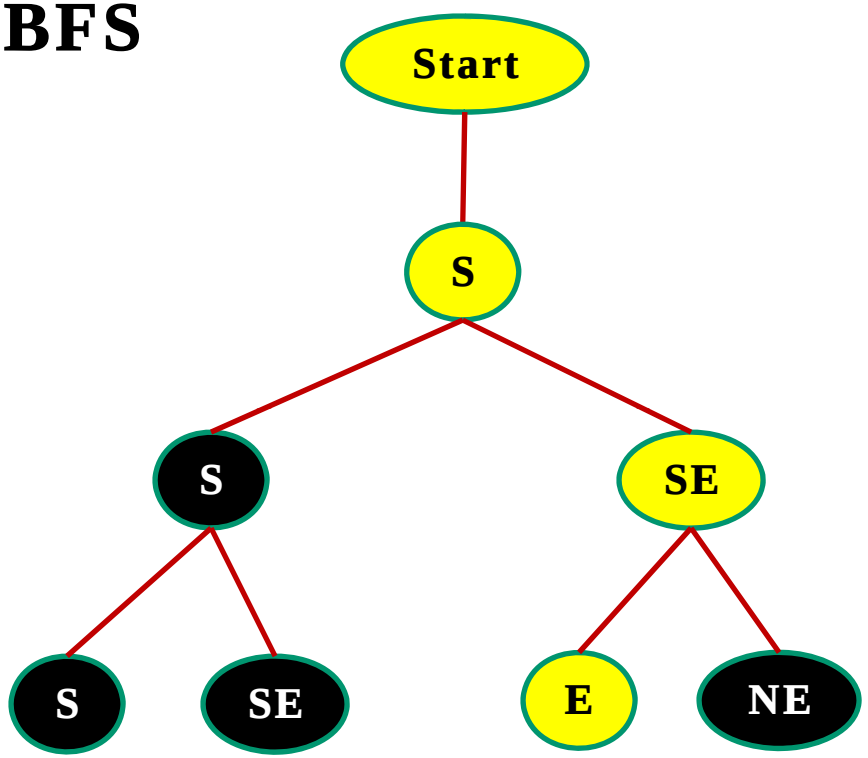




◇ The FIFO queue continues until the goal node is found

Uninformed/Blind Search

- **Motion Planning Problem: BFS**
  - ◇ The path leading to the goal node (E) is **traced back up the tree** which maps out the directions that the robot must follow to reach the goal.
  - ◇ Traveling **back up the tree**, we can see that the robot from the start would have to go south, then south east, then east to reach the goal.



Assume that **E** is the goal,

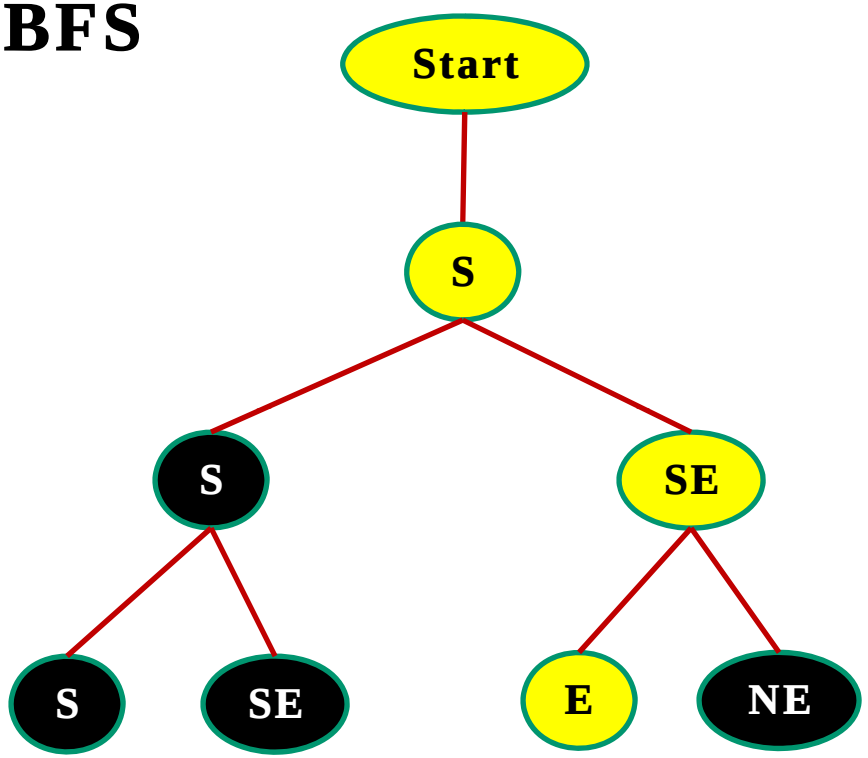
Path is: **Start→ S→SE→E**

Uninformed/Blind Search

- **Motion Planning Problem: BFS**
  - ◇ In BFS, every node generated must remain in memory and the **number of nodes generated** is at most:

$$O(b^d)$$

where **b** represents the **maximum branching factor** for each node and **d** is the **depth one must expand to reach the goal**.



b=2 and d=3

Total # of nodes=2^3=8

Uninformed/Blind Search

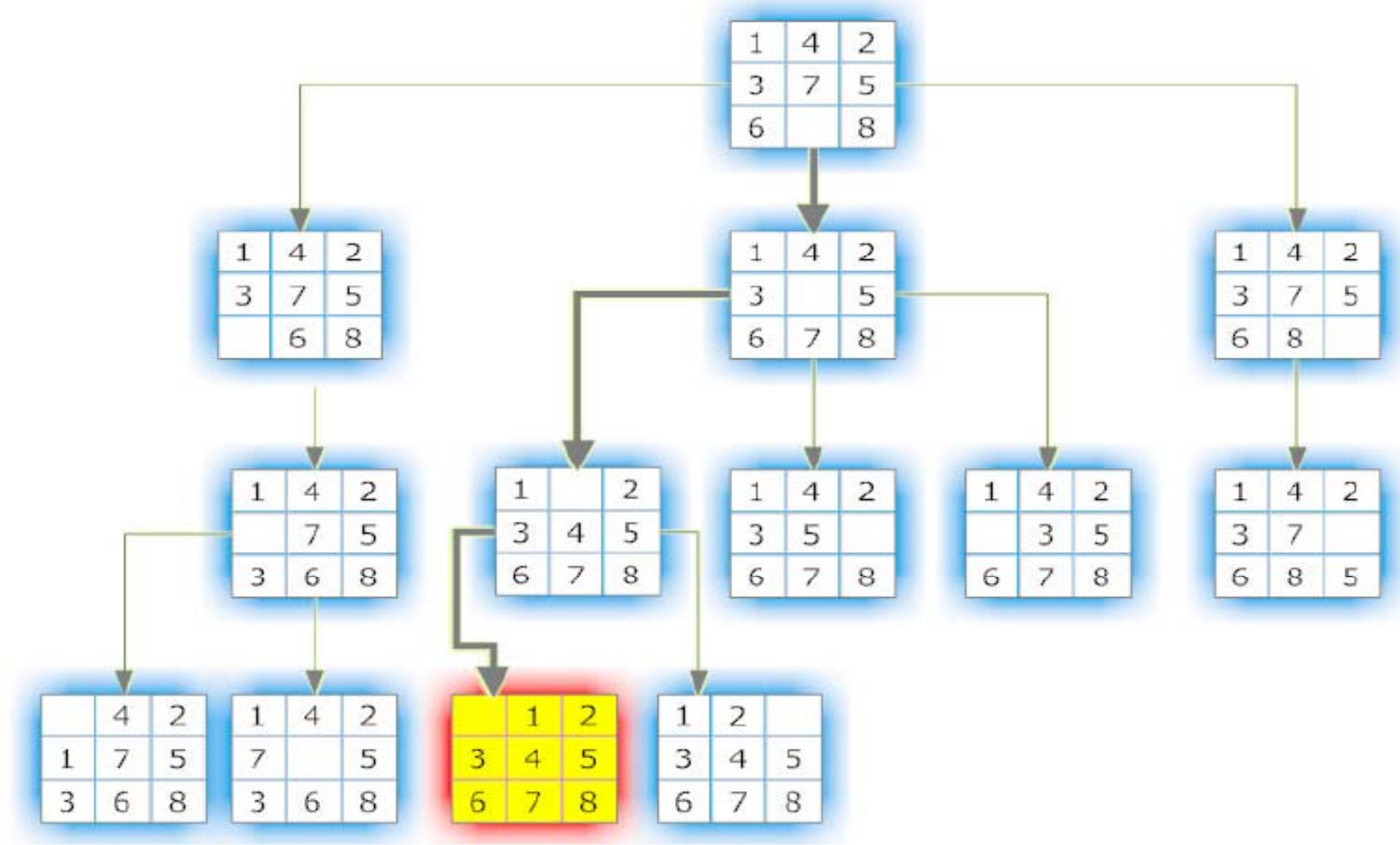
- **Motion Planning Problem: BFS**
  - ◇ Space complexity: **$O(b^d)$** (keep every node in memory)
  - ◇ We can see from this that a for **very large workspace** where the **goal is deep within the workspace**, the number of nodes could **expand exponentially** and demand a **very large memory requirement**.
  - ◇ **Time Complexity:**

    $$1 + b + b^2 + ... + b^d = (b^{d+1} - 1)/(b - 1) = O(b^d)$$

    i.e., **exponential** in d.

Uninformed/Blind Search

- **8-Puzzle Problem: BFS**



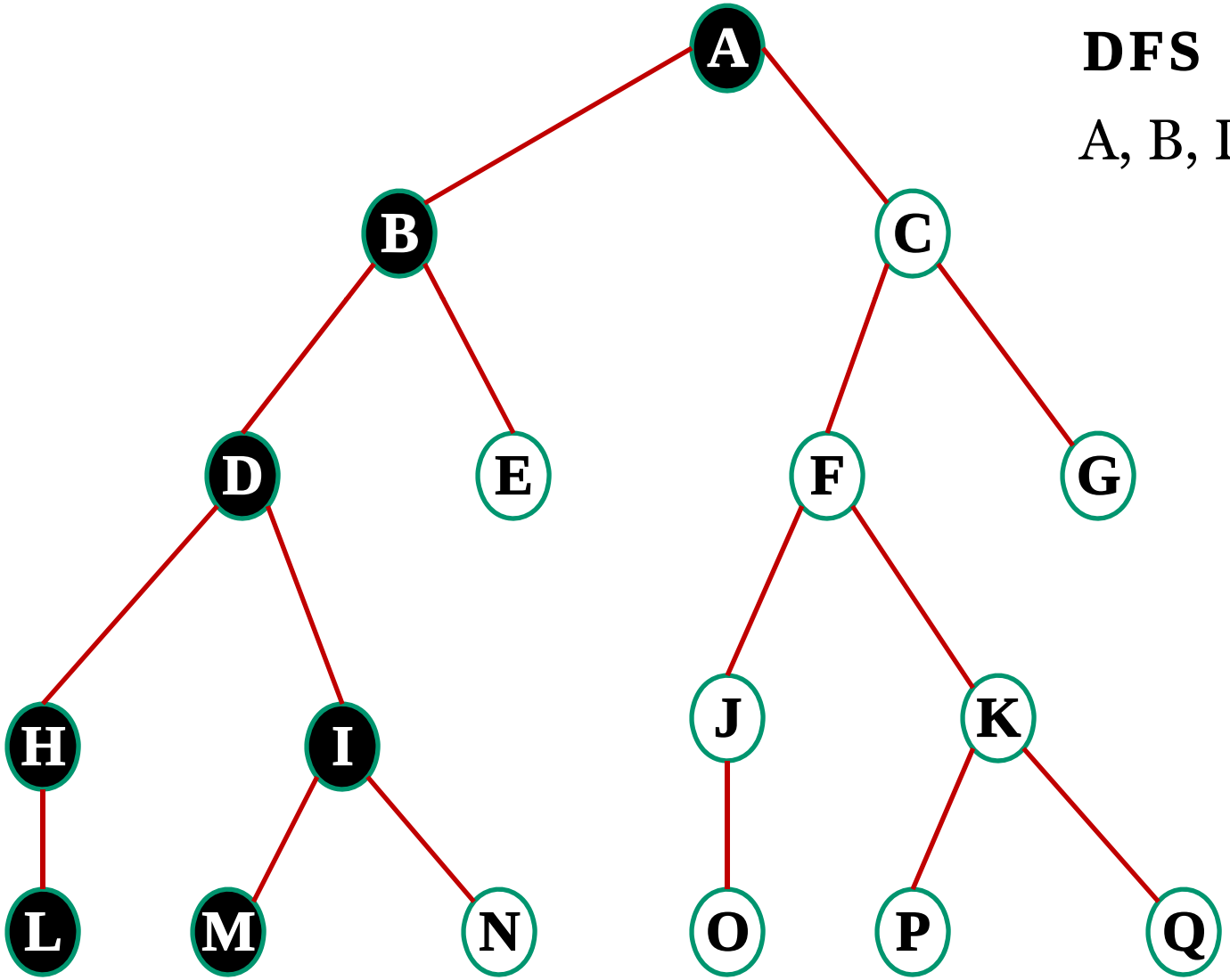
[1]

Uninformed/Blind Search

- **Breadth-first Search (BFS)**
  - ◇ High memory requirement.
  - ◇ **Exhaustive** search as it will process every node.
  - ◇ Doesn't get stuck.
  - ◇ Finds the shortest path (minimum number of steps).

Uninformed/Blind Search

- **Depth-first Search (DFS)**

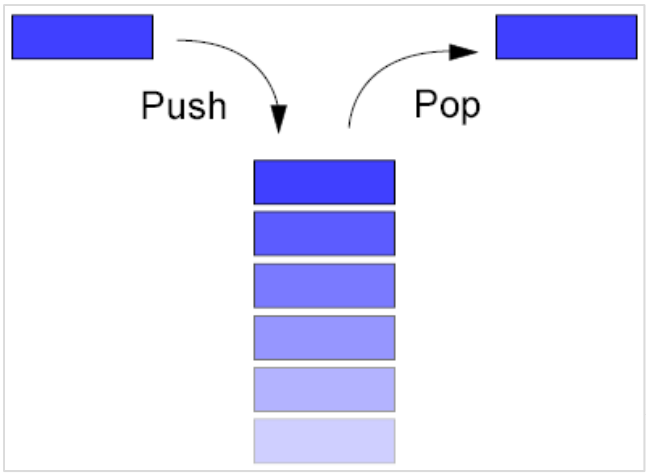


DFS

A, B, D, H, L, I, M ..

Uninformed/Blind Search

- **Motion Planning Problem: DFS**
  - ◇ DFS uses the **stack** as data structure.
  - ◇ Stack is a **Last-In-First-Out (LIFO)** data structure.
  - ◇ The stack contains the list of discovered nodes. The **most recent discovered node** is put (pushed) on top of the LIFO stack.
  - ◇ The **next node** to be expanded is then taken (popped) **from the top** of the stack and all of its successors are added to the stack.



Stack (LIFO)

http://wiki.ugcnet4cs.in/t/Stack

Uninformed/Blind Search

- **Depth-first**

Step 1. Form a stack S and set it to the initial state (for example, the Root).

Step 2. Until the S is empty or the goal state is found

do:

Step 2.1 Determine if the first element in the S is the goal.

Step 2.2 If it is not

Step 2.2.1 Remove the first element in S.

Step 2.2.2 Apply the rule to generate new state(s) (successor states).

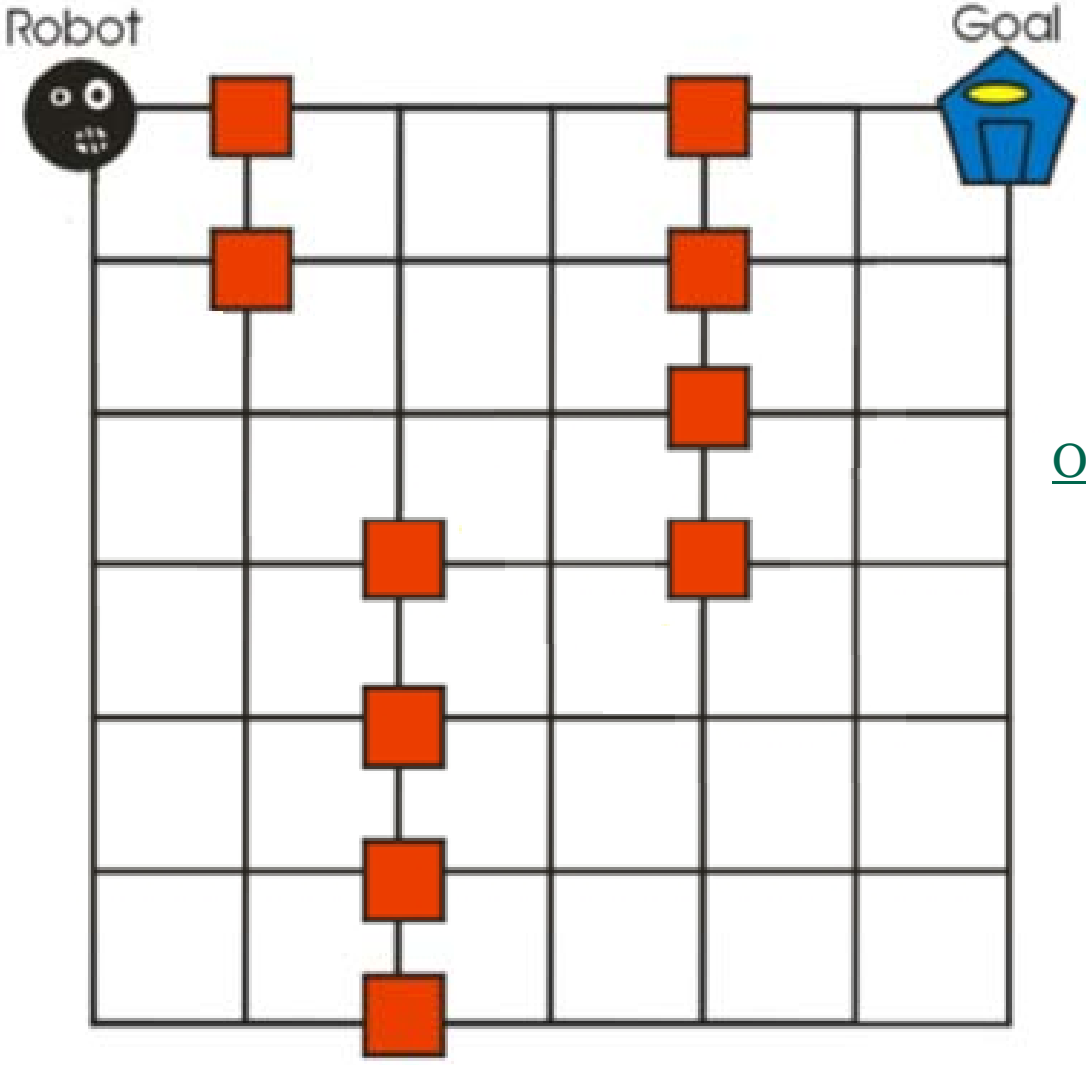
Step 2.2.3 If the new state is the goal state quit and return this state

Step 2.2.4 Otherwise add the new state to the beginning of the stack

Step 3. If the goal is reached, success; else failure.

Uninformed/Blind Search

- **Motion Planning Problem: DFS**



Start

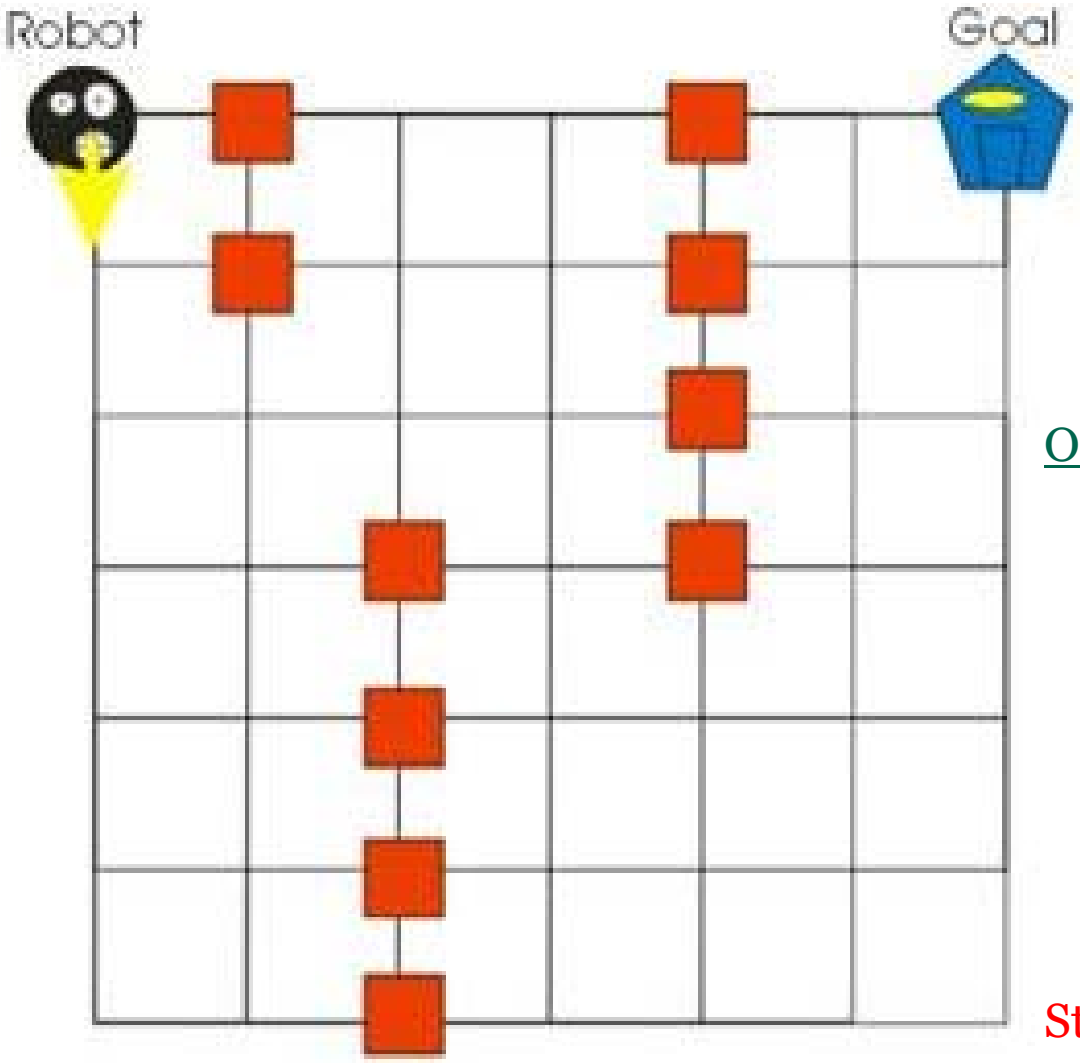
OUT IN

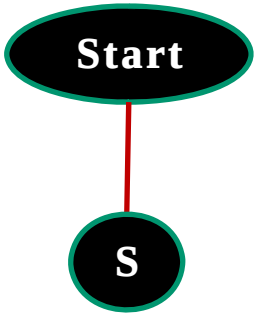
Stack

Start

Uninformed/Blind Search

- **Motion Planning Problem: DFS**





OUT IN

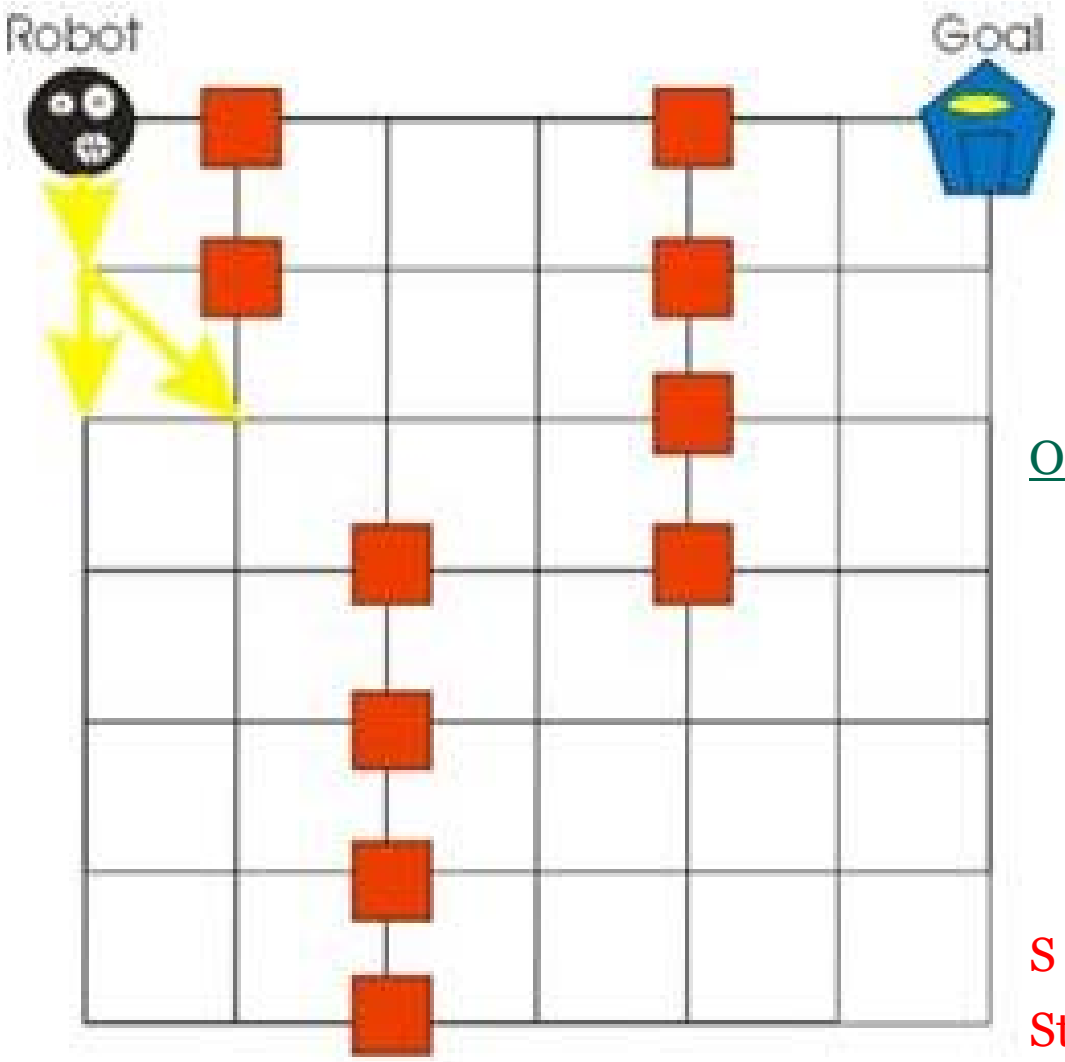
Stack

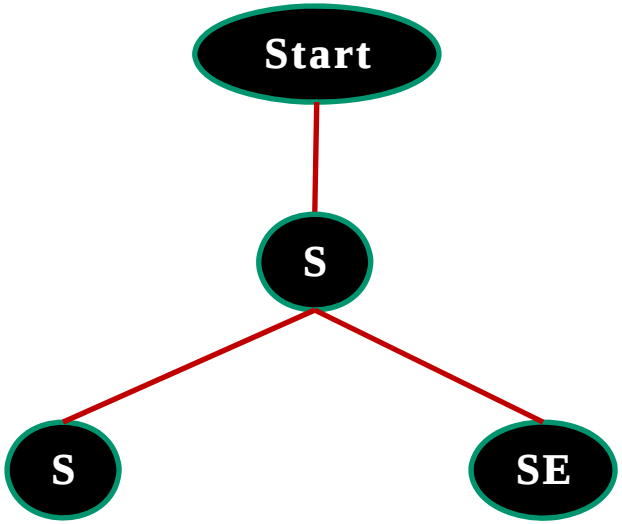
Start

S

Uninformed/Blind Search

- **Motion Planning Problem: DFS**





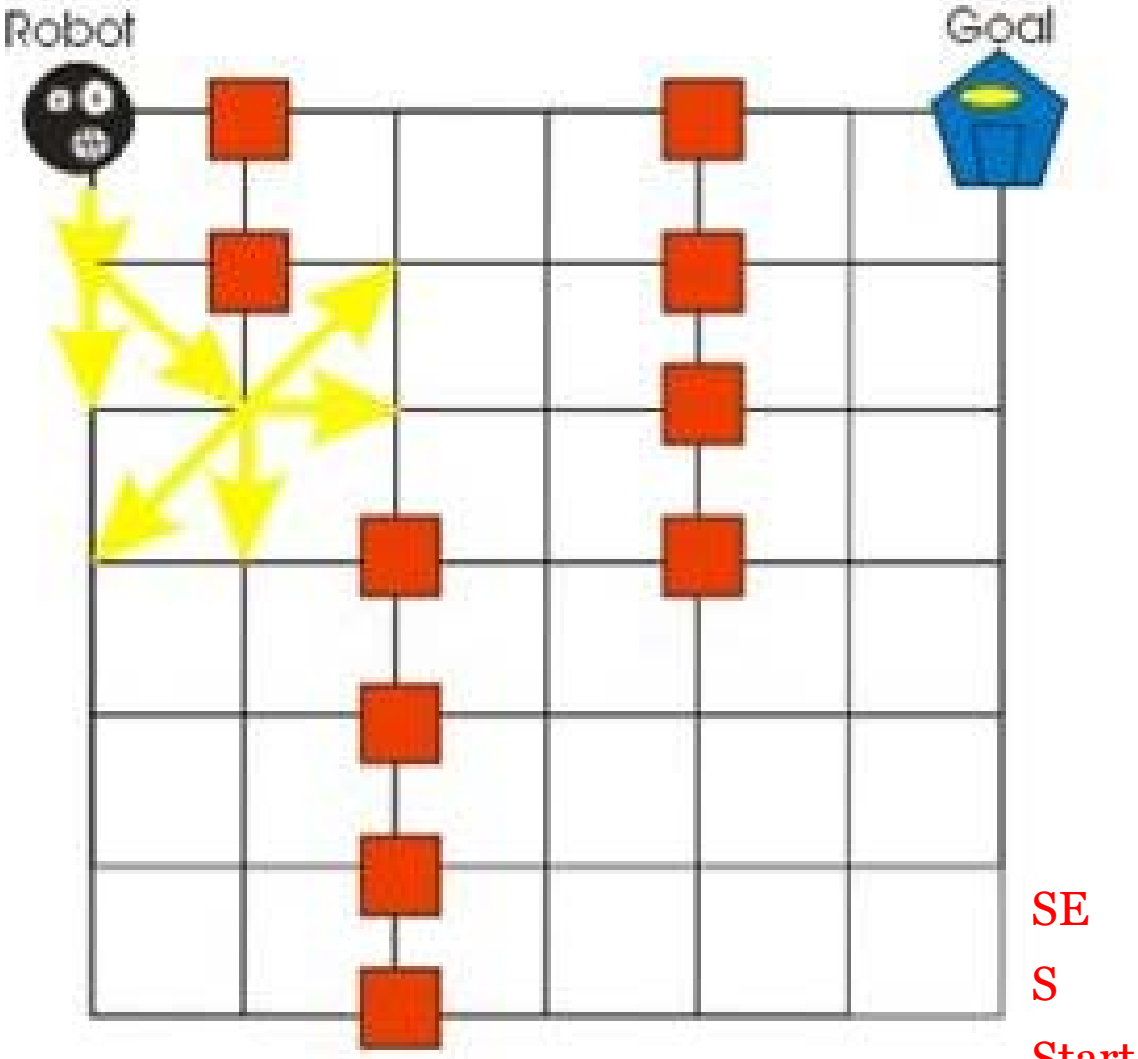
OUT IN

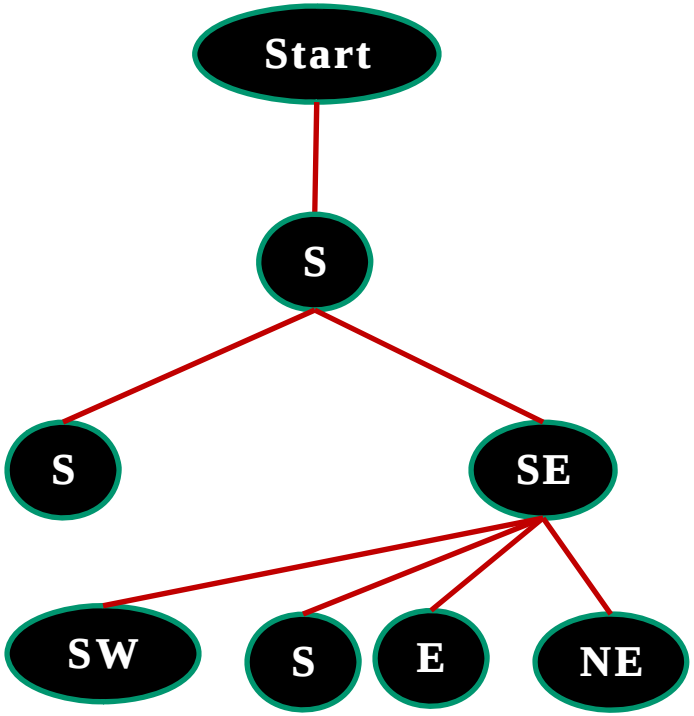
Stack

OUT	Stack
	SE
S	S
Start	

Uninformed/Blind Search

- **Motion Planning Problem: DFS**



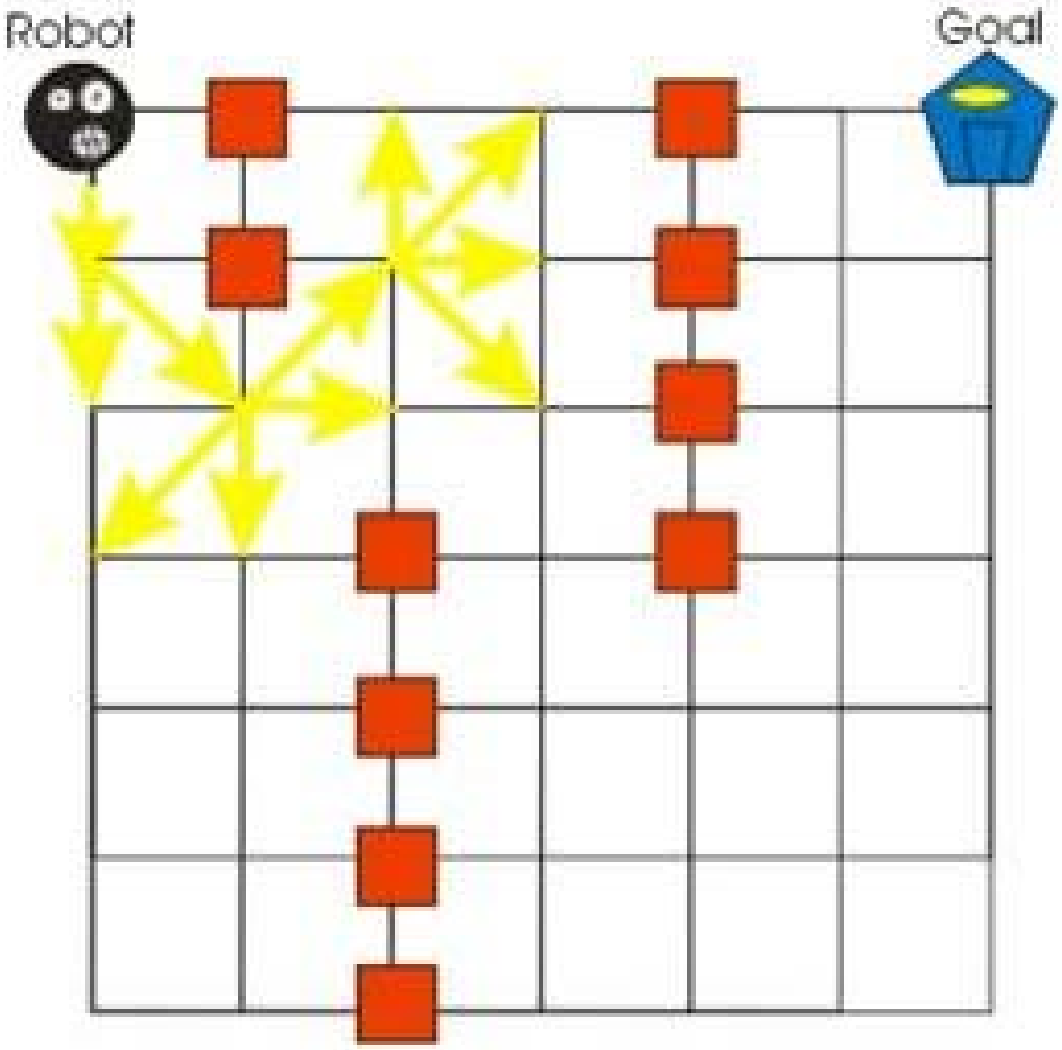


SE
S
Start

NE
E
S
SW
S

Uninformed/Blind Search

- **Motion Planning Problem: DFS**



◇ The next node to be expanded would be **NE** and its successors would be added to the stack and this loop continues until the goal is found.

◇ Once the goal is found, you can then **trace back** through the tree to obtain the path for the robot to follow.

Uninformed/Blind Search

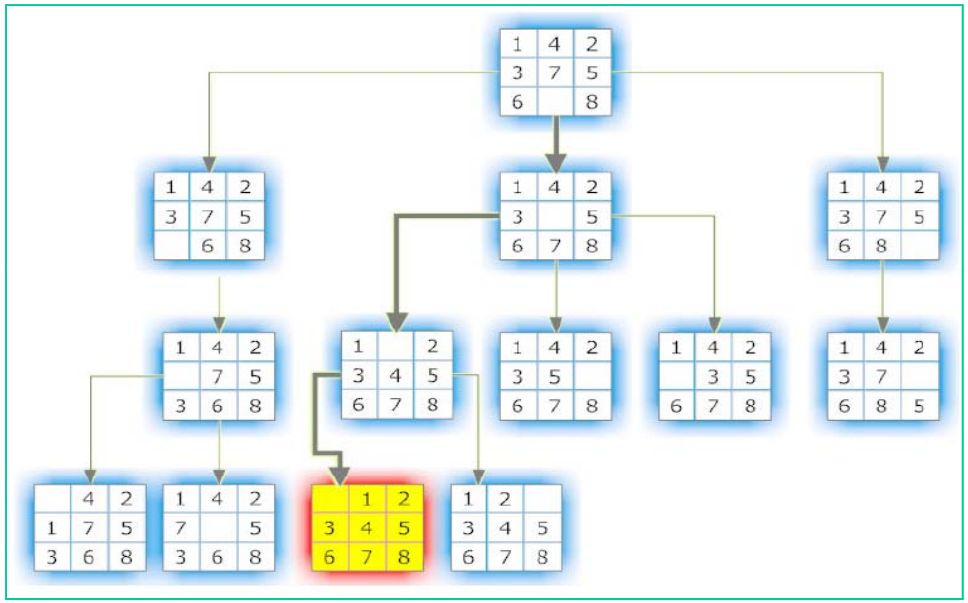
- **Motion Planning Problem: DFS**
  - ◇ Depth first search usually requires a **considerably less amount of memory** that BFS.
  - ◇ This is mainly because DFS does not always expand out every single node at each depth.
  - ◇ However the DFS could **continue down an unbounded branch forever even if the goal is not located** on that branch.
  - ◇ **Time Complexity:** $O(b^d) \rightarrow$ terrible if d is much larger than b but if solutions are dense, may be much faster than BFS.
  - ◇ **Space Complexity:** O(bd), i.e., linear space!

Uninformed/Blind Search

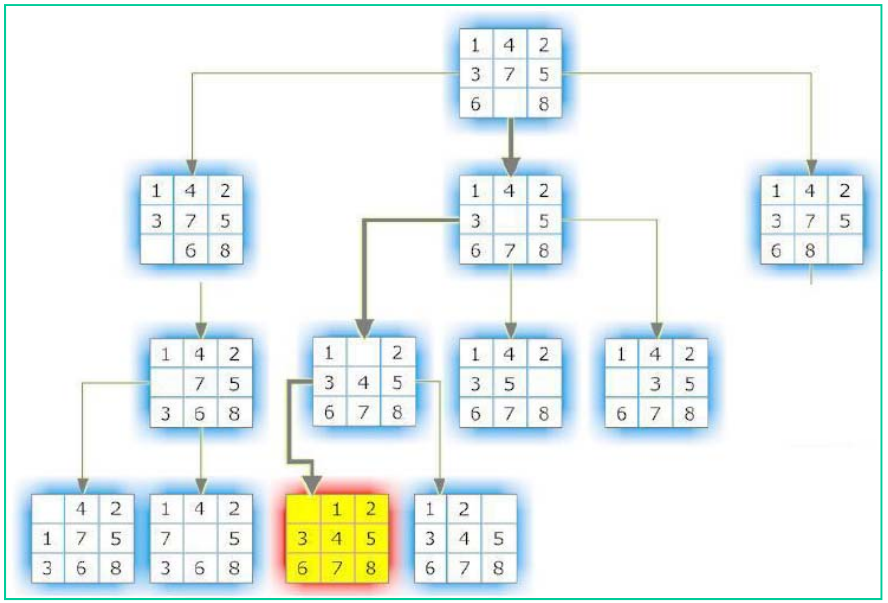
- **Depth-first Search (DFS)**
  - ◇ Low memory requirement.
  - ◇ Full state space search in that every node is processed, i.e, it's **exhaustive** within its set limits.
  - ◇ Could get stuck exploring infinite paths.
  - ◇ Used if there are many solutions and you need only one.

Uninformed/Blind Search

- **8-Puzzle Problem: BFS vs. DFS**



BFS



DFS

[1]

Uninformed/Blind Search

- **BFS vs. DFS: Space-Time Tradeoff**

	BFS	DFS
Space Complexity	More expensive	Less expensive. Requires only O(d) space irrespective of number of children per node.
Time Complexity	More time efficient. A vertex at lower level (closer to the root) is visited first before visiting a vertex that is at higher level (far away from the root)	Less time efficient.

Uninformed/Blind Search

- **BFS vs. DFS: Summary**

  DFS is preferred if

  ◇ The tree is deep;

  ◇ The branching factor is not excessive;

  ◇ Solutions occur deeply in the tree.

  BFS is preferred if

  ◇ The branching factor is not too large (hence memory costs);

  ◇ A solution appears at a relatively shallow level;

  ◇ No path is excessively deep.

Uninformed/Blind Search

- **Dijkstra's Algorithm**

  Dijkstra's algorithm (**Dynamic Programming**) is a graph search algorithm that solves **the single-source shortest path problem** for a fully connected graph with nonnegative edge path costs, producing a shortest path tree.

  Dynamic programming (DP) approaches are based on the recursive division of a problem into simpler sub-problems.

  Dijkstra's algorithm is uninformed, meaning it does not need to know the target node before hand and doesn't use heuristic information.

Uninformed/Blind Search

- **Dijkstra's Algorithm**

1. Init

Set start distance to 0, dist[s]=0,

others to infinite: dist[i]=∞ (for i ≠s),

Set Ready = { } .

2. Loop until all nodes are in Ready

Select node n with shortest known distance that is not in Ready
set Ready = Ready + {n} .

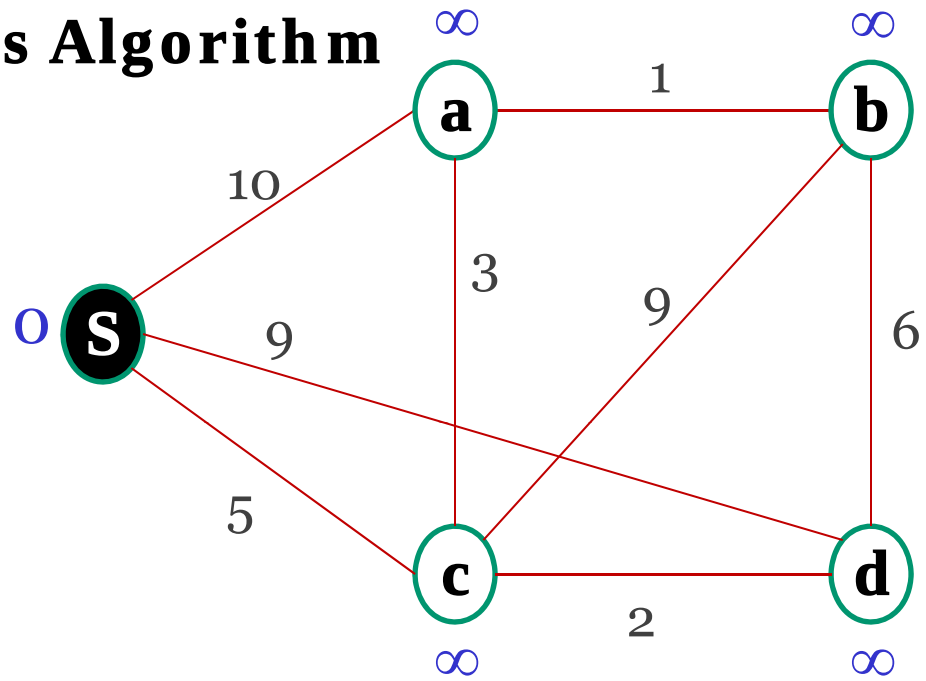
FOR each neighbor node m of n

IF dist[n]+edge(n,m) < dist[m] /* shorter path found */

THEN { dist[m] = dist[n]+edge(n,m); pre[m] = n;}

Uninformed/Blind Search

- **Dijkstra's Algorithm**



From s to:	S	a	b	c	d
Distance	0	∞	∞	∞	∞
Predecessor	-	-	-	-	-

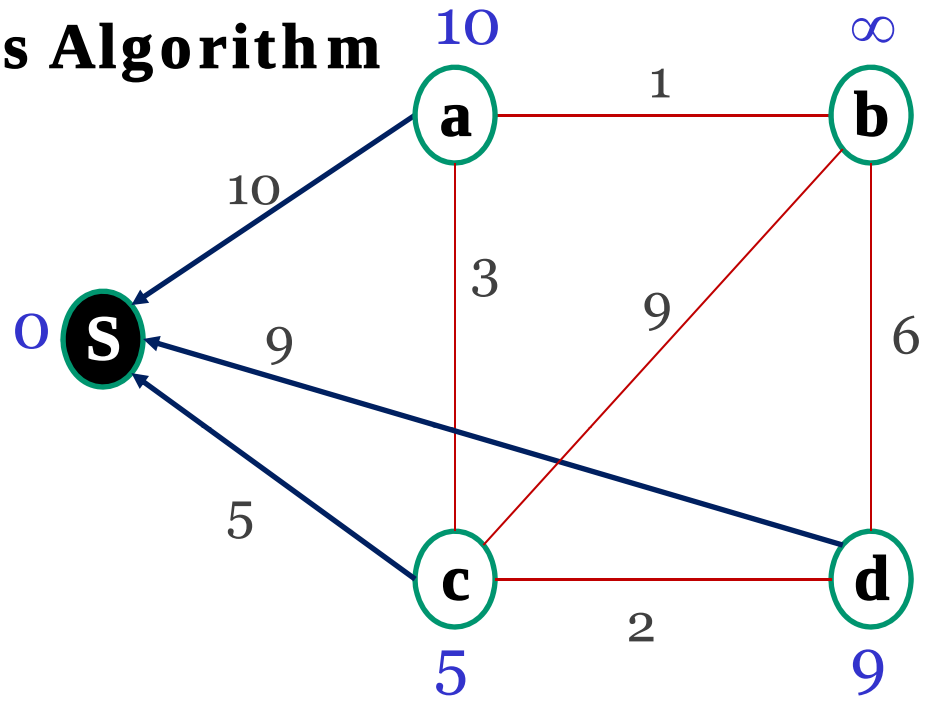
Step 0: Init list, no predecessors

Ready = {}

[3]

Uninformed/Blind Search

- **Dijkstra's Algorithm**



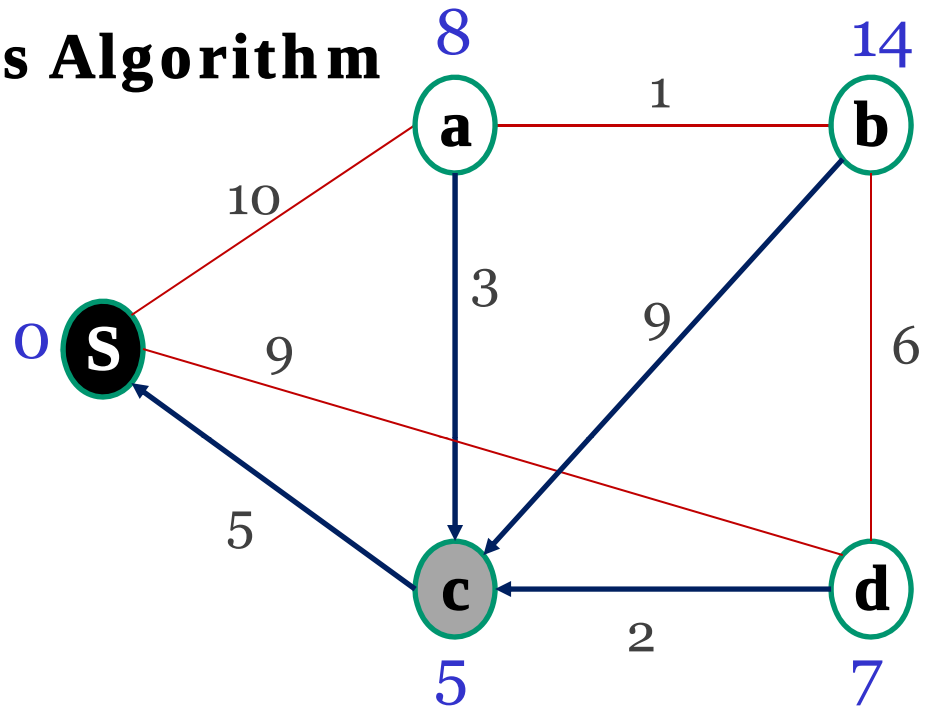
From s to:	S	a	b	c	d
Distance	0	10	∞	5	9
Predecessor	-	s	-	s	s

Step 1: Closest node is s, add to Ready

Update distances and pred. to all neighbors of *s*, **Ready = {S}**

Uninformed/Blind Search

- **Dijkstra's Algorithm**



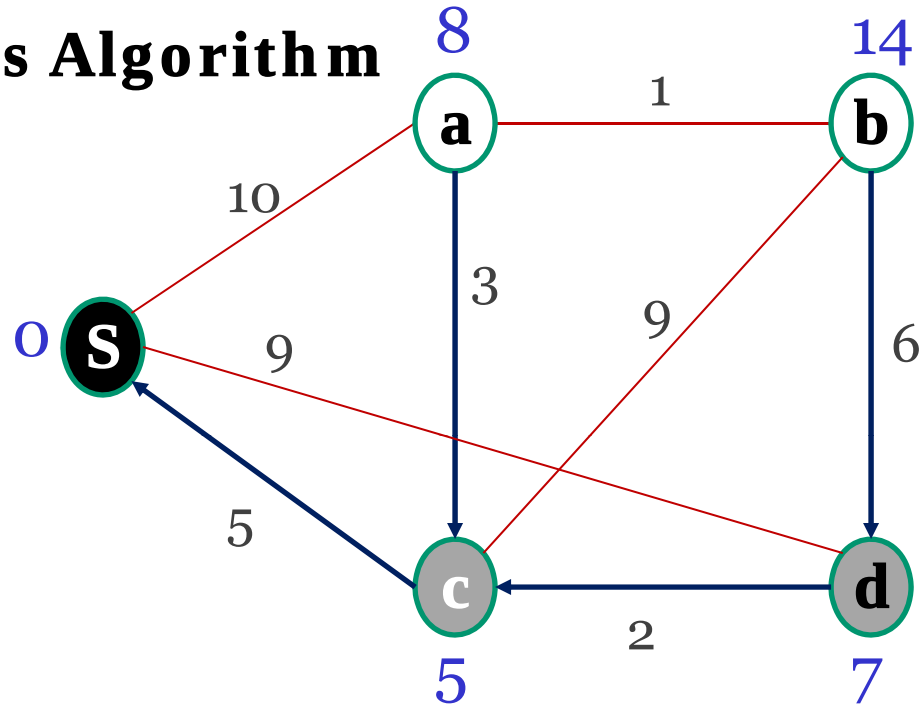
From s to:	S	a		b	c	d	
Distance	0	~~10~~	8	14	5	~~9~~	7
Predecessor	-	~~s~~	c	c	s	~~s~~	c

Step 2: Next closest node is c, add to Ready

Update distances and pred. for a and d, **Ready = {S, c}**

Uninformed/Blind Search

- **Dijkstra's Algorithm**



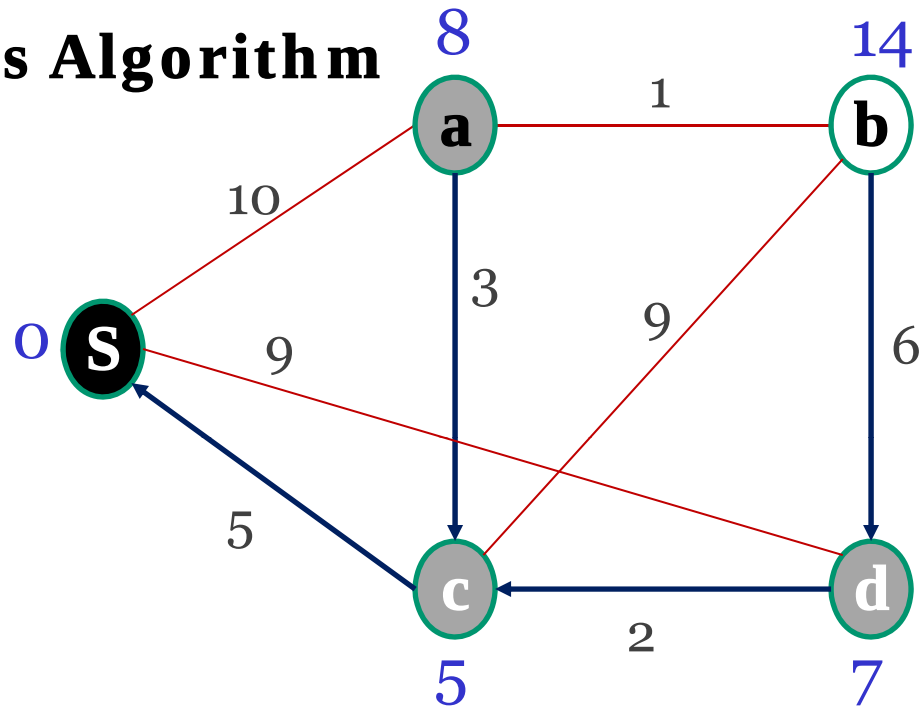
From s to:	S	a	b		c	d
Distance	0	8	~~14~~	13	5	7
Predecessor	-	c	~~c~~	d	s	c

Step 3: Next closest node is d, add to Ready

Update distance and pred. for b. **Ready = {s, c, d}**

Uninformed/Blind Search

- **Dijkstra's Algorithm**



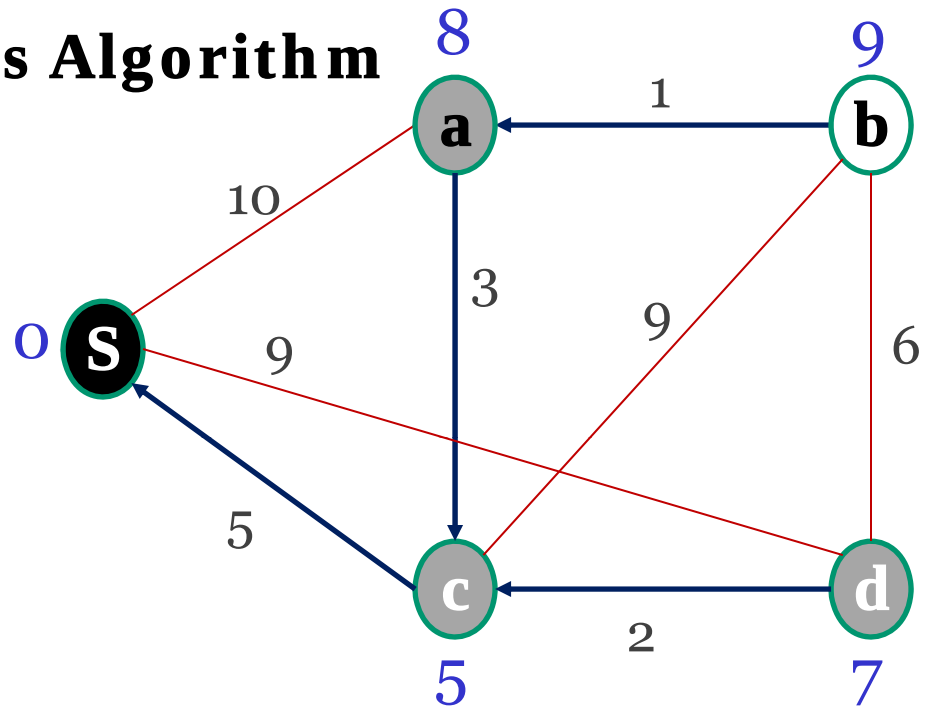
From s to:	S	a	b		c	d
Distance	0	8	~~13~~	9	5	7
Predecessor	-	c	~~d~~	a	s	c

Step 4: Next closest node is a, add to Ready

Update distance and pred. for b. **Ready = {S, a, c, d}**

Uninformed/Blind Search

- **Dijkstra's Algorithm**



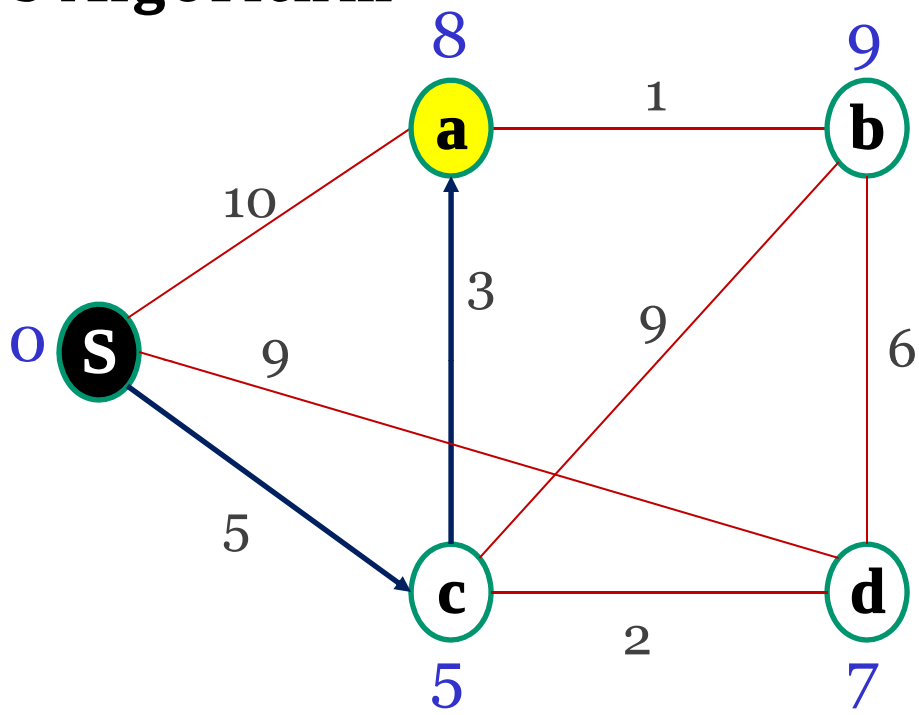
From s to:	S	a	b	c	d
Distance	0	8	9	5	7
Predecessor	-	c	a	s	c

Step 5: Closest node is b, add to Ready

check all neighbors of s. Ready = {S, a, b, c, d} **complete!**

Uninformed/Blind Search

- **Dijkstra's Algorithm**



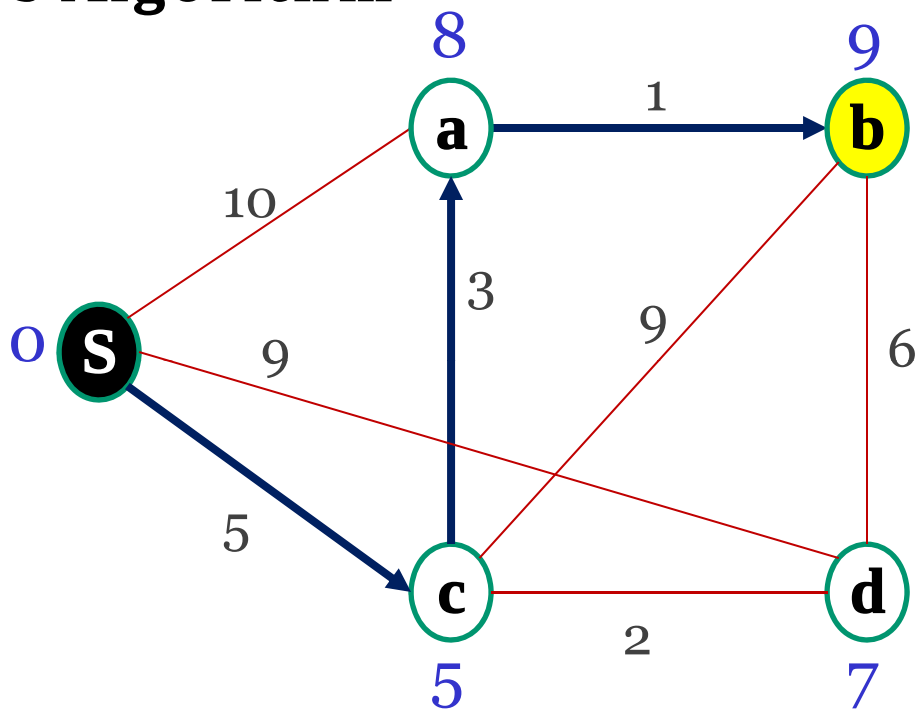
- dist[a] = 8
- pre[a] = c
- pre[c] = S
- Shortest path: S→c →a,
- Length is 8

From s to:	S	a	b	c	d
Distance	0	8	9	5	7
Predecessor	-	c	a	s	c

Shortest path between s and a is {S, c, a} ⇒ length=8

Uninformed/Blind Search

- **Dijkstra's Algorithm**



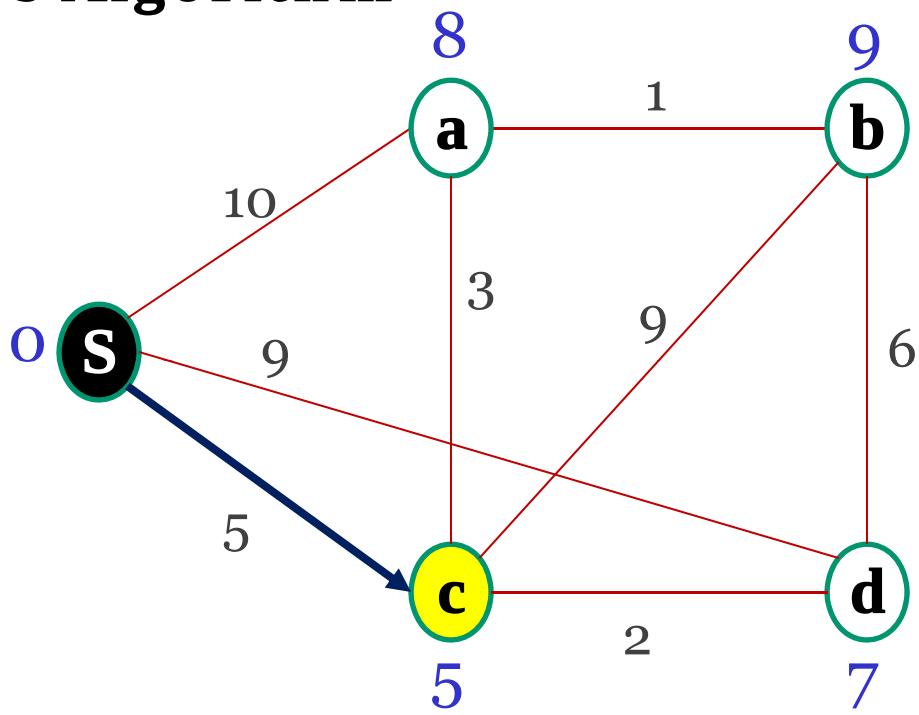
- ◇ dist[b] = 9
- ◇ pre[b] = a
- ◇ pre[a] = c
- ◇ pre[c] = S
- ◇ Shortest path: S→c →a →b

From s to:	S	a	b	c	d
Distance	0	8	9	5	7
Predecessor	-	c	a	s	c

Shortest path between s and b is {S, c, a,b} ⇒ length=9

Uninformed/Blind Search

- **Dijkstra's Algorithm**



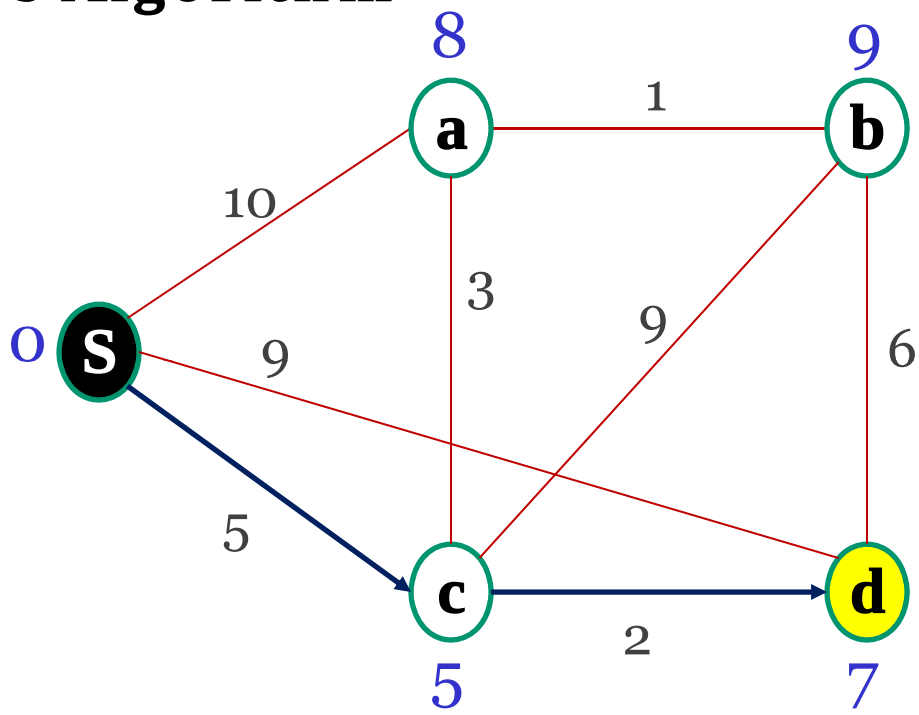
- ◇ dist[c] = 5
- ◇ pre[c] = S
- ◇ Shortest path: S→c

From s to:	S	a	b	c	d
Distance	0	8	9	5	7
Predecessor	-	c	a	s	c

Shortest path between s and d is {S, c} ⇒ length=5

Uninformed/Blind Search

- **Dijkstra's Algorithm**



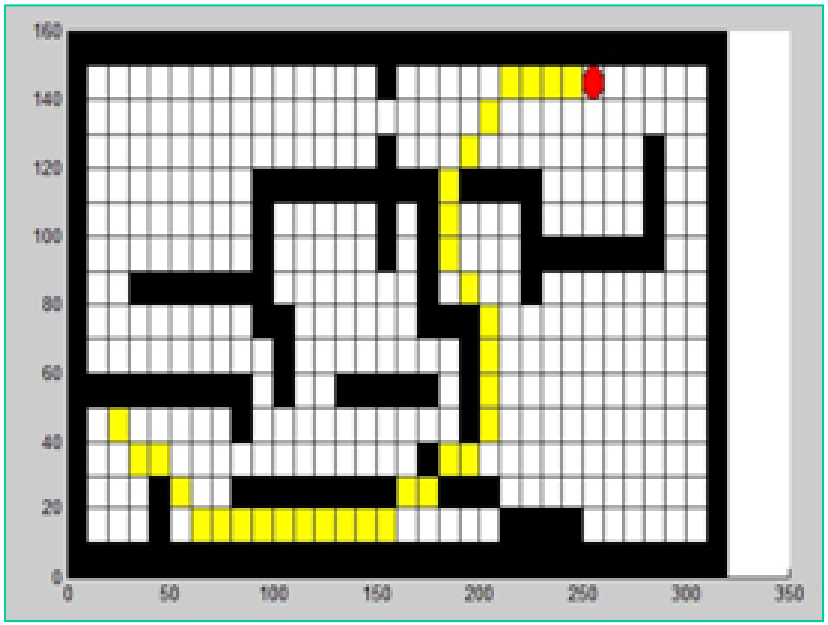
- ◇ dist[d] = 7
- ◇ pre[d] = c
- ◇ pre[c] = S
- ◇ Shortest path: S→c →d

From s to:	S	a	b	c	d
Distance	0	8	9	5	7
Predecessor	-	c	a	s	c

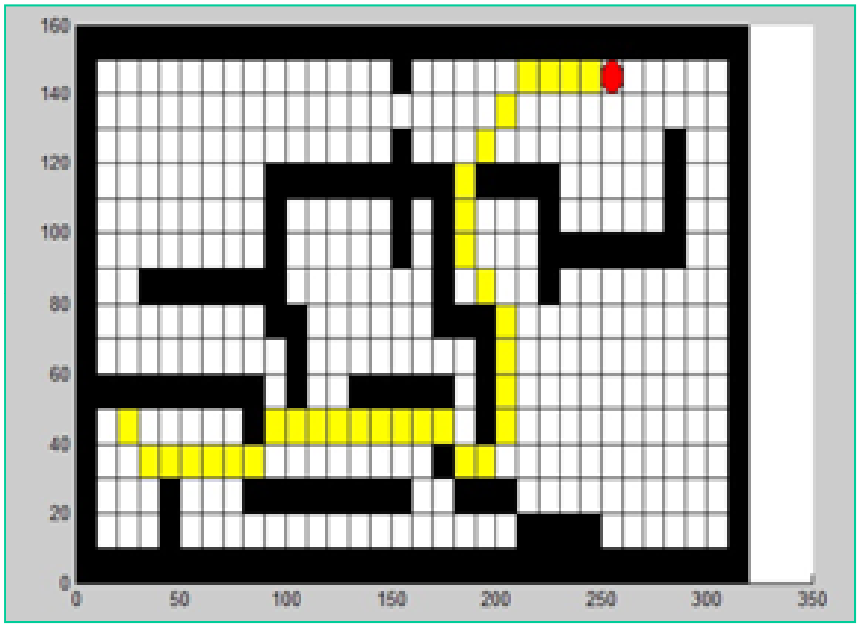
Shortest path between s and d is {S, c, d} ⇒ length=7

Uninformed/Blind Search

- **Motion Planning Problem**



BFS



Dijkstra

Source code is available on the course website…

Uninformed/Blind Search

- **Branch and Bound (BB or B&B) Algorithm**

  BB is a general algorithm **for finding optimal solutions** of various optimization problems, especially in **discrete and combinatorial optimization**.

  The algorithm is based on an **implicit enumeration of all solutions** of the considered optimization problem.

[For reading]

Uninformed/Blind Search

- **Branch and Bound (BB or B&B) Algorithm**

Return a solution or failure

Q is a priority queue sorted on the current cost from the start to the goal

Step 1. Add the initial state (or root) to the queue.

Step 2. Until the goal is reached or the queue is empty

do

Step 2.1 Remove the first path from the queue;

Step 2.2. Create new paths by extending the first path to all the neighbors of the terminal node.

Step 2.3. Remove all new paths with loops.

Step 2.4. Add the remaining new paths, if any, to the queue.

Step 2.5. Sort the entire queue such as the least-cost paths are in front.

End

[2]

Uninformed/Blind Search

- **BB Algorithm: TSP**
  - ◇ Given **n** cities,
  - ◇ A travelling salesman must visit the n cities and return home, making a loop (roundtrip).

  - ◇ He would like to travel in the most efficient way (cheapest way or shortest distance or some other criterion).

- **Search Space**

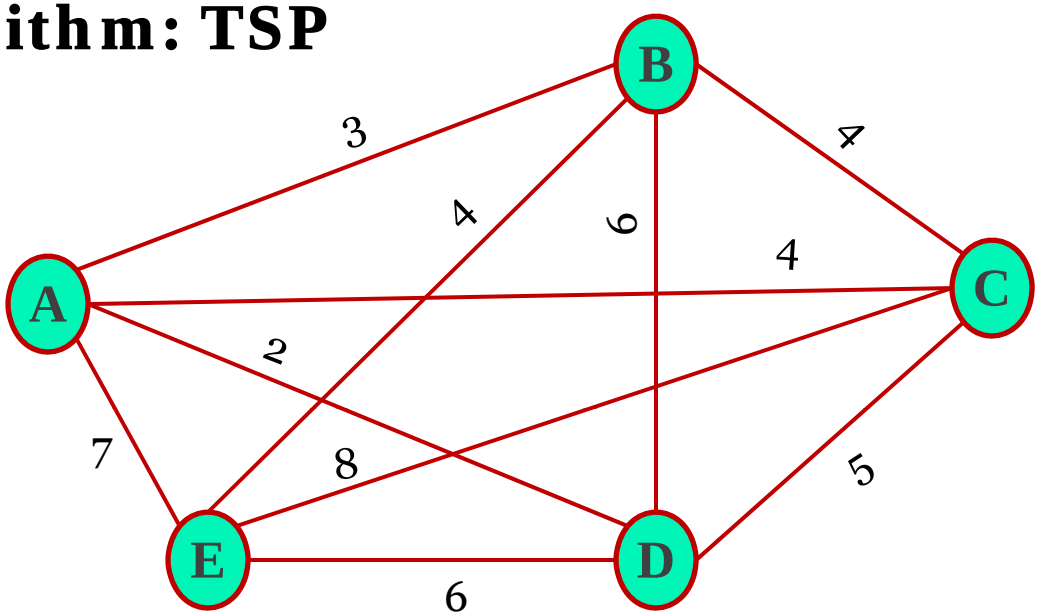
  Search space is BIG:

  for 21 cities there are 21! ≈ 51090942171709440000 **possible** tours. This is NP-Hard problem.

[For reading]

Uninformed/Blind Search

- **BB Algorithm: TSP**



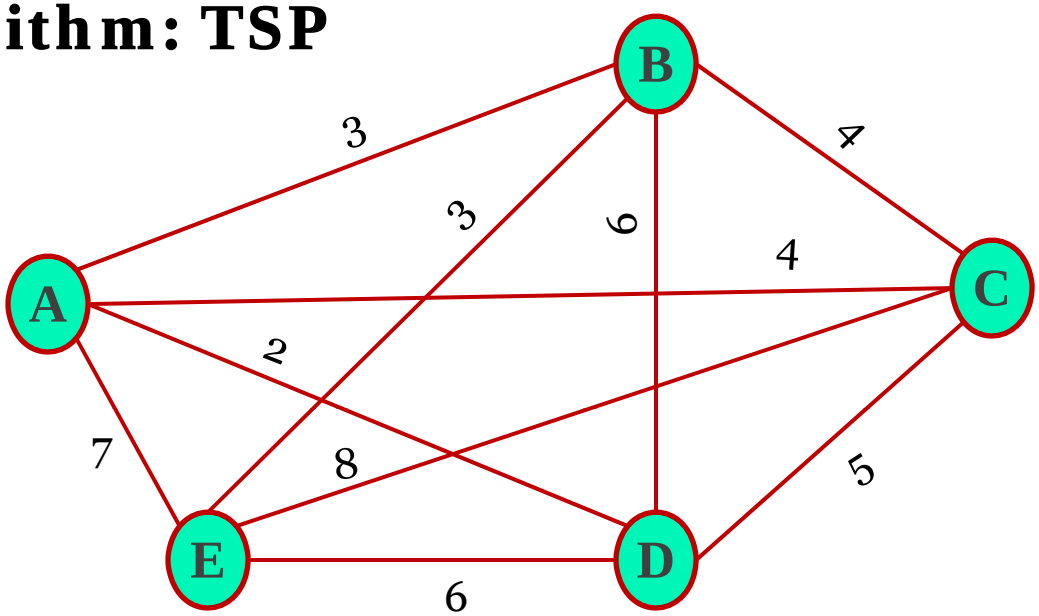
A straightforward method for computing a **lower bound on the cost of any solution** may be the following:

$$\frac{1}{2}\sum_{v \in V}$$ sum of the costs of the **two least** cost edges adjacent to v

[For reading]

Uninformed/Blind Search

- **BB Algorithm: TSP**

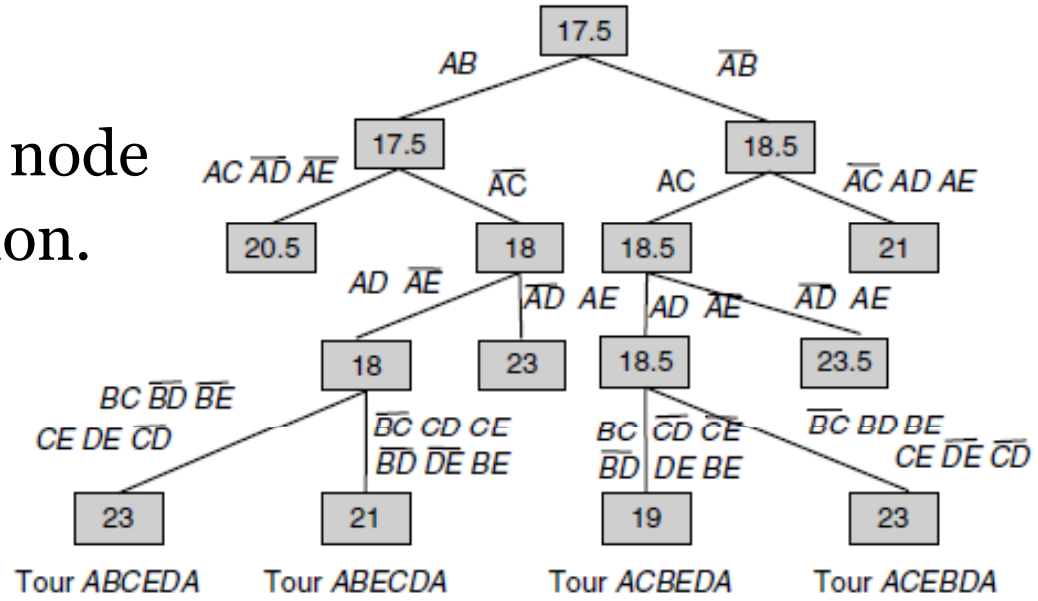


For this example, the lower bound is associated with the edges AD, AB, BA, BE, CB, CA, DA, DC, EB, ED and is equal to 17.5.

[For reading]

Uninformed/Blind Search

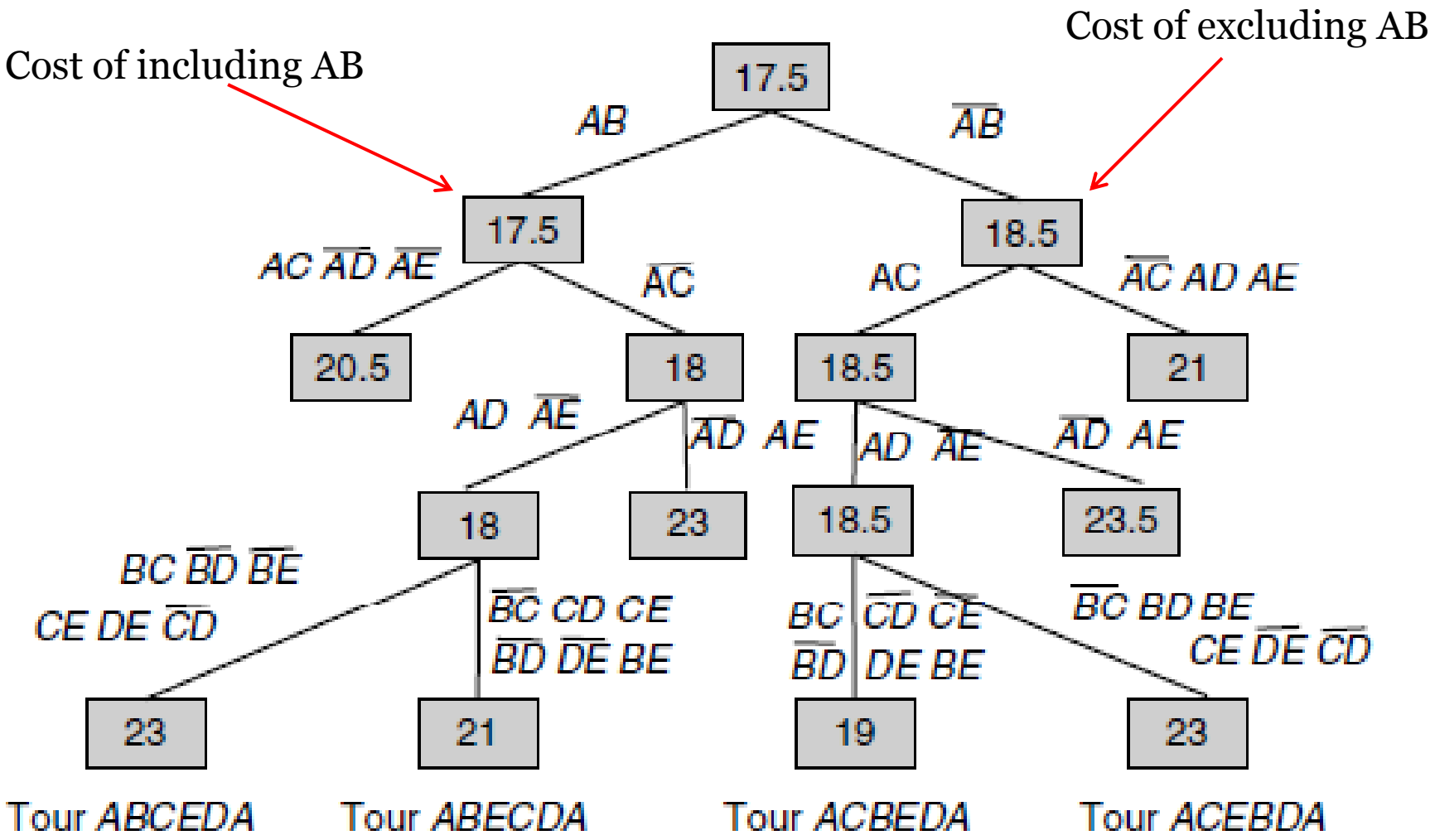
- **BB Algorithm: TSP**
  - ◇ In the **search tree**, each node represents a partial solution.
  - ◇ Each partial solution is represented by the set of **associated edges** (i.e., edges that must be in the tour) and the **nonassociated edges** (i.e., set of edges that must not be on the tour).
  - ◇ The branching consists in generating two children nodes. A set of additional **excluding and including edges** is associated with each child.



[For reading]

Uninformed/Blind Search

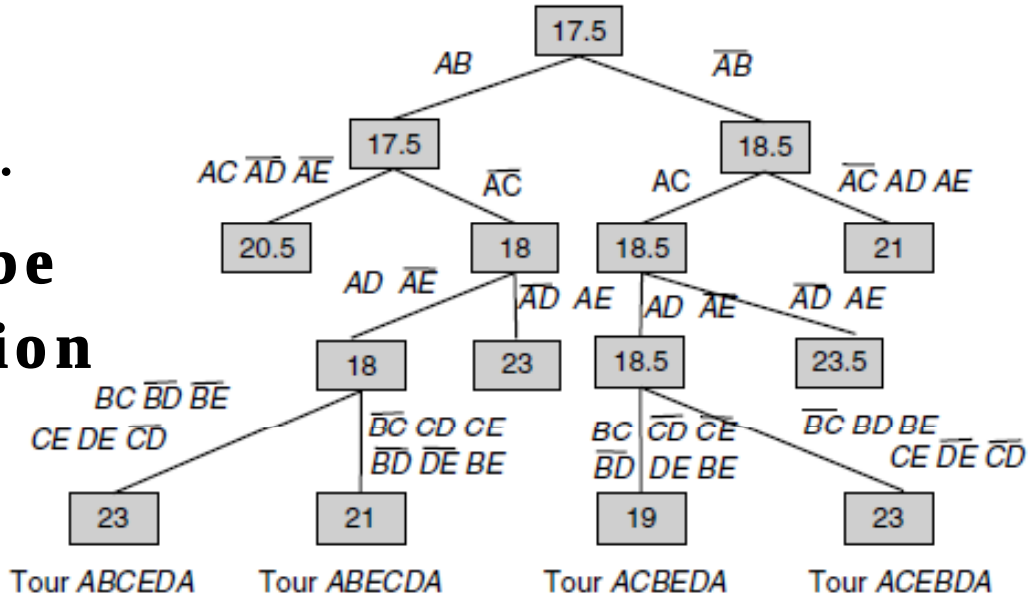
- **BB Algorithm: TSP**



[For reading]

Uninformed/Blind Search

- **BB Algorithm: TSP**
  - ◇ Two rules may be applied.
    - An **edge (a, b) must be included** if its **exclusion** makes it **impossible** for a or b to have **two adjacent edges** in the tour.

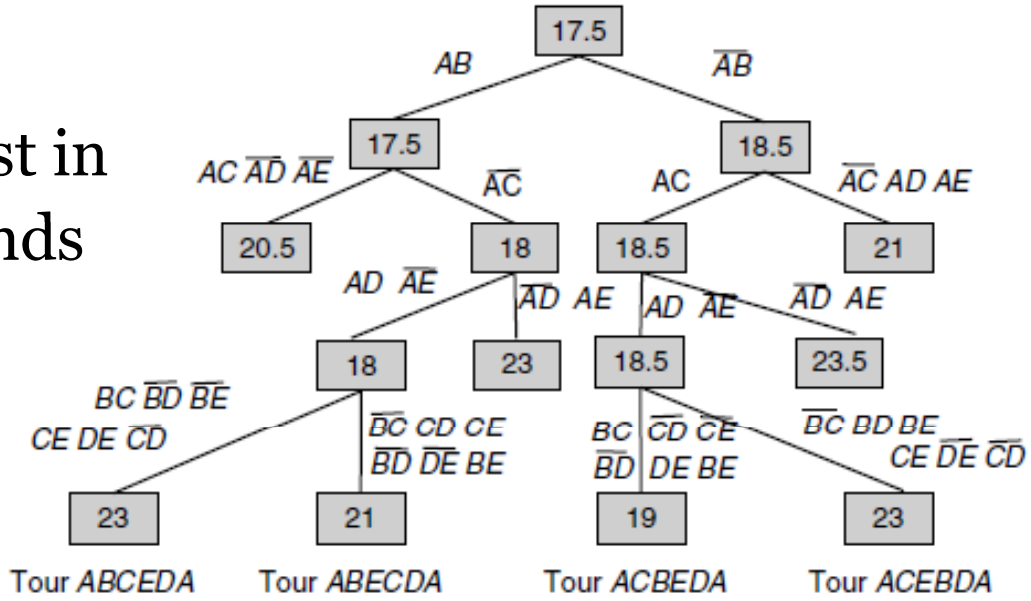


    - An **edge (a, b) must be excluded** if its **inclusion** causes for a or b to have more **than two adjacent edges in the tour** or would **complete a nontour with edges already included**.

[For reading]

Uninformed/Blind Search

- **BB Algorithm: TSP**
  - ◇ The **pruning** consists first in computing the lower bounds for each child.
  - ◇ For instance, if the edge (A,E) is included and the edge (B,C) is excluded, the **lower bound** will be associated with the following selected edges (A,D), (A,E), (B,A), (B,E), (C,D), (C,A), (D,A), (D,C), (E,B), (E,A) and is **equal to 20.5**.
  - ◇ If the **lower bound** associated with a node is **larger than** the known upper bound, the node is proved to be **unable** to generate an optimal solution and then is **not explored**.



Outline

- Graph Search Methods
- Uninformed/Blind Search
- **Informed Search**
- Summary

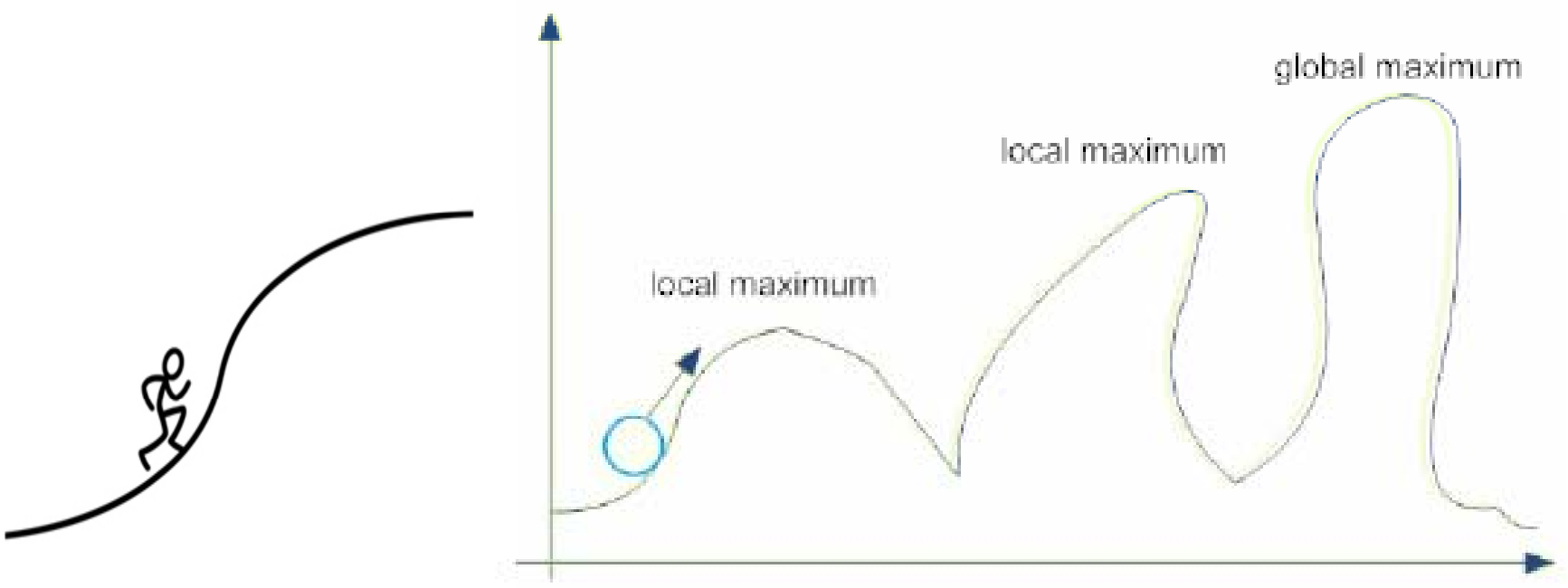
Informed Search

- Informed search tries to use the **knowledge known** about the problem to prune the search.
- This knowledge is used in the form of a **heuristic function**.
- The heuristic function should give an **estimate of the distance to the goal**.

Informed Search

- **Hill Climbing**

  In the hill climbing algorithm (also known as **gradient ascent** or **gradient descent**) the idea is to keep improving current state, and stop when we can't improve any more.



Hill -climbing is analogous to the way one might set out to climb a mountain in the absence of a map: always move in the direction of increasing altitude

[1]

Informed Search

- **Hill Climbing Algorithm**

Step 1. Set current_state to take initial state (starting state).

Step 2. loop

Step 2.1 Generate successors of current_state;

Step 2.2 Get the successor with the highest value;

Step 2.3 if value(successor) < value(current_state)

then Return current_state

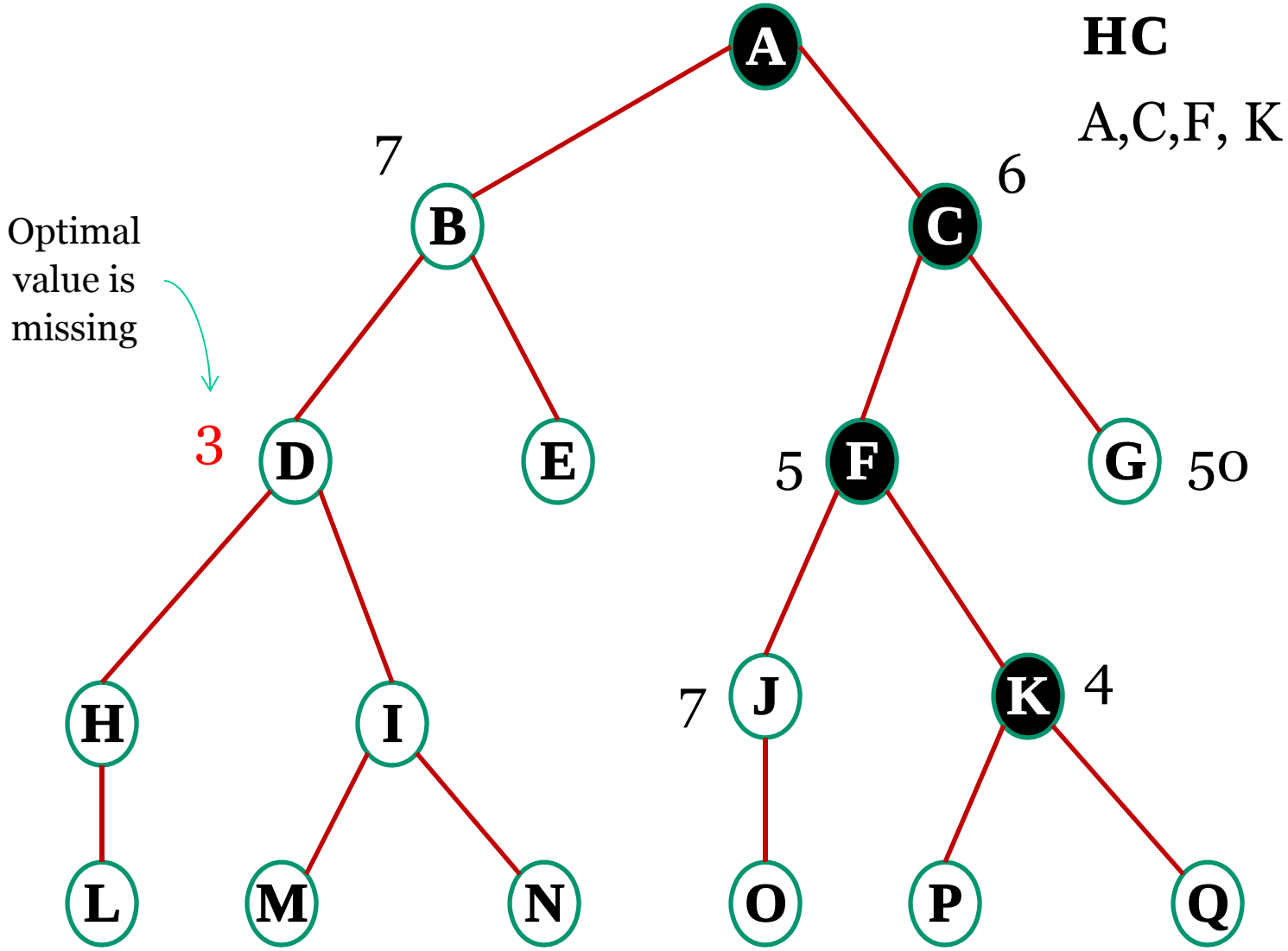
else currenst_state = successor

end

[1]

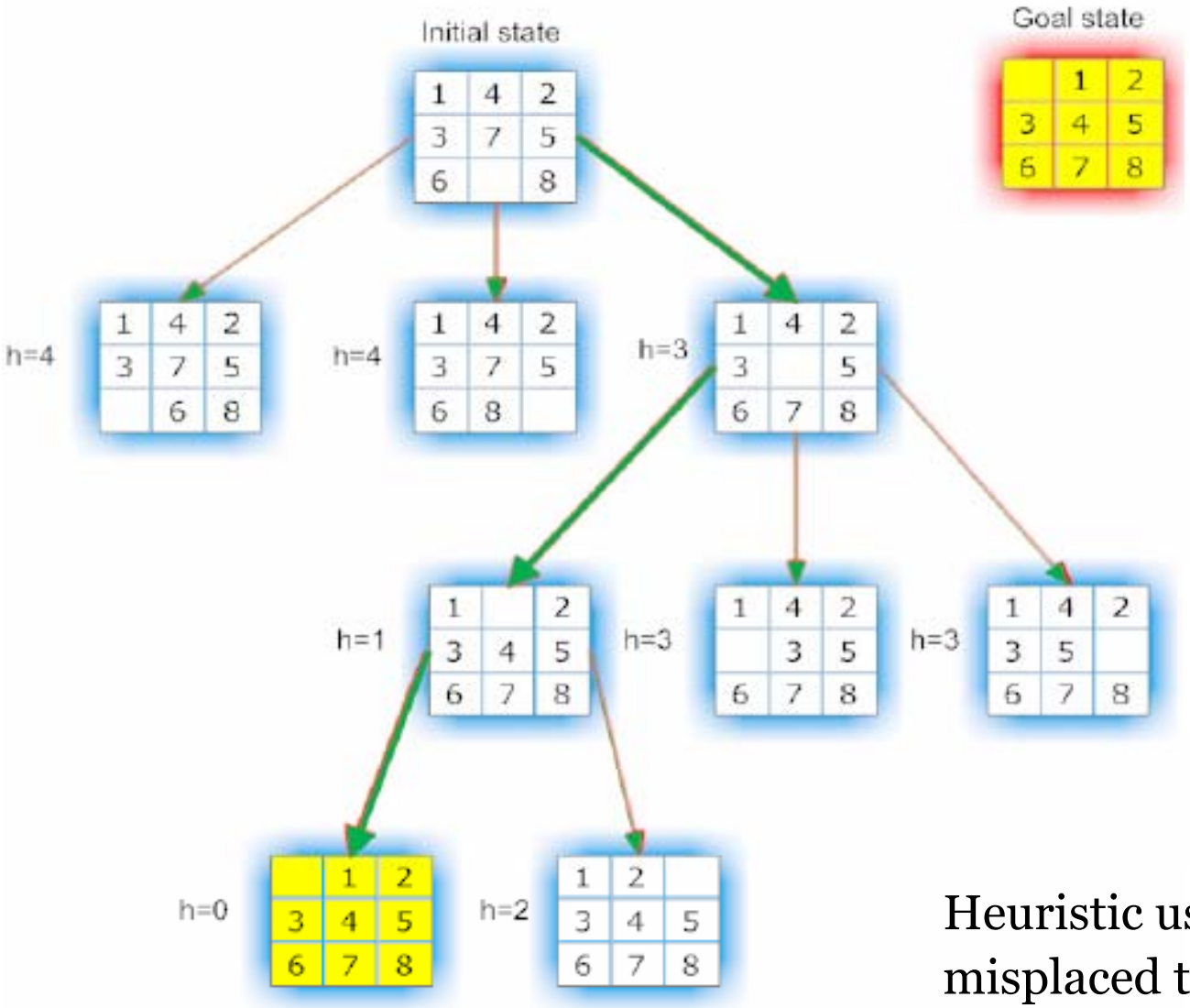
Informed Search

- **Hill Climbing**



Informed Search

- **Hill Climbing: 8-Puzzle Problem**



Heuristic used is the number of misplaced tails, which has to be 0.

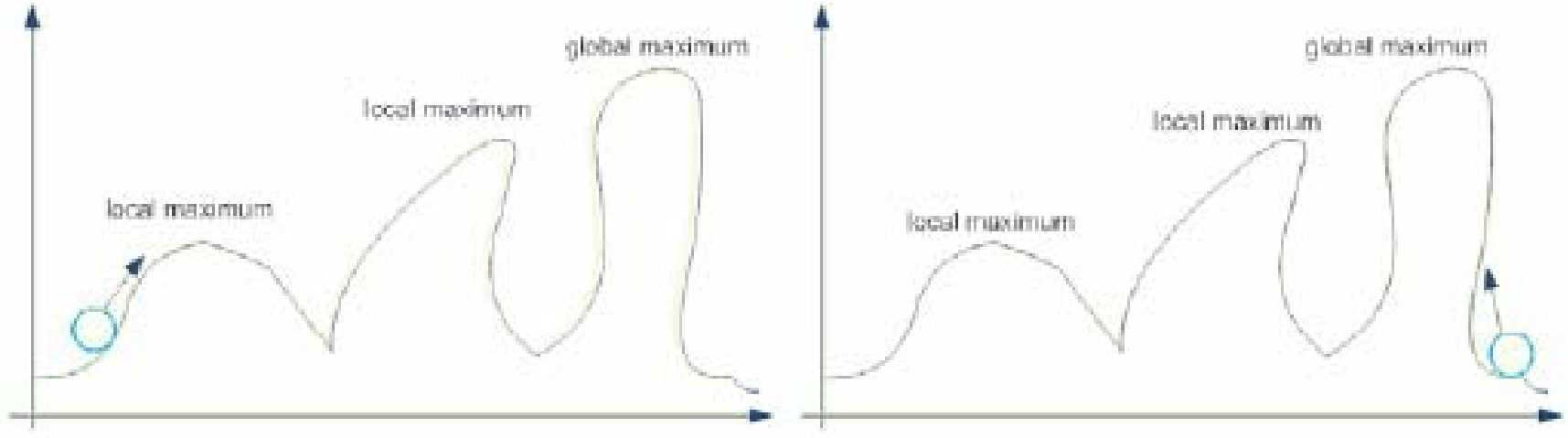
Informed Search

- **Hill Climbing**
  - ◇ Tries to improve the efficiency of **depth-first**.
  - ◇ **Informed depth-first** algorithm.
  - ◇ It sorts the successors of a node (according to their heuristic values) before adding them to the list to be expanded.
  - ◇ Demands very little in the way of memory and computational overheads, as it simply remembers the current successors as the current path it is working on.
  - ◇ It's **non-exhaustive** technique; it does not examine all the tree, so its performance will be reasonably fast.

Informed Search

- **Hill Climbing**

  Depending on the **initial state**, hill-climbing may **get stuck into local optima**. Once it reaches a hill the algorithm will stop since any new successor will be down the hill.

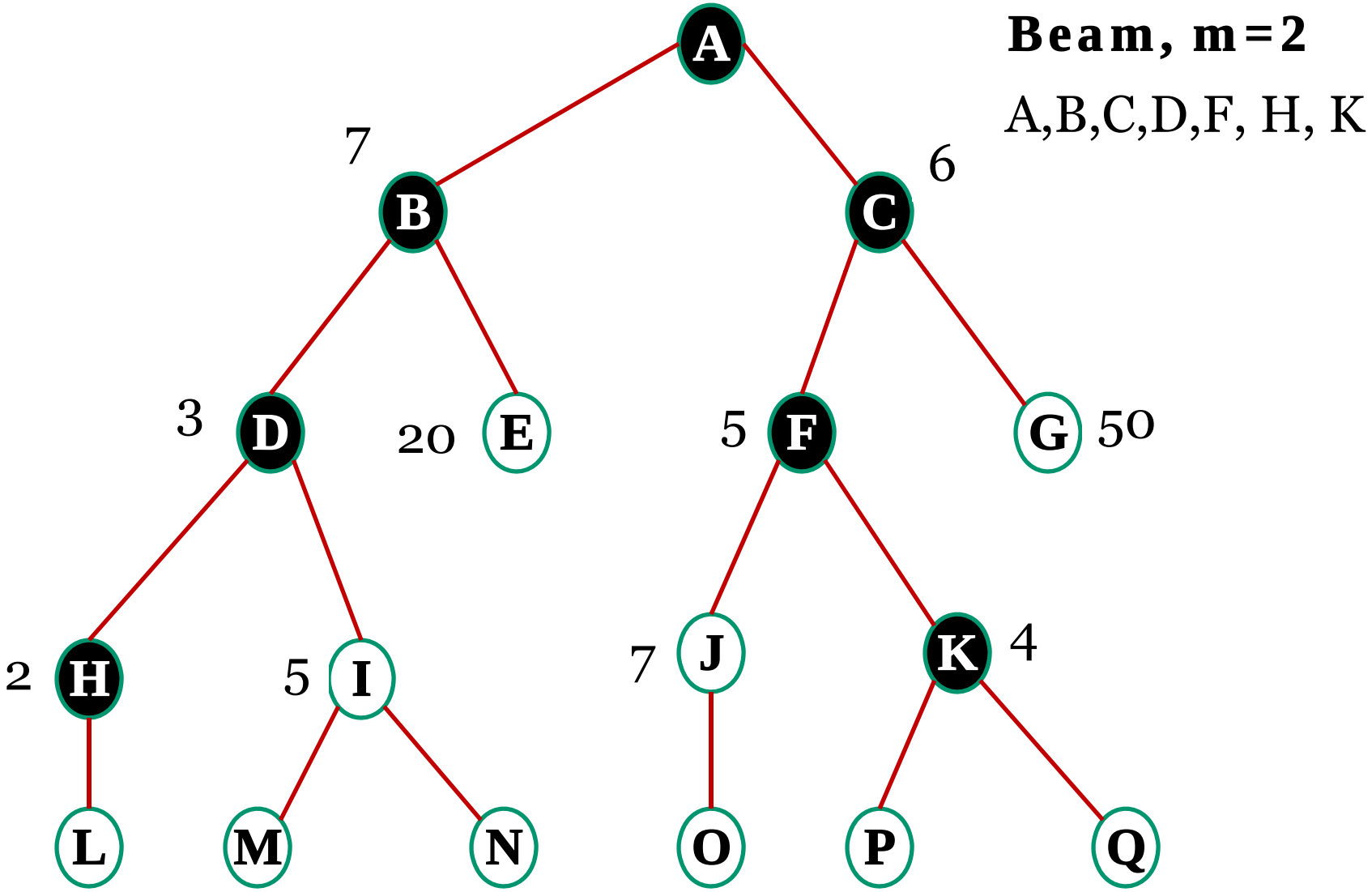


Two different initial states situation; the one on the left will lead the search process to a local optimum (maximum in this case) and the one on the right will reach the global maximum.

[1]

Informed Search

- **Beam Search**



Beam, m=2

A,B,C,D,F, H, K

Informed Search

- **Beam Search**
  - ◇ Tries to minimize the memory requirement of the breadth-first algorithm.
  - ◇ **Informed breadth-first** algorithm.
  - ◇ Expands only the first **m** promising nodes at each level.
  - ◇ It is **non-exhaustive** search, it is a hazardous process because a goal state might be missed.

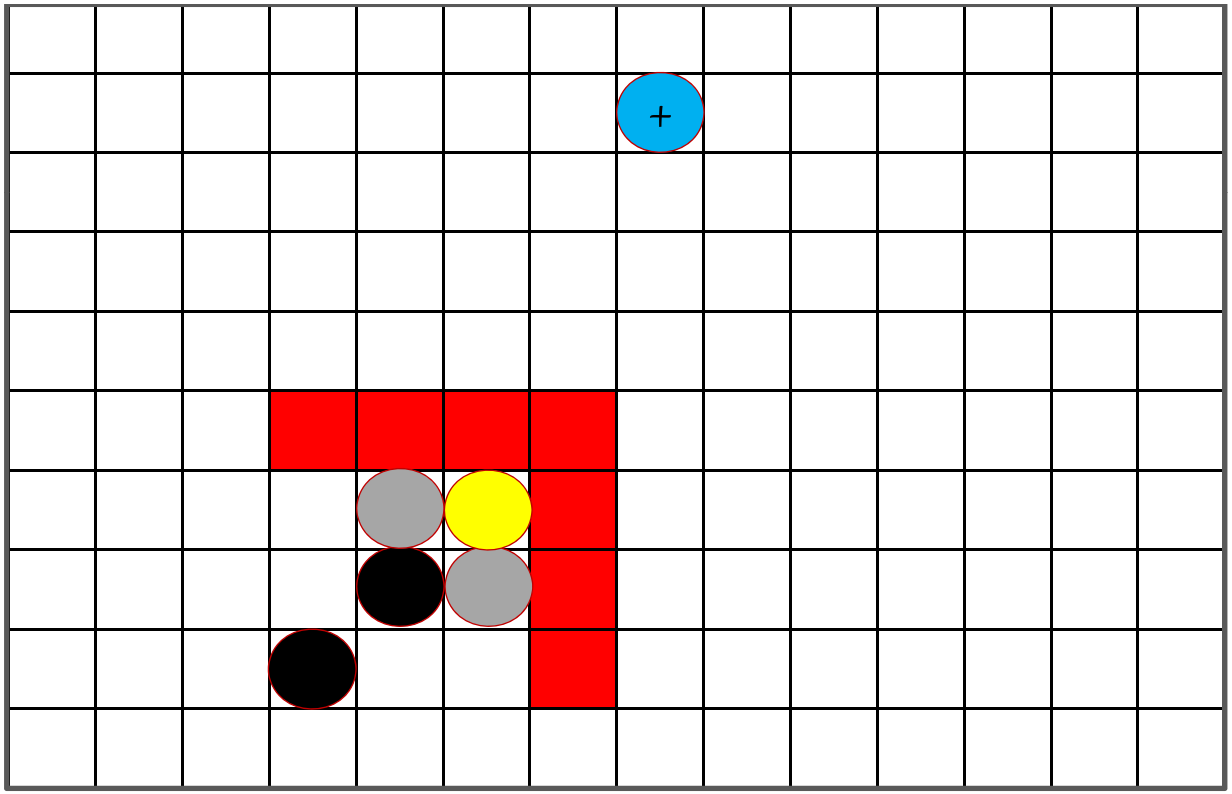
Informed Search

- **Best-first**

1. **Workspace** discretized into cells
2. **Insert** (x_{init},y_{init}) into list **OPEN**
3. **Find** all **8-way neighbors** to (x_{init},y_{init}) that have not been previously visited and insert into OPEN
4. **Sort** neighbors by minimum potential
5. **Form** paths from neighbors to (x_{init},y_{init})
6. **Delete** (x_{init},y_{init}) from OPEN
7. (x_{init},y_{init}) = minPotential(OPEN)
8. **GOTO** 2 until (x_{init},y_{init})=goal (SUCCESS) or OPEN empty (FAILURE)

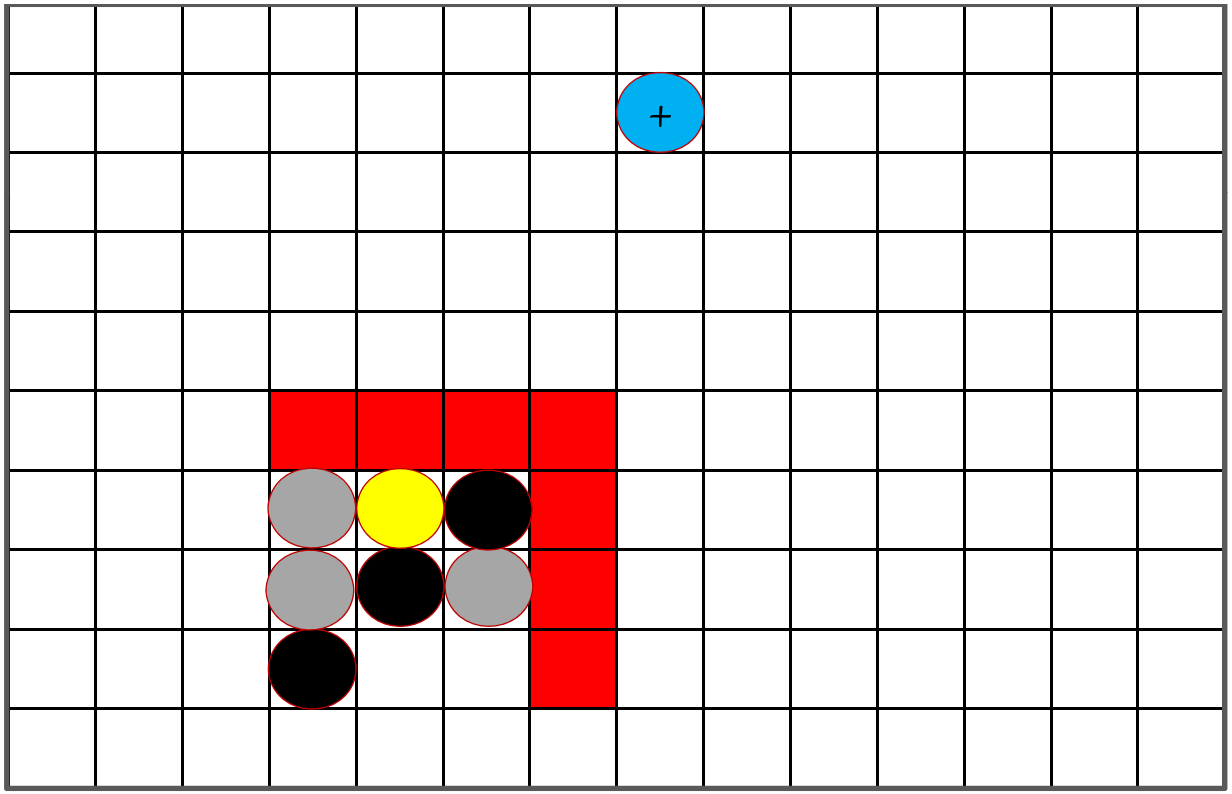
Informed Search

- **Best-first**



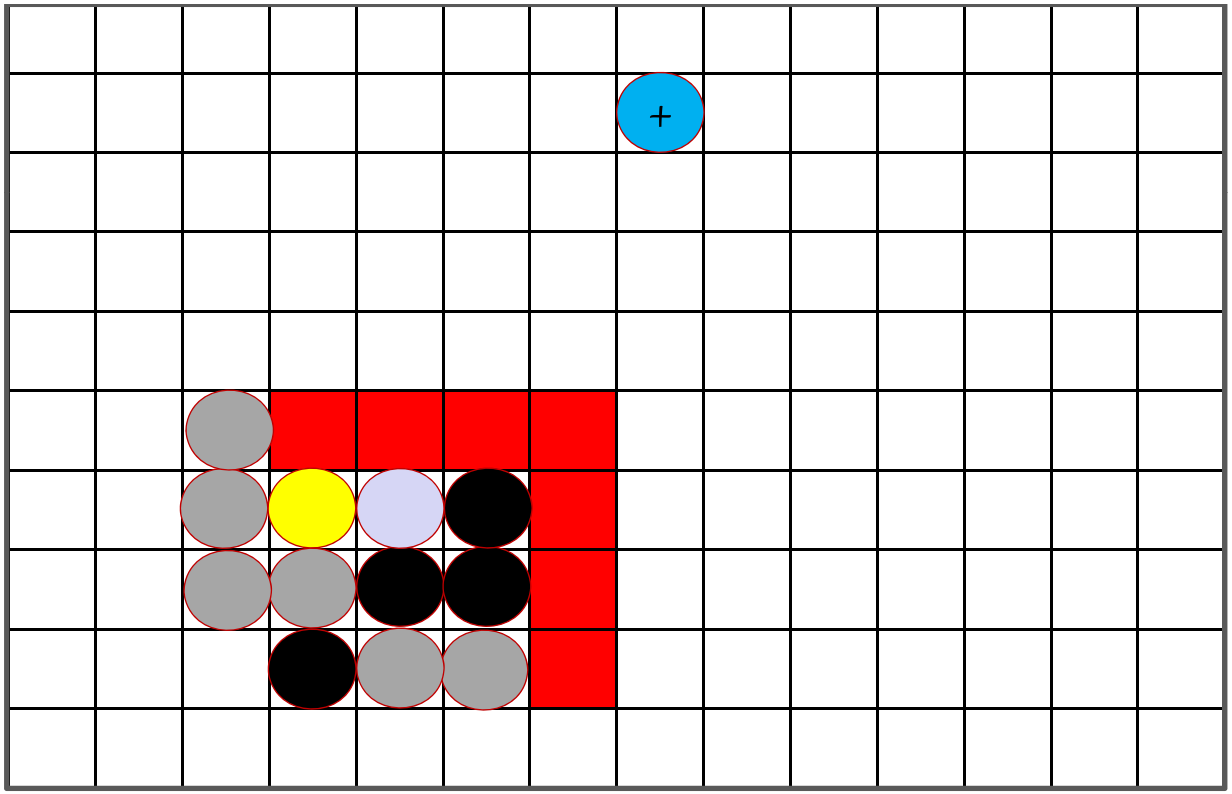
Informed Search

- **Best-first**



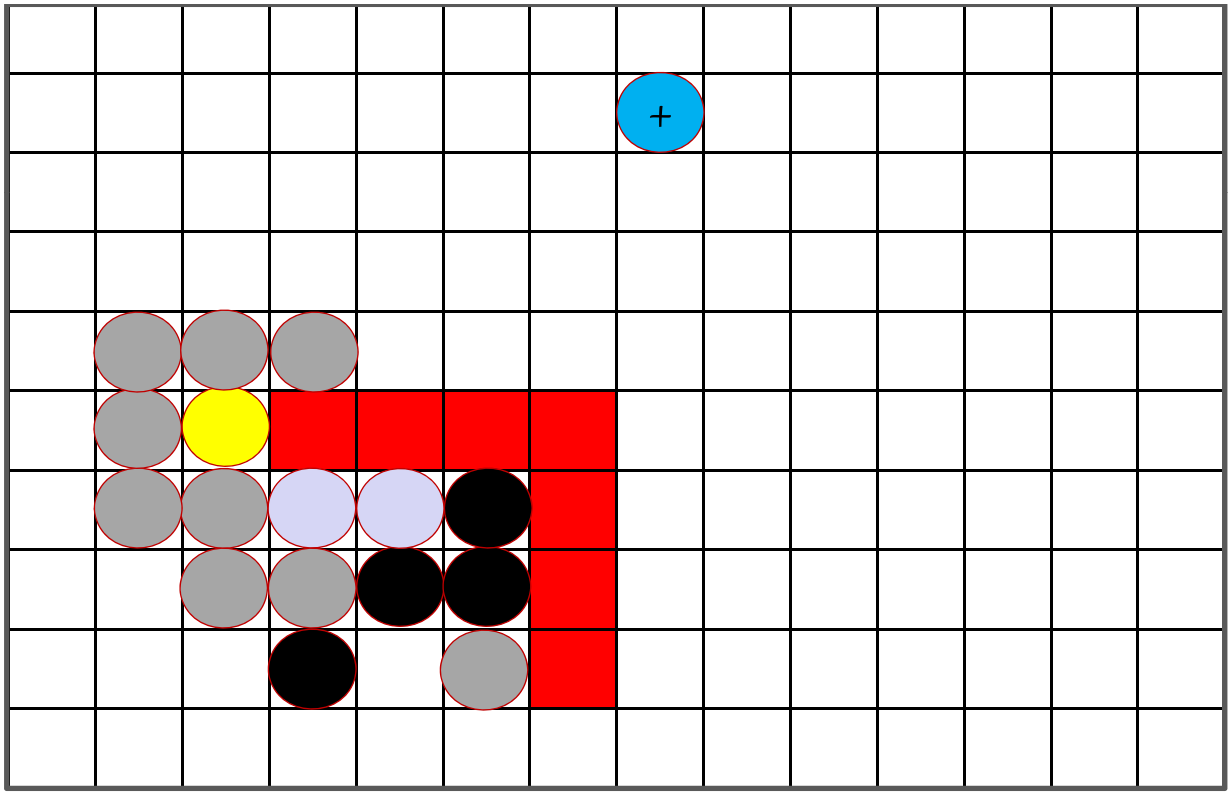
Informed Search

- **Best-first**



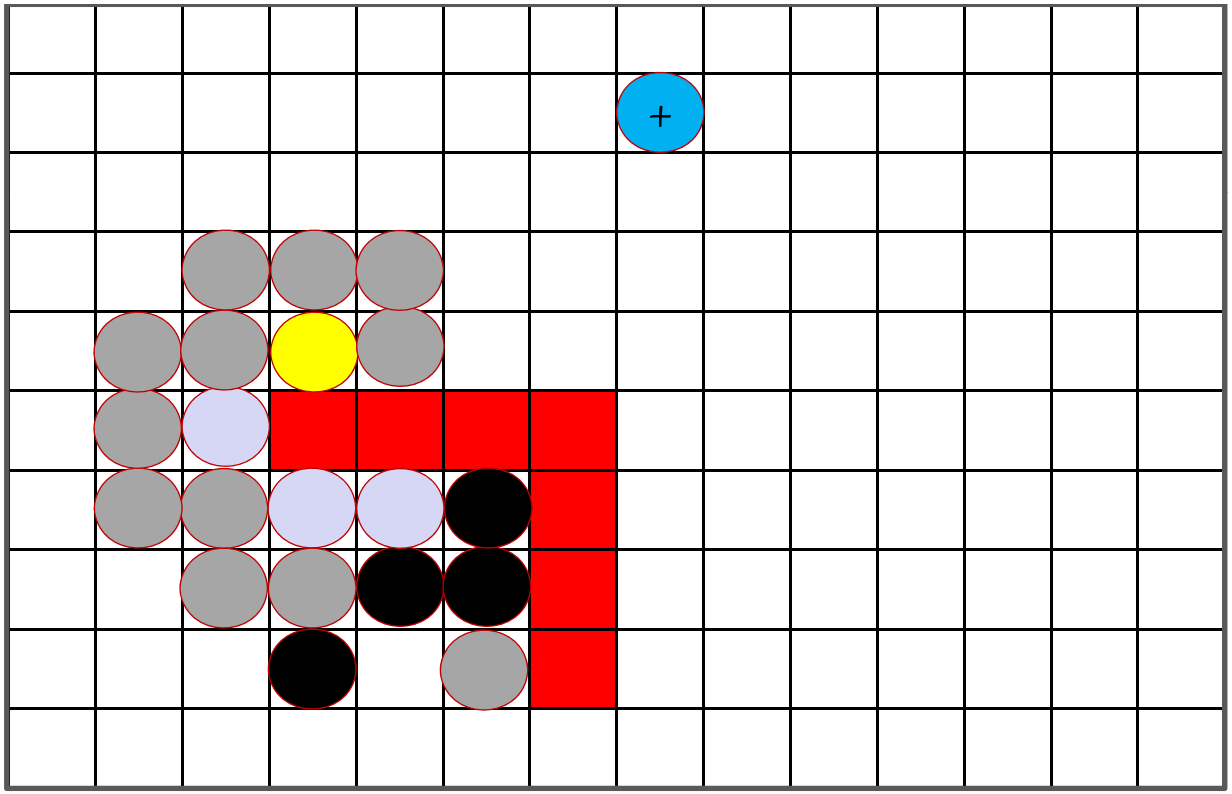
Informed Search

- **Best-first**



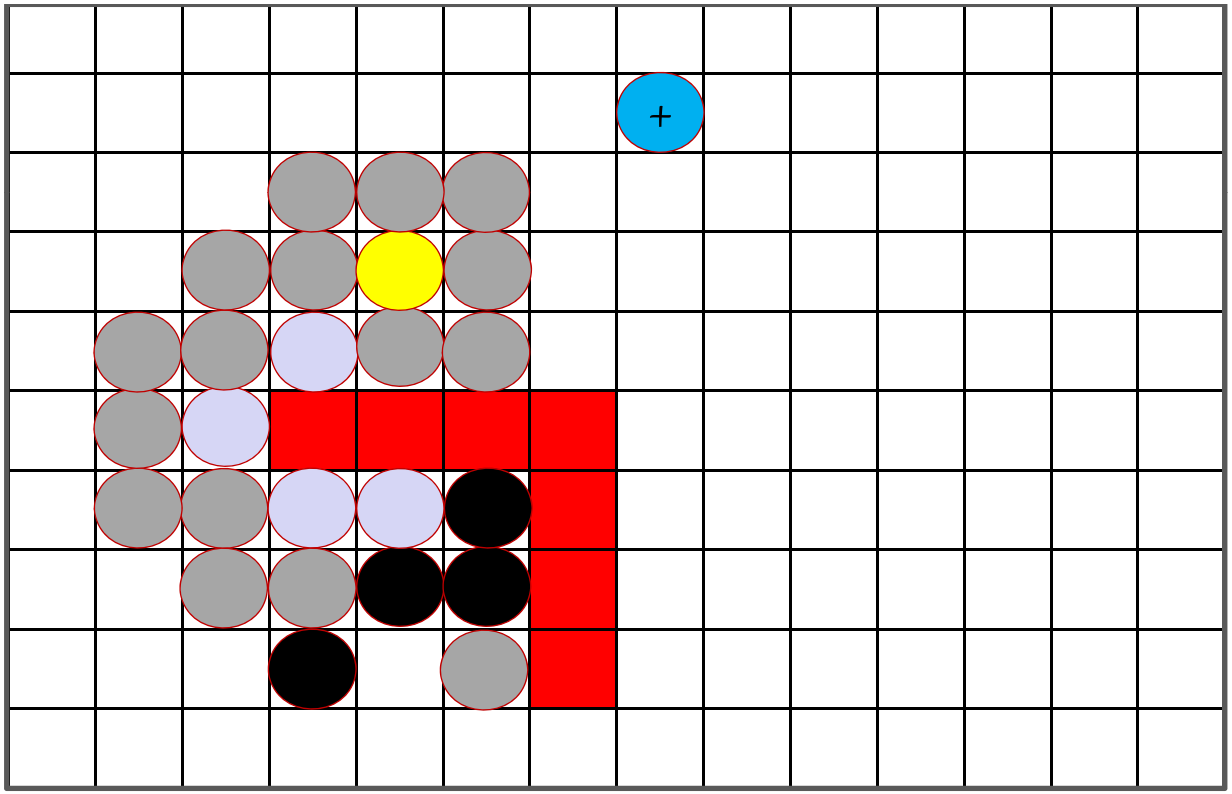
Informed Search

- **Best-first**



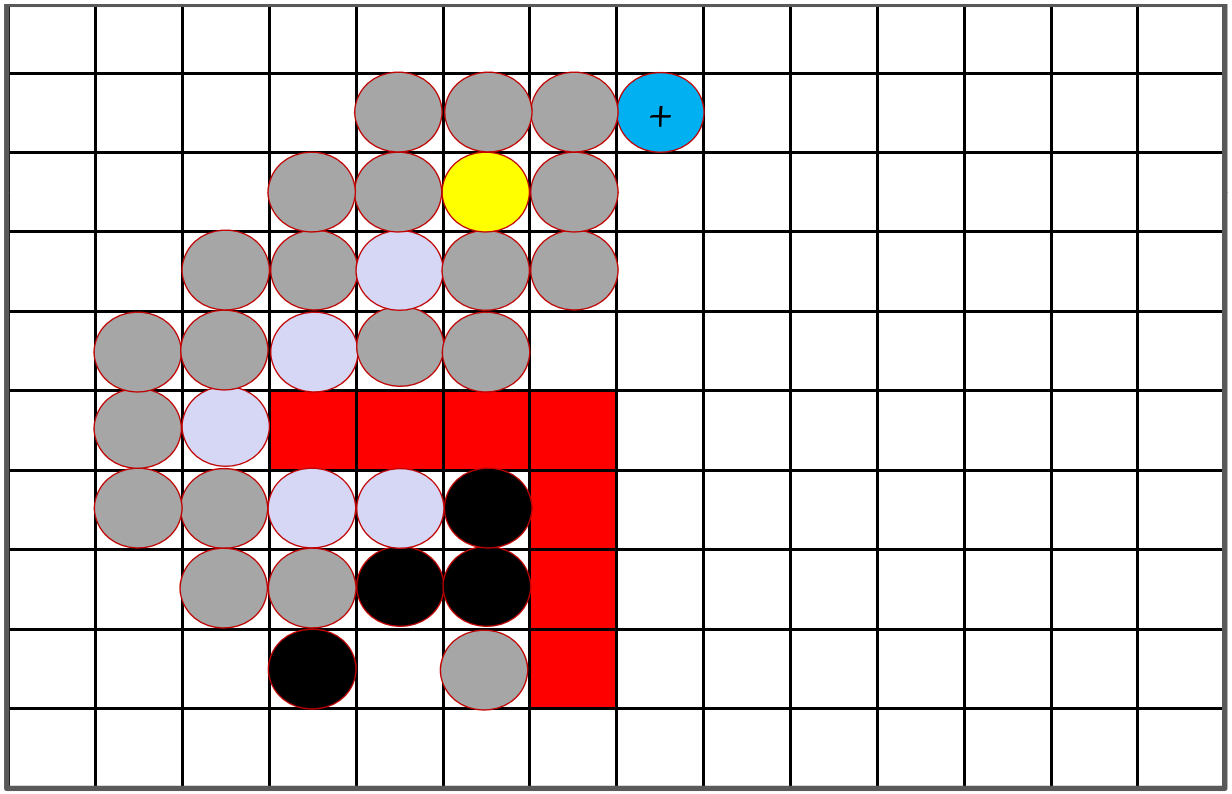
Informed Search

- **Best-first**



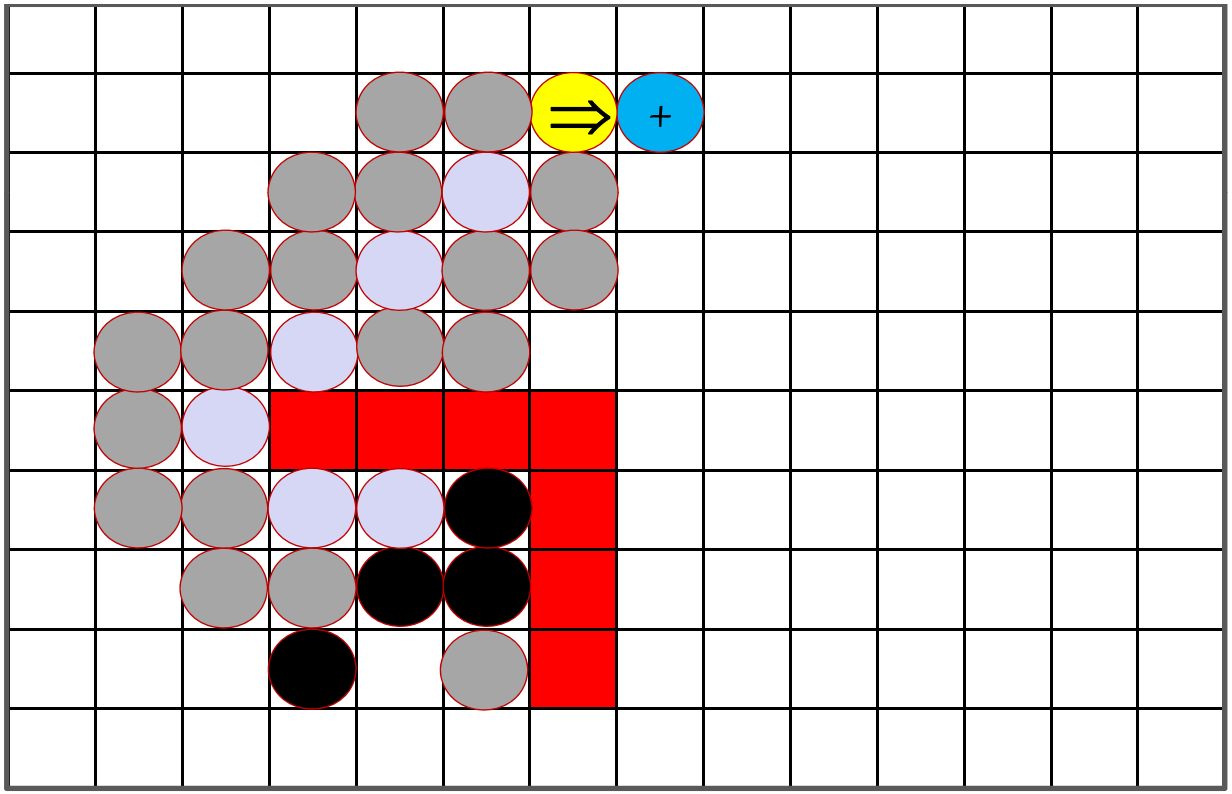
Informed Search

- **Best-first**



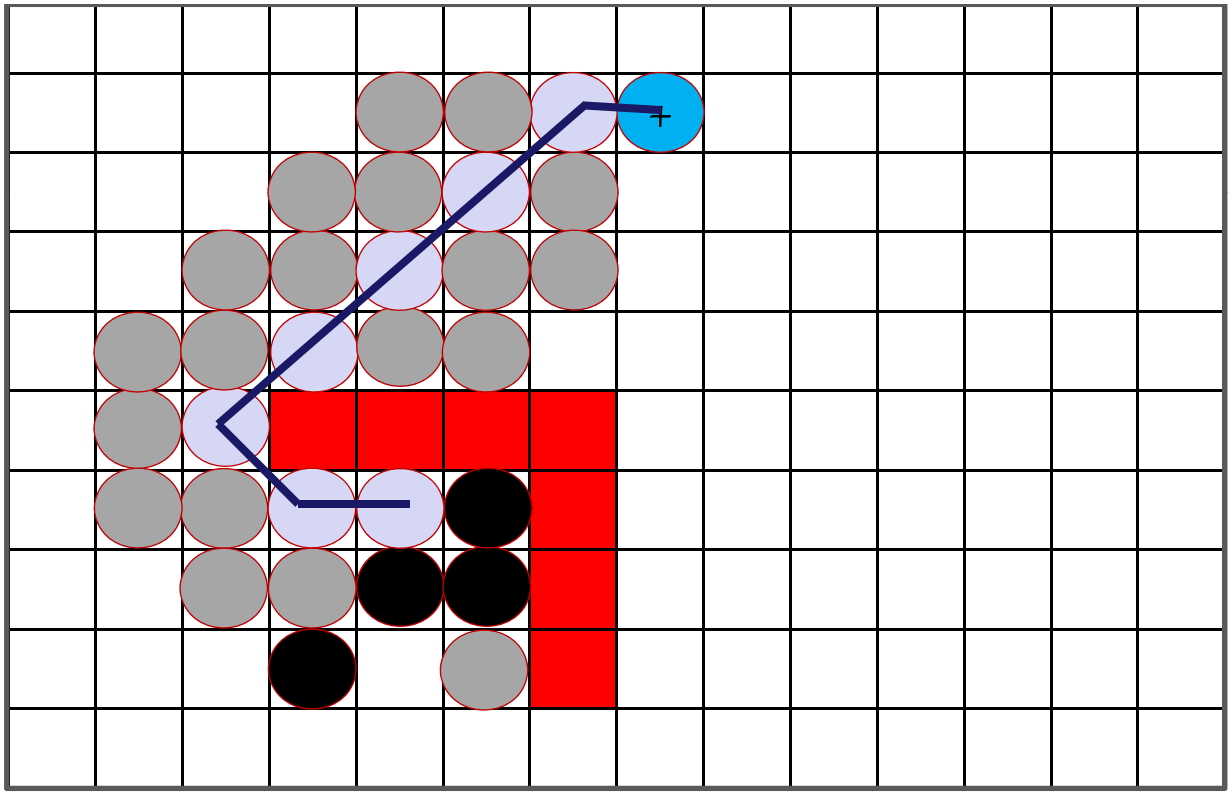
Informed Search

- **Best-first**



Informed Search

- **Best-first**



Informed Search

- **Best-first**
  - ◇ It is a kind or **mixed depth and breadth first** search.
  - ◇ Adds the successors of a node to the expand list.
  - ◇ All nodes on the list are sorted according to the **heuristic values**.
  - ◇ Expand **most desirable unexpanded** node.
  - ◇ Special Case: **A***.

Informed Search

- **A* Algorithm**

Pronounced "**A-Star**"; heuristic algorithm for computing the **shortest path** from one given **start node** to one given **goal node**.

The A^* search algorithm is a variant of dynamic programming (Dijkstra) that tries to **reduce the total number of states explored** by incorporating a **heuristic estimate** of the cost to get the goal from a given state.

Informed Search

- **A* Algorithm**

1. Use a queue to store all the partially expanded paths.
2. Initialize the queue by adding to the queue a zero length path from the root node to nowhere.
3. **Repeat**

   Examine the first path on the queue.

   If it reaches the goal node then success.

   Else {continue search}

   Remove the first path from the queue.

   Expand the last node on this path by one step.

   Calculate the cost of these new paths.

   Add these new paths to the queue.

Informed Search

- **A* Algorithm**

Sort the queue in ascending order according to the sum of the cost of the expanded path and the estimated cost of the remaining path for each path.

If more than one path reaches a subnode

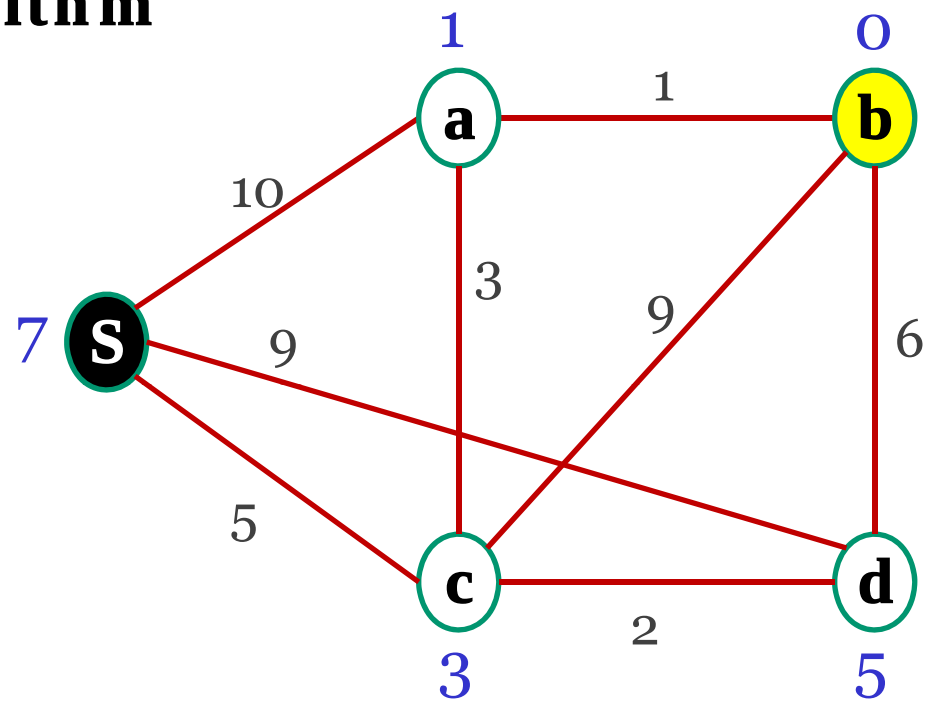
Then delete all but the minimum cost path.

Until the goal has been found or the queue is empty.

4. **If** the goal has been found return success and the path, otherwise return failure.

Informed Search

- **A* Algorithm**

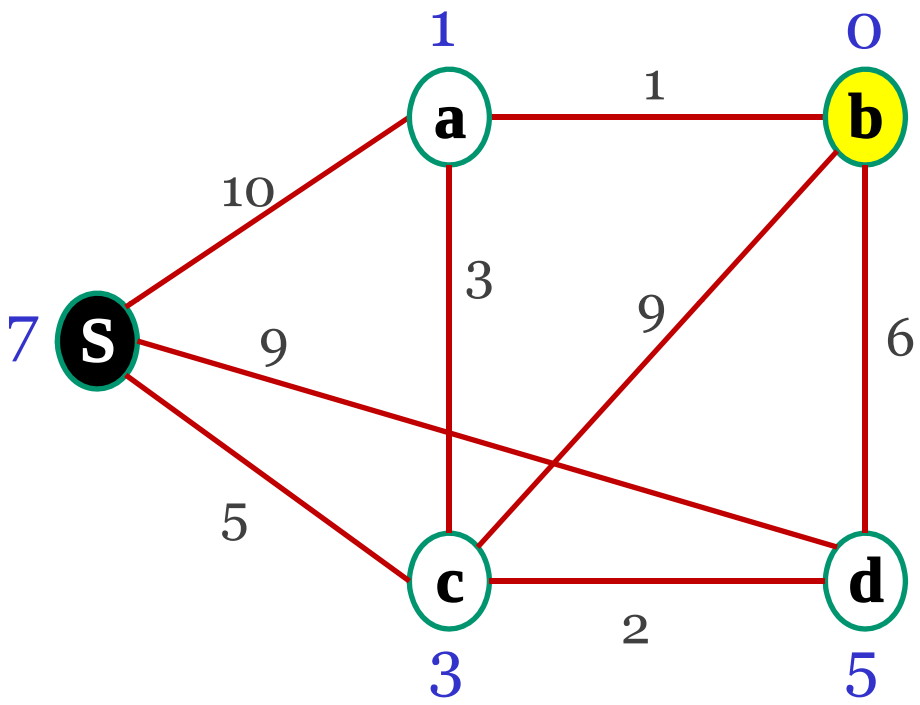


◇ Node values are **lower bound** distances to goal *b* (e.g. linear distances using the Euclidean distance from a global positioning system or the **straight-line distance to the goal position**).

◇ Arc values are distances between neighboring nodes.

Informed Search

- **A* Algorithm**



◇ For the first step, there are three choices:

- {S, a} with min. length

  10 + 1 = 11

- {S, c} with min. length

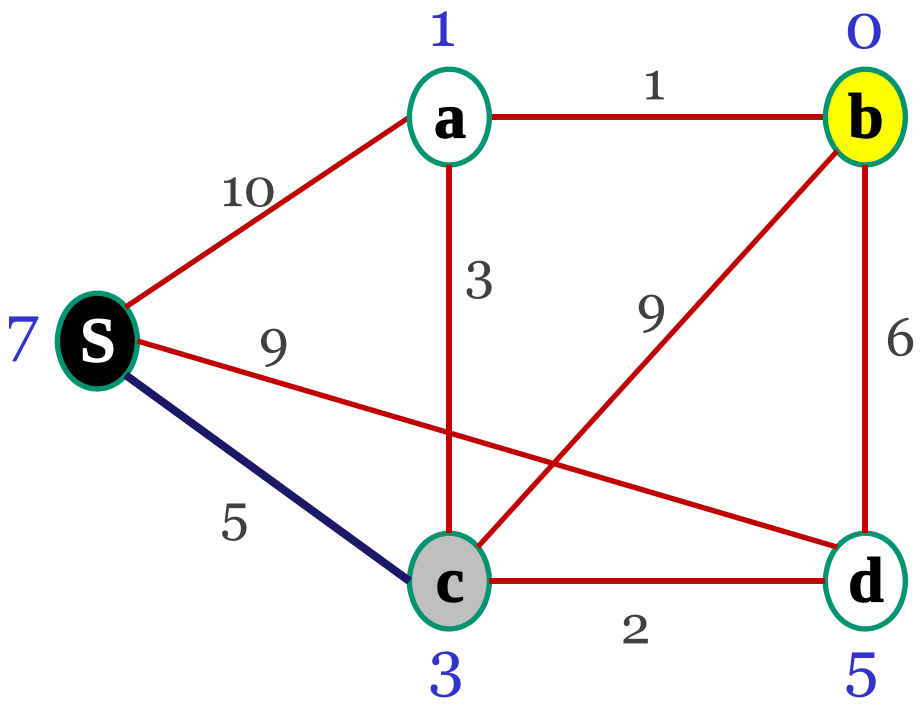
  5 + 3 = **8**

- {S, d} with min. length

  9 + 5 = 14

Informed Search

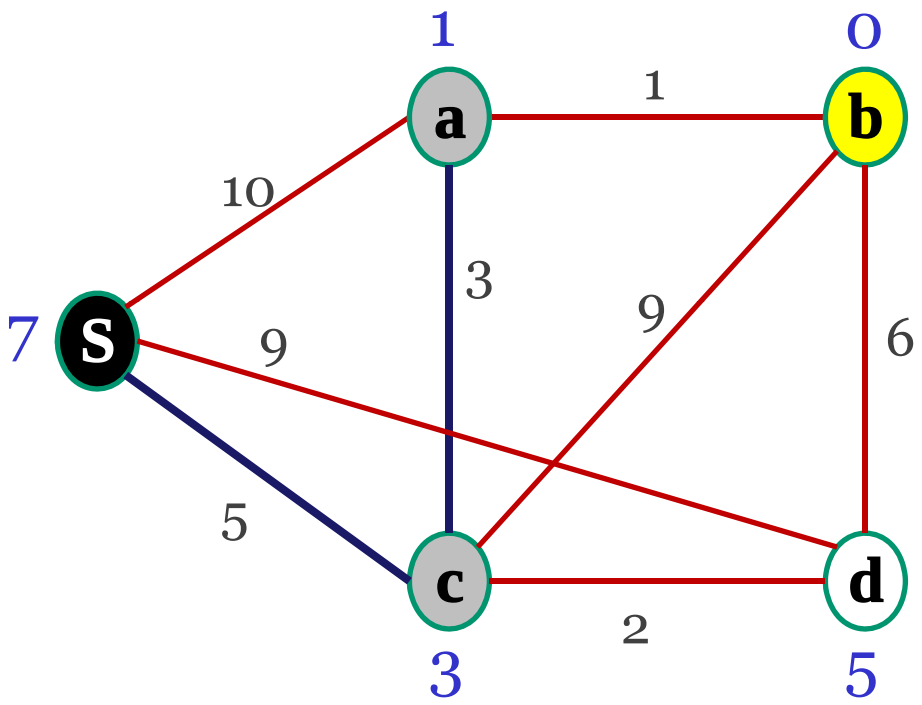
- **A* Algorithm**



◇ Using a "**best-first**" algorithm, we explore the shortest estimated path first: **{S, c}**.

Informed Search

- **A* Algorithm**



◇ Now the next expansion from partial path {S, c} are:

- **{S, c, a}** with min. length

  5 + 3 + 1 = **9**

- {S, c, b} with min. length

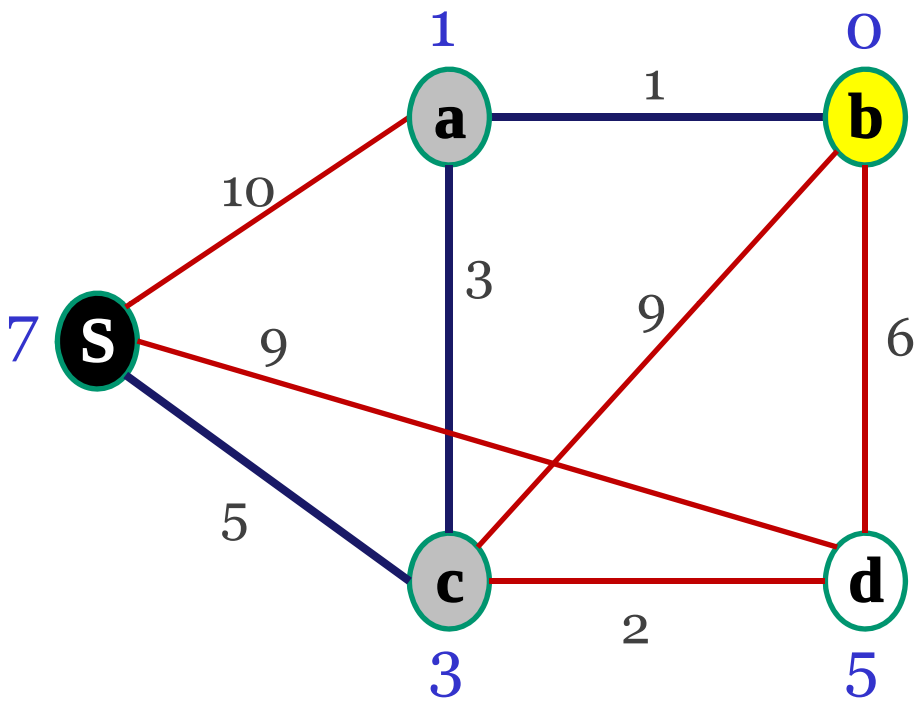
  5 + 9 + 0 = 14

- {S, c, d} with min. length

  5 + 2 + 5 = 12

Informed Search

- **A* Algorithm**



Shortest Path

◇ As it turns out, the currently shortest partial path is

{S, c, a}, which we will now expand further:

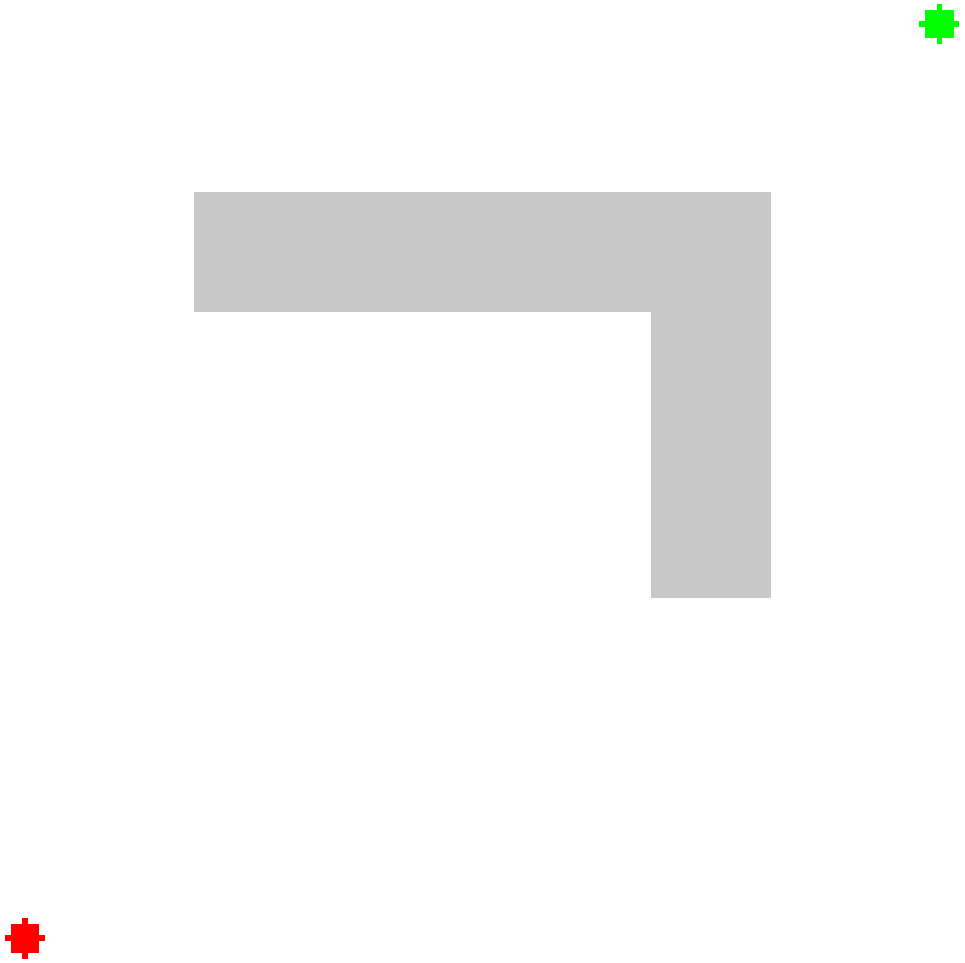
- {S, c, a, b} with min. length

  5 + 3 + 1 + 0 = **9**.

Corresponding distance

Informed Search

- **A* Algorithm**



[4]

Informed Search

- **A* Algorithm**
  - ◇ This algorithm may look complex since there seems to be the need to store incomplete paths and their lengths at various places.
  - ◇ However, using a **recursive best-first search** implementation can solve this problem in an elegant way without the need for explicit path storing.
  - ◇ The quality of the lower bound goal distance from each node greatly influences the timing complexity of the algorithm.
  - ◇ The **closer the given lower bound is to the true distance**, the **shorter the execution time**.

Outline

- Graph Search Methods
- Uninformed/Blind Search
- Informed Search
- **Summary**

Summary

- Uninformed search algorithms such as DFS and BFS are basic brute-force search (also called “naïve” or “blind” search).
- These algorithms work with no information about the search space, other than to distinguish the goal state from all the others.
- Informed search tries to reduce the amount of search that must be done by making intelligent choices for the nodes that are selected for expansion. This implies the existence of some way of evaluating the likelihood that a given node is on the solution path. In general this is done using a heuristic function.

References

1. Crina Grosan and Ajith Abraham. *Intelligent Systems: A Modern Approach*. Springer, 2011.
2. http://dasl.mem.drexel.edu/Hing/BFSDFSTutorial.htm, Last accessed: May 12, 2014.
3. Thomas Braunl. *Embedded Robotics*. Springer, 2006.
4. http://en.wikipedia.org/wiki/File:Astar_progress_animation.gif
5. Kevin Leyton-Brown. CPSC 322 - Introduction to Artificial Intelligence, University of British Columbia, 2009.